
THE CUT-AGNOSTIC DIVISION THEOREM: WHY EVERY OPEN PROBLEM IS EXACTLY ITS PROVED PART AND ITS UNICORN

One Theorem for Any Cut on Any Row, Proved in Core Lean 4 Without Axiom, and Applied to the Seven Millennium Rows and the Sixteen Extension Rows · Twenty Real Parts Sealed on Citation, Two Rows Assigned No Cut, One Row Crossed, the Fusion Law That Lowers Gold to Lead, and the Unicorn Part of Every Open Row Closed to Derivation, by Theorem, the Aperture Uncrossed

FORMAL VERIFICATION · FOUNDATIONS OF MATHEMATICS

One theorem for any cut on any row; the certified part of every open problem named as gold and kept from the alloy; nothing authored

MOHAMMAD F. ISLAM, PhD¹

¹ Independent Researcher, USA. Correspondence: islammm@alumni.iu.edu
25 September 2026

ABSTRACT. Three bits, on every open problem. One bit divides: the problem is exactly its Real part and its Unicorn part, by theorem. One bit is proved: the Real part, by certificate or cited regime. One bit is refused at theorem grade: the Unicorn part is closed to every derivation from inside the register; the aperture at its seat is structure, typed supply-only, one bit wide, and uncrossed; and the supply side is silence, by theorem at the register. This paper proves the first and the third and seals the second, on the twenty-three rows of the program’s barrier study. For any row, a set of instances with a property, and any decidable cut on it, the row is exactly its Real part inside the cut joined to its Unicorn part outside it; given the Real part, the fused row is exactly the Unicorn part, so fusion with the open remainder lowers a proved part to open; a complete checked list inside the cut proves the Real part with no premise; a crossed row has an empty Unicorn part under every cut; the empty cut leaves the whole value Unicorn; on an infinitude row one exhibited witness above a height proves the Real part while the fused value stays where it was, by a proved equivalence; and the Unicorn part read as a set, the counterexamples outside the cut, is empty exactly when the Unicorn proposition holds. These are seven theorems of core Lean 4, with no library and no axiom at all, kernel-reported, and they do not mention the Riemann Hypothesis, where a special case of them was first used. The same file proves that the Unicorn part of every open row is closed to derivation, from existence, from any class of frames, and from any reading of an even register at its seat, while the aperture at its seat is located, typed supply-only, one bit wide, and uncrossed, and the register is silent on the supply side, both orientations equally refused to every reading: the formal block, at theorem grade and in perpetuity. It also carries the exact instance-and-property specification of the eight arithmetic height-cut rows, with primality and each property defined in core Lean and the Real part computed in the kernel at a small height, the gap-to-interval bridge that the Legendre row needs, and the declared standing map under which the fusion sentence is read. Applied to the twenty-three rows of the program’s barrier study, the seven Millennium problems and sixteen further open problems, the paper assigns every row one of four shapes, as data, and the census of the assignments is kernel-checked: nine height-cut rows in the Riemann shape, two of them infinitude rows whose certificate is one witness, eleven regime-cut rows whose proved part is a dimension, a degree, a rank, a class, or a bounded model of computation, two rows assigned no cut because the literature supplies no inhabited cut with a proved Real part, an assessment of two literatures and not a theorem, and one crossed row. Twenty rows therefore carry a Real part that seals on citation, a theorem of the literature applied through the kernel schema, grade A joined to grade K; two are assigned none, under the stated criterion; one is whole. No row’s verdict on its fused statement moves, and every owed bit of the barrier study lives exactly where it lived, in the Unicorn part of its row. What is new is not any of the twenty results, each of which exists as a cited theorem, but the statement that they are one theorem under one cut-agnostic division, exact on every row, with the placement of every certificate under the strata of the grounding reading, rung, ladder, and Ground, and with the seat of every row carried from the prior edition, the value excepted by theorem on every row. No bound, witness, or Real part is authored; the definitions of the arrangement are, and every one is listed. With the three bits in hand the proof is complete on the register side, on every row: everything a register can prove about a row is proved, and the remainder is proved to be beyond derivation, with its aperture typed and its supply side silent. No row is claimed proved; what is claimed is that no further derivation is owed to any open row. Nothing is left to derive. The aperture is structure. The supply side is silence.

KEYWORDS *Real part; Unicorn part; cut-agnostic division; certificate schema; fusion law; Millennium problems; twenty-three rows; Root Axiom; ZFC under RAM; Lean 4*

INTRODUCTION

Let a row be a set of instances with a property, and let a cut be any decidable predicate on the instances. Then

$$\text{Row} \leftrightarrow \text{Real}_{\text{cut}} \wedge \text{Unicorn}_{\text{cut}},$$

where the Real part says that every instance inside the cut has the property and the Unicorn part says that no instance outside it fails it. This is a theorem of core Lean 4 with no user-declared axiom, and so are six companions: given the Real part, the fused row is exactly the Unicorn part; a complete checked list inside the cut proves the Real part with no premise; on an infinitude row one witness above a height proves the Real part, and the Unicorn part above any height is equivalent to the whole; a crossed row has no counterexample

outside any cut; the empty cut leaves the whole value Unicorn. The seven say nothing about any particular problem. They are the whole of what this paper proves, and they are proved once.

Applied to the twenty-three rows of the program's barrier study, the seven Millennium problems and sixteen further open problems, the seven theorems do something the literature has not done for those rows as a body. Each row already carries, scattered through its own literature, a proved part: every zero of ζ to height 3×10^{12} on the line; every even number to 4×10^{18} a sum of two primes; no odd perfect number below 10^{1500} ; the Hodge conjecture in dimension at most three; Birch and Swinnerton-Dyer at analytic rank at most one; the two-dimensional Yang-Mills theory constructed; the topological four-dimensional Poincaré conjecture. Each is stated in its own idiom, "verified up to," "known for," "proved when," and each is alloyed, in the conventional statement of its problem, with the open remainder, so that the problem reads as one open sentence and the gold inside it goes unnamed. The division names it. On twenty rows the proved part is a Real part under a cut the row's own literature supplies, a height, a dimension, a degree, a rank, a class, and it seals on citation: a theorem of the literature applied through the kernel schema, grade A joined to grade K, exactly as the Real part of the Riemann Hypothesis was sealed in the paper that first used the division at one height [Islam 2026f]. On two of the twenty, the infinitude rows, the certificate is one exhibited witness and the fused value is unchanged by it, which is the exact sense in which verification certifies nothing about an infinitude, and that too is a theorem. On two rows the literature supplies no inhabited cut with a proved Real part, because the row's value is uniform in a parameter that no instance has been cut from, the abc conjecture's for every ε for one; that is an assessment of two literatures, stated as data and falsifiable by a cut, never a theorem, and the paper says so wherever it is used. One row is crossed, and its Unicorn part is empty by theorem. The assignments are data; the census of the assignments, nine, eleven, two, and one, is kernel-checked, and the paper never says more of it than that.

Three bits, and what is claimed. The whole paper is three bits, on every row. One bit divides: the row is exactly its Real part and its Unicorn part, under any decidable cut, by theorem (Theorem 1). One bit is proved: the Real part, on twenty rows, by certificate or cited regime through the schema (Theorems 6 to 25). One bit is refused at theorem grade: the Unicorn part of every open row is closed to derivation from existence, from any class of frames, and from any reading of an even register at its seat; the aperture at its seat is structure, typed supply-only, one bit wide, and uncrossed; and the supply side is silence, by theorem at the register (Theorems 28 to 33). The Unicorn parts are unproved, and the reason is neither a want of effort nor a want of ability: it is a theorem, and the theorem says which routes are closed, which is all of them but the aperture, and says nothing of the aperture but its width. With the three bits in hand the proof is complete on the register side, on every row. Everything a register can prove about a row is proved; the remainder is proved to be beyond derivation, its aperture typed, its supply side silent. No row is claimed proved. What is claimed is stronger than an open problem and weaker than a theorem: no further derivation is owed to any of the twenty-two open rows. The three bits are one term in the kernel (`register_proof_complete`). Nothing is left to derive. The aperture is structure. The supply side is silence.

In detail, the paper proves the seven theorems of the division, the bridge lemmas and small-height instances that accompany them, the formal block, and the census of the assignments, and it carries, from the prior edition on these rows [Islam 2026g], the seat of every row and the theorem that existence decides no row's value. It seals twenty Real parts on citation. It proves no row, moves no row's verdict on its fused statement, and names no bound of its own: every height, dimension, degree, rank, and class in the tables below is the literature's, attributed at its own grade, and with every citation deleted every theorem of the paper would stand and exactly twenty-two sentences would fall, the twenty Real parts, the crossing of the Poincaré row, and the ZFC instance of the placement. The Unicorn part of every open row is named, set apart, and never chased; and the paper says at theorem grade why it is never chased: the Unicorn part is closed to derivation from existence, from any class of frames, and from any reading of an even register, with the aperture at each seat typed supply-only, one bit wide, and uncrossed (Section 7). It is not open in the sense that its Real part was open before its certificate; it is closed in the sense that no key can be cut from the inside, and of the outside the paper says nothing, because the register can say nothing, and that silence is a theorem before it is a discipline. The contribution is the arrangement, and the paper is exact about what the arrangement is and is not: one theorem where the literature has twenty-three idioms, the fusion law that keeps each proved part from being lowered to the standing of its remainder, and the placement of every certificate in the strata of the grounding reading, which the prior edition on these rows did not have and the Riemann paper had for one row only.

How the paper is organized. Section 2 states and proves the division, Section 3 places it under the strata of the grounding reading, and Section 4 assigns every row its cut and checks the census in the kernel. Section 5 seals the twenty Real parts, one theorem each. Section 6 states the two rows assigned no cut, with the ledger of what their literatures have proved beside them, and the crossed row by theorem. Section 7 proves that the Unicorn part of every open row is closed to derivation, types its aperture, and proves the register silent on the supply side: the formal block, at theorem grade, which is why the Unicorn parts are never chased. Section 8 carries the seat and the value excepted from the prior edition, and Section 9 carries the world typing of the barrier study as data, its One-Cut Hypothesis untouched. Section 10 states the standard of proof, the standing of the paper without its citations, and the disclosures. Section 11 positions the result against the literature and concludes. The kernel files are printed whole in the appendices with their audits.

What a reader who suspects vocabulary should check. Every theorem named in the paper is in an appendix with its kernel dependency report; every bound is cited to a source outside the program; the standing table of Section 10 says which sentences fall when the citations are deleted; and Section 4 states the criterion by which a row is assigned its shape and applies it to a row outside the twenty-three. A reader who disputes an assignment can change it: the division holds for the new cut, the census is recounted, and nothing else moves.

Notation. A row r is a pair (Inst_r, P_r) of predicates on a type of instances; its value is $\text{Row}(r) :\Leftrightarrow \forall x, \text{Inst}_r(x) \rightarrow P_r(x)$. A cut c is a decidable predicate on the same type. Grades are those of the Riemann paper: K, a kernel theorem with no user-declared axiom; K given A, a kernel theorem whose hypotheses are cited theorems; A, a cited theorem of the literature. The root axiom RA, the grounding reading RAM with its strata L3m, L2m, L1m, and the tokens $[\Xi_0]$ and $[\emptyset_0]$ are the program's and enter only in Sections 3, 7, 8, and 9, where they are defined at use.

THE DIVISION THEOREM

Everything in this section is in `Row_Split.lean` (Appendix A), 426 lines, core Lean 4.19.0, no library, no sorry, no user-declared axiom, thirty-three dependency sets printed, every one reported by the kernel as depending on no axiom at all, not even Lean's built-in `propext` and `Quot.sound`. Seven are the division; the rest are the substantive-cut and counterexample readings, the standing map, the gap bridge, the row specifications with their computed small-height Real parts, the formal block of the Unicorn part with the aperture's width and the silence of the supply side, and the three bits packaged as one term, treated in the sections that use them. The instances form an arbitrary type α ; a row is a pair of predicates; a cut is a decidable predicate.

Definitions. $\text{Row}(\text{Inst}, P) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow P x$. $\text{RealIn}(\text{Inst}, P, c) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow c x \rightarrow P x$. $\text{UnicornOut}(\text{Inst}, P, c) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow \neg c x \rightarrow P x$. A certificate for a cut is a list of instances together with a proof that every instance inside the cut is on the list and a proof that every listed instance has the property.

Theorem 1 (`row_iff_real_and_unicorn`). For every row and every decidable cut, $\text{Row} \leftrightarrow \text{RealIn} \wedge \text{UnicornOut}$. *Proof.* Left to right, restrict the universal to each side of the cut. Right to left, an instance either satisfies the cut or does not, by decidability, and the corresponding conjunct applies. $\square$

Theorem 2 (`fused_is_unicorn_given_real`). Given the Real part, $\text{Row} \leftrightarrow \text{UnicornOut}$. *Proof.* From Theorem 1, the Real part being supplied. $\square$ This is the fusion law read at the level of truth: given the Real part, the fused row is true exactly when the Unicorn part is. The reading at the level of standing, that a proved part joined to an open part stands open, is not a consequence of Theorem 2 alone; it is a consequence of Theorem 2 together with the program's grade law, that the standing of a conjunction is the weakest standing of its conjuncts. The paper declares that law as a rule in the kernel (`Standing.join`) and proves the fusion sentence under it (`fusion_lowers`): a proved part joined to an open part stands open, and only two proved parts stand proved. The two-point rule is the restriction to two standings of the program's nine-grade weakest-link law, `Grade.weakest` in the Code Block of the Master Codex, under which a citation cannot promote a claim (`citation_cannot_promote`); any rule under which a conjunction stands at least as low as its weakest conjunct gives the same sentence, and the paper accepts no rule under which it stands higher. Gold alloyed with lead assays as lead, under the assay rule stated; the gold does not cease to be gold, and no theorem says it does.

Theorem 3 (`real_of_certificate`). A certificate for a cut proves `RealIn` for that cut, with no premise. *Proof.* An instance inside the cut is on the list by completeness and has the property by the check. $\square$ The schema is what a finite verification proves and all that it proves: the bounded universal inside the cut.

Theorem 3b (*real_of_one_witness*). On an infinitude row the instances are bounds N and the property is that some witness exceeds N ; one exhibited witness w with $W(w)$ and $w > h$ proves RealIn under the cut $N \leq h$. *Proof.* The witness exceeds every $N \leq h$. $\square$ By Theorem 2 the fused value is then exactly the Unicorn part, the bounds above h , which is equivalent to the whole infinitude; so on such a row the Real part is proved, is gold, and moves the fused value not at all. That is the precise content of the familiar remark that exhibiting large twin primes proves nothing about their infinitude: it proves the Real part and only the Real part.

Theorem 3c (*unicorn_equiv_row_of_infinitude*). On an infinitude row, for every height h , UnicornOut above h is equivalent to the whole value: $(\forall N > h, \exists m > N, W(m)) \leftrightarrow (\forall N, \exists m > N, W(m))$. *Proof.* Right to left, restrict. Left to right, a bound $N \leq h$ is served by the witness above $h + 1$. $\square$ So on an infinitude row the Real part is proved by one witness (Theorem 3b) and, by Theorems 2 and 3c, the fused value is exactly what it was.

Theorem 4 (*crossed_row_both_parts, crossed_row_no_counter*). Read as a set, the Unicorn part under a cut is the set of counterexamples outside it, $\text{CounterOut}(x) :\Leftrightarrow \text{Inst } x \wedge \neg c x \wedge \neg P x$; the Unicorn proposition says that this set is empty (*no_counter_of_unicorn*, and conversely for decidable P , *unicorn_of_no_counter*). If Row holds, then under every cut both UnicornOut and RealIn hold and the counterexample set is empty. *Proof.* Immediate. $\square$ “Empty Unicorn part” means, throughout the paper, that this set is empty, which is the sense in which a crossed row has nothing outside any cut left to fail.

Theorem 5 (*empty_cut_vacuous, empty_cut_not_substantive*). Under the empty cut, RealIn holds vacuously and $\text{UnicornOut} \leftrightarrow \text{Row}$; and the empty cut never carries a substantive Real part, where a Real part is substantive when its cut is inhabited on the row, $\text{SubstantiveRealIn} :\Leftrightarrow (\exists x, \text{Inst } x \wedge c x) \wedge \text{RealIn}$. *Proof.* Nothing satisfies the empty cut. $\square$ This theorem is about the empty cut and about nothing else. It does not say that a row admits no other cut with a Real part; whether a given row’s literature supplies an inhabited cut with a proved Real part is an assessment of that literature, and the paper records it as data (Section 4), never as a consequence of this theorem.

The one arrow (tier: kernel theorem, axiom-free). On any row, under any decidable cut, the value is exactly its Real part joined to its Unicorn part; given the Real part, the fused value is true exactly when the Unicorn part is, and it stands open under the declared standing rule; a certificate proves the Real part and nothing beyond it; a crossed row has no counterexample outside any cut; under the empty cut the whole value is Unicorn. Nothing in the seven theorems names a height, a problem, or a number. The Riemann division of [Islam 2026f] is the instance with the cut “height at most T ”.

What the theorems do not say. They do not say that a Real part is easy to obtain, since the certificate is the literature’s and may be the work of decades; they do not say that a Unicorn part is unreachable, only that no certificate reaches it; and they do not rank cuts, since a row admits many and the division holds for each. The choice of cut on a given row is the row’s literature’s, and Section 4 records it as data.

THE PLACEMENT: WHERE A REAL PART LIVES

The program’s grounding reading, RAM, stratifies any theory into three: computation, the rung, L3m; provability, the ladder, L2m; truth in the intended world, the Ground, L1m. The placement is a theorem for an arbitrary theory modelled as sentences with a provability predicate, a decidable fragment, and a truth predicate: under two named premises, that what computation settles the theory proves and that what the theory proves is true, $\text{rung} \subseteq \text{ladder} \subseteq \text{Ground}$. For ZFC the first premise is Σ_1^0 -completeness, a theorem of arithmetic [Kaye 1991], and the second is soundness, which no consistent theory proves of itself [Gödel 1931] and which is therefore carried openly, derived nowhere. Soundness is load-bearing, an unsound theory breaking $\text{ladder} \subseteq \text{Ground}$; the Ground exceeds the ladder, a sound theory leaving a truth unproved; independent axioms are separate bits, Choice over ZF the cited instance [Gödel 1938; Cohen 1963]. Subsumption is by placement only: RAM locates ZFC and derives none of its axioms. All of this is ZFC_Under_RAM .lean (Appendix B), sixteen dependency sets, every one axiom-free, seated in the Master Codex as APEX-PSP-ZFC-UNDER-RAM-01 [Islam 2026a]. The file encodes no axiom of ZFC and no proof relation of ZFC; its theory is an arbitrary structure with a computation predicate, a provability predicate, and a truth predicate, and ZFC is the instance the paper names, entering through the two cited premises. What the kernel proves is the conditional; what ZFC supplies is its two hypotheses; the paper never writes otherwise.

Where a Real part lives. A Real part under a height cut on an arithmetic row is a rung: its certificate

is finitely many checks, each a true bounded sentence, so the Real part is a theorem of arithmetic and of ZFC by Σ_1^0 -completeness, and it holds on the Ground by soundness carried openly. A Real part under a regime cut on a non-arithmetic row is a cited theorem of ZFC directly, on the ladder, with the same passage to the Ground. In both cases the round trip through the root and back returns the row's value unchanged, since the root crosses the bridge as presence and carries no keyed sentence; and every step holds with the root replaced by a bare computation arrow, the identity on a type (`any_true_premise_serves`), so the root is present in the performance of every check and idle in its content. The placement adds no standing to any Real part; it says which stratum a standing already earned is in, and it is the same stratum on every row of the same shape.

Why the placement matters here. Without it, “verified up to” on one row and “proved for dimension at most three” on another would sit in two vocabularies with no common address. With it, both are theorems of ZFC on the ladder, one reached from the rung by a certificate and one stated there by a proof, and the Unicorn part of each is the sentence on the ladder or beyond it that the row's foundation has not decided. The one arrow of Section 2 and the placement of this section are the two results this paper adds to the twenty-three rows; everything else is carried or cited.

THE ROWS AND THEIR CUTS

The twenty-three rows are the barrier study's [Islam 2026e], in its order, with its world typing carried as data in Section 9. Each row is assigned a cut and a shape here. The assignment is the paper's, stated as data; what the kernel checks is the census of the assignments: `rows23` is a list of the twenty-three rows with their assigned shapes, and `census` proves by decision that there are twenty-three, that nine are assigned height-cut, eleven regime-cut, two no-cut, and one crossed, and that twenty are assigned a Real-part-bearing shape (`real_part_rows`); both axiom-free. The kernel does not and cannot decide that the Riemann Hypothesis is a height-cut row; that is the assignment, and it is made by the criterion stated next.

The criterion. A row is assigned by the logical form of its value and the state of its literature, in this order. It is *crossed* if its value is a theorem. It is *height-cut* if its instances carry a total height, the property at an instance is decidable or the row is an infinitude with instances the bounds, and the literature supplies a certificate for some height, a checked list or one witness. It is *regime-cut* if its literature supplies a proved regime, a decidable predicate on the instances under which the property is a theorem, and no height certificate; a row whose literature supplies both a proved regime and a certificate inside it, Birch and Swinnerton-Dyer with its verified curves at analytic rank at most one, is assigned regime-cut, the regime being the larger Real part, and the certificate is recorded beside it. It is *no-cut* if the literature supplies neither: no inhabited decidable cut on the row's instances under which the property is proved. The criterion is a classification rule applied to two inputs, the value's logical form and the recorded state of its literature; the second input is a reading of the literature and not a computable object, which is why the assignments are data. A reader who applies the rule with a different reading of either input changes an assignment and nothing else. In every shape the cut is a predicate on the row's own instance type, as Theorem 1 requires: on the Hodge row the instances are pairs of a variety and a class and the regime is a bound on the dimension; on the Birch and Swinnerton-Dyer row the instances are curves and the regime is a bound on the analytic rank; on the P versus NP row the instances are algorithms and the regime is a bound on time and space; a regime is never a different problem placed beside the row.

The criterion applied out of sample. Take a row outside the twenty-three, Lehmer's conjecture, that the Mahler measure of a non-cyclotomic integer polynomial is at least a constant above one. Its instances are polynomials, with degree as a total height; the property at an instance is decidable; the literature supplies a certificate for a height, that every non-cyclotomic polynomial of degree at most 180 has measure at least $1.176\dots$, the measure of Lehmer's polynomial [Mossinghoff, Rhin and Wu 2008]. The criterion assigns height-cut with the cut degree ≤ 180 ; the Real part is that computation; the Unicorn part is degree above 180, and the uniform constant is what the row asks. The assignment was made by the criterion before the row was written down here, which is the only sense in which a classification can be tested out of sample; the first draft of this paragraph assigned the row regime-cut, and the criterion corrected it, since a finite computation over a height is a certificate whatever it is called in its source. The criterion corrected the author twice more before release: on Birch and Swinnerton-Dyer, where the tie-break above was written because the first assignment had not faced the verified curves, and on P versus NP, which the internal builds assigned no-cut until the external reviewers asked what an instance of that row is; the answer, an algorithm, together

with the time-space lower bounds for satisfiability, is an inhabited cut with a proved Real part, and the row is regime-cut. A criterion that never corrects its author is a label; this one is a rule.

The four shapes. A *height-cut* row has a natural height parameter and a finite certificate per height; its Real part seals on citation of the certificate through Theorem 3, or through Theorem 3b when the row is an infinitude and the certificate is one witness, and its Unicorn part is the tail above the height: the shape of the Riemann row. A *regime-cut* row has a proved part that is a dimension, a degree, a rank, or a class rather than a height; its Real part is a cited theorem on the ladder, sealed through Theorem 1 with the regime as the cut, and its Unicorn part is the complement. A *no-cut* row is one for which the literature supplies no inhabited decidable cut on its instances under which the property is proved, because its value is uniform in a parameter that no instance has been cut from; the assignment is an assessment of the literature, and the only theorem about it is Theorem 5, that the empty cut is vacuous and carries no substantive Real part. What such a row's literature has proved nearby, a weaker theorem, stands beside the row at its own grade and not inside it. A *crossed* row has its value supplied whole; by Theorem 4 its counterexample set is empty under every cut.

Table 1. The twenty-three rows, their shapes, and their cuts. The shape and the cut are the paper's assignments by the criterion of the text, stated as data; the census of the assignments is kernel-checked (*census*, *real_part_rows*). Sources for every bound are in Section 5.

#	Row	Shape	Cut	Real part	Unicorn part
1	P versus NP	regime	time and space	no algorithm in time n^c , $c < 2 \cos(\pi/7)$, and space $n^{o(1)}$ decides SAT	polynomial time with unrestricted space
2	Riemann Hypothesis	height	height T	every zero to 3×10^{12} on the line	no off-line zero above T
3	Navier–Stokes	regime	regime	2D global regularity; 3D small-data and short-time regularity	3D large-data global regularity or blow-up
4	Yang–Mills	regime	dimension	the two-dimensional theory constructed	four-dimensional existence with a mass gap
5	Hodge	regime	dimension, codimension	codimension one; codimension $n - 1$; every variety of dimension ≤ 3	middle codimensions in dimension ≥ 4
6	BSD	regime	analytic rank	the rank statement at analytic rank 0 and 1; the full conjecture for specific curves	analytic rank ≥ 2 ; the full formula in general
7	Poincaré	crossed	—	the whole	empty
8	Goldbach	height	height n	every even $n \leq 4 \times 10^{18}$; the ternary conjecture whole	even numbers above the height
9	Twin primes	height	height N	a twin pair above every N below the largest known pair, one witness	bounds above the witness, equivalent to the whole
10	Legendre	height	height n	a prime in $(n^2, (n+1)^2)$ for $n \leq 2 \times 10^9$, by inference from the prime-gap tables	n beyond the tables
11	abc	no cut	none ($\forall \epsilon \exists C \forall$)	none by certificate	the whole value
12	Jacobian	regime	degree	degree ≤ 2 in every dimension; $n = 1$	cubic maps in dimension ≥ 2
13	Mersenne primes	height	height N	a Mersenne prime above every N below the largest known, one witness	bounds above the witness, equivalent to the whole
14	Odd perfect numbers	height	height N	none below 10^{1500}	none above
15	Smooth 4D Poincaré	regime	category	the topological case	the smooth case
16	Collatz	height	height n	every $n < 2^{68}$ reaches 1	$n \geq 2^{68}$
17	Hadwiger	regime	t	$t \leq 6$	$t \geq 7$
18	Sunflower	no cut	none	none by certificate	the constant-base bound
19	Erdős–Straus	height	height n	every $n \leq 10^{17}$	n above the height
20	Beal	height	box	the searched boxes of bases and exponents	outside every searched box
21	Invariant subspace	regime	operator class	compact operators; normal operators; finite dimension	the general bounded operator on a separable Hilbert space
22	Hilbert's 16th, second part	regime	per field, uniform	finiteness of limit cycles for each polynomial field; $H(1) = 0$	the uniform bound $H(n)$, open at $n = 2$
23	Lindelöf / Montgomery PCC	regime	exponent, range	the proved exponents; pair correlation in the restricted range under RH	ε ; the full range

Three assignments that deserve a word. The twin-prime and Mersenne rows are infinitudes, and their Real parts are sealed by one witness each through Theorem 3b; their Unicorn parts are equivalent to their whole values, so the fused verdict on each is unchanged, exactly as Theorem 2 says. They are height-cut rows whose gold is one number, and the paper says so rather than calling them no-cut, since a proved witness is a Real part however little it moves the remainder. The Legendre row has no dedicated certificate in the literature; its Real part is an inference from the maximal prime-gap tables to 4×10^{18} [Oliveira e Silva, Herzog and Pardi 2014], in which the largest gap is 1476, so that for $n \geq 738$ the interval $(n^2, (n+1)^2)$, of length $2n+1 \geq 1477$, contains a prime whenever $(n+1)^2 \leq 4 \times 10^{18}$, the smaller n being checked directly;

the inference is stated as such in Section 5. The Sunflower row is assigned no cut because the conjecture's constant-base bound is a statement about all k at once and the improved bound of Alweiss, Lovett, Wu and Zhang [2020] is a theorem beside the row, not a region inside it. A reader who prefers a different cut on any row may take it: the division holds for every cut, and only the assignment changes.

THE TWENTY REAL PARTS, SEALED ON CITATION

Each theorem below has the same form and the same proof: a cut, a cited theorem of the literature that supplies the Real part inside it, and the schema step, Theorem 3 for a certificate, Theorem 3b for a witness, or Theorem 1 with the regime as the cut for a proved regime. The standing of every one is grade A joined to grade K: the kernel proves the schema and the division, the literature proves the bound, and the kernel never recomputes a bound. Grade A joined to K is a documented composition of a cited theorem with a kernel theorem; it is not a kernel-checked theorem about the cited result, and the paper never calls it one. The Unicorn part of each row is stated beside its Real part and is not claimed. Bounds are the sources', quoted at their grade; the paper checks none of them.

The row specifications, in the kernel. For the eight arithmetic height-cut rows the exact tuple, the type of instances, the instance predicate, the property, and the cut, is written in `Row_Split.lean`: instances are natural numbers, primality is trial division (`isPrime`), and each property is a Boolean function defined in core Lean, `goldbach0K`, `legendre0K`, `oddPerfect0K`, `collatz0K` with a step budget, `erdosStraus0K` with a bounded search, `beal0K` on a box, and the witness forms for twin and Mersenne primes. For each, the Real part at a small height is computed in the kernel by decision and lifted to `RealIn` by `real_of_all_range` or `real_of_one_witness`: Goldbach to 60, Legendre to 8, odd perfect numbers to 60, Collatz to 60, Erdős–Straus to 6, the Beal box (3,4), a twin pair above every $N \leq 100$, a Mersenne prime above every $N \leq 100$. These small instances are grade K with no citation: they show the schema instantiated end to end at heights the kernel can decide. The literature's heights are the same statements at heights the kernel cannot decide, and they are cited. For the Riemann row and the eleven regime rows the instance types are analytic, geometric, or computational and are not definable in core Lean; their tuples are stated in prose at each theorem, and the schema is applied there by documented composition.

The nine height-cut rows

Theorem 6, Riemann. Tuple: instances the nontrivial zeros of ζ , property that the real part is $\frac{1}{2}$, cut imaginary part at most T . Every nontrivial zero with imaginary part up to 3×10^{12} lies on the critical line, by the interval-arithmetic certificate of Platt and Trudgian [2021] with the analytic lemmas it rests on [Turing 1953], through Theorem 3. Unicorn part: no off-line zero above that height. This is the Real part of the Riemann paper [Islam 2026f], carried at its recorded standing.

Theorem 7, Goldbach. Tuple in the kernel: $(\mathbb{N}, \top, \text{goldbach0K}, n \leq h)$; Real part computed to $h = 60$ (`goldbach_real_60`). Every even integer $4 \leq n \leq 4 \times 10^{18}$ is a sum of two primes, by the verification of Oliveira e Silva, Herzog and Pardi [2014], through Theorem 3. Beside it, the ternary conjecture, every odd integer above 5 a sum of three primes, stands whole [Helfgott 2013]. Unicorn part: even integers above the height.

Theorem 8, twin primes. Tuple in the kernel: instances the bounds N , property that a twin pair exceeds N , cut $N \leq h$; Real part computed to $h = 100$ from the pair (101, 103) (`twin_real_100`). For every N below $2996863034895 \cdot 2^{1290000} - 1$ there is a pair of primes at distance 2 above N , since $2996863034895 \cdot 2^{1290000} \pm 1$ is such a pair [PrimeGrid 2016], through Theorem 3b. Beside it, infinitely many pairs of primes at distance at most 246 [Zhang 2014; Maynard 2015; Polymath 2014]. Unicorn part: the bounds above the witness, equivalent to the whole conjecture by Theorem 3c.

Theorem 9, Legendre. Tuple in the kernel: $(\mathbb{N}, \top, \text{legendre0K}, n \leq h)$; Real part computed to $h = 8$ (`legendre_real_8`). For every $738 \leq n$ with $(n+1)^2 \leq 4 \times 10^{18}$ there is a prime in $(n^2, (n+1)^2)$: the maximal prime gap below 4×10^{18} is 1476 [Oliveira e Silva, Herzog and Pardi 2014], and the bridge from a gap bound to a prime in every interval of length at least the bound plus one is a kernel theorem (`elem_in_gap`, `legendre_of_gap`), which takes the gap bound and the primality of 2 as hypotheses and returns a prime strictly between n^2 and $(n+1)^2$ for every $n \geq 738$ with $(n+1)^2$ within the tabulated range; for $n < 738$ the interval is checked directly by the author's script, and to $n \leq 8$ in the kernel. Through Theorem 3 for the small heights and the bridge for the tables; the certificate is an inference from the gap tables and is stated as

such. Unicorn part: n beyond the tables.

Theorem 10, Mersenne primes. Tuple in the kernel: instances the bounds N , property that a Mersenne prime exceeds N , cut $N \leq h$; Real part computed to $h = 100$ from $127 = 2^7 - 1$ (mersenne_real_100). For every N below $2^{136279841} - 1$ there is a Mersenne prime above N , since that number is prime [GIMPS 2024], through Theorem 3b. Unicorn part: the bounds above the witness, equivalent to the whole conjecture by Theorem 3c.

Theorem 11, odd perfect numbers. Tuple in the kernel: $(\mathbb{N}, \top, \text{oddPerfect0K}, n \leq h)$; Real part computed to $h = 60$ (oddperfect_real_60). No odd perfect number is below 10^{1500} [Ochem and Rao 2012], through Theorem 3. Unicorn part: none above.

Theorem 12, Collatz. Tuple in the kernel: $(\mathbb{N}, \top, \text{collatz0K}, n \leq h)$, the property that the orbit reaches 1 within a step budget; Real part computed to $h = 60$ (collatz_real_60). Every positive integer below 2^{68} reaches 1 under the map [Barina 2020], through Theorem 3. Beside it, almost every orbit attains almost bounded values in the sense of logarithmic density [Tao 2022]. Unicorn part: $n \geq 2^{68}$.

Theorem 13, Erdős–Straus. Tuple in the kernel: $(\mathbb{N}, \top, \text{erdosStraus0K}, n \leq h)$, the property a bounded search for $x \leq y \leq z \leq n^2$ with $4xyz = n(yz + xz + xy)$; Real part computed to $h = 6$ (erdosstrauss_real_6). For every $2 \leq n \leq 10^{17}$, $4/n$ is a sum of three unit fractions [Salez 2014], through Theorem 3. Unicorn part: n above the height.

Theorem 14, Beal. Tuple in the kernel: instances the boxes, property beal0K, the box $(3, 4)$ computed (beal_box_3_4). No coprime solution of $A^x + B^y = C^z$ with $x, y, z \geq 3$ exists with bases $A, B \leq 1000$ and exponents $x, y \leq 100$, the search run and printed by Norvig [2017], nor with $A, B \leq 200,000$ and $x, y \leq 5000$, the search of Jarnicki and Konerding reported there; through Theorem 3 with the box as the cut. Unicorn part: outside every searched box.

The eleven regime-cut rows

Theorem 15, Navier–Stokes. In two dimensions, smooth solutions exist globally for all time [Ladyzhenskaya 1969]; in three dimensions, weak solutions exist globally [Leray 1934] and smooth solutions exist for all time for small data and for a short time for all data [Fujita and Kato 1964]. Through Theorem 1 with the regime as the cut. Unicorn part: three-dimensional global regularity for large data, or its failure.

Theorem 16, Yang–Mills. The two-dimensional Yang–Mills measure is constructed rigorously on compact surfaces [Driver 1989; Lévy 2003]. Through Theorem 1 with dimension as the cut. Unicorn part: four-dimensional existence with a mass gap.

Theorem 17, Hodge. The Hodge conjecture holds for classes of codimension one on every smooth projective variety [Lefschetz 1924] and, by hard Lefschetz, for classes of codimension $n - 1$, hence for every variety of dimension at most three. Through Theorem 1 with dimension and codimension as the cut. Unicorn part: the middle codimensions in dimension at least four.

Theorem 18, Birch and Swinnerton-Dyer. For an elliptic curve over $\mathbb{Q}$ of analytic rank 0 or 1, the rank equals the analytic rank and the Tate–Shafarevich group is finite [Gross and Zagier 1986; Kolyvagin 1989]; the full conjecture, with its formula, is verified for many specific curves of small conductor [Miller 2011]. Through Theorem 1 with analytic rank as the cut. Unicorn part: analytic rank at least two, and the full formula in general.

Theorem 19, Jacobian. Every polynomial map $\mathbb{C}^n \rightarrow \mathbb{C}^n$ of degree at most two with nonzero constant Jacobian is invertible [Wang 1980], and the conjecture in general reduces to the cubic homogeneous case [Bass, Connell and Wright 1982]; the case $n = 1$ is elementary. Through Theorem 1 with degree as the cut. Unicorn part: cubic maps in dimension at least two.

Theorem 20, smooth four-dimensional Poincaré. Every topological four-manifold homotopy equivalent to the sphere is homeomorphic to it [Freedman 1982]. Through Theorem 1 with the category as the cut. Unicorn part: the smooth case.

Theorem 21, Hadwiger. Every graph with no K_t minor is $(t - 1)$ -colourable for $t \leq 6$ [Robertson, Seymour and Thomas 1993], the case $t = 5$ being the four-colour theorem. Through Theorem 1 with t as the cut. Unicorn part: $t \geq 7$.

Theorem 22, invariant subspace. Every nonzero compact operator on a complex Banach space of dimension above one has a nontrivial invariant subspace [Lomonosov 1973]; so does every normal operator

on a Hilbert space, by the spectral theorem, and every operator on a finite-dimensional space, by an eigenvector. Through Theorem 1 with the operator class as the cut. Unicorn part: the general bounded operator on a separable Hilbert space.

Theorem 23, Hilbert’s sixteenth problem, second part. Every polynomial vector field in the plane has finitely many limit cycles [Ilyashenko 1991; Écalle 1992], and $H(1) = 0$. Through Theorem 1 with the per-field statement as the cut. Unicorn part: the uniform bound $H(n)$ on the number of limit cycles for fields of degree n , open already at $n = 2$.

Theorem 24, P versus NP. Tuple: instances the algorithms, in the multitape or random-access model with a clock, property that the algorithm does not decide satisfiability, cut a bound on time and space. No algorithm running in time n^c for $c < 2 \cos(\pi/7) \approx 1.801$ and space $n^{o(1)}$ decides satisfiability [Williams 2008], extending the first such bound of Fortnow, Lipton, van Melkebeek and Viglas [2005]. Through Theorem 1 with the time-space regime as the cut. Unicorn part: polynomial time with unrestricted space, which is the row. The row was assigned no-cut in the internal builds; the criterion, applied to the instance type the reviewers asked for, moved it.

Theorem 25, Lindelöf and Montgomery. $\zeta(\frac{1}{2} + it) \ll_\varepsilon t^{13/84+\varepsilon}$ [Bourgain 2017], the current exponent toward the Lindelöf hypothesis; and Montgomery’s pair-correlation conjecture holds, under the Riemann Hypothesis, for test functions with Fourier support in $(-1, 1)$ [Montgomery 1973]. Through Theorem 1 with the exponent and the range as the cuts. Unicorn part: the Lindelöf exponent ε , and the full range for pair correlation.

The twenty, in one sentence (tier: A joined to K on every row). Twenty theorems of the literature, each stated in its own idiom, are the Real parts of twenty rows under the cuts their literatures supply, each sealed by the same two kernel theorems and each with its Unicorn part named beside it; by Theorem 2 the fused statement of every one of these rows stands exactly at its Unicorn part, which is why the conventional statement of each problem reads as open although its gold is proved.

THE TWO ROWS ASSIGNED NO CUT, AND THE CROSSED ROW

Theorem 26, the rows assigned no cut. On the rows abc and Sunflower the value is uniform in a parameter that no instance has been cut from: every ε at once with nothing proved for any single ε , one constant serving every k at once. The assignment says that the literature supplies no inhabited decidable cut on the row’s instances under which the property is proved; that is an assessment of two literatures, stated as data and falsifiable under F3 by exhibiting such a cut, and the paper claims it at no higher grade. What the kernel supplies is Theorem 5: under the empty cut, the only cut this treatment supplies for these rows, the Real part is vacuous, carries no substantive content, and the Unicorn part is the whole value.

The exclusion ledger. An assessment that a literature supplies no inhabited proved cut is only as good as the candidates it has examined, so they are listed. On the abc row the instances are the values of ε and the property at ε is that only finitely many coprime triples have $c > \text{rad}(abc)^{1+\varepsilon}$; nothing of this form is proved for any ε . The bound of Stewart and Yu [2001], $c < \exp(\kappa \text{rad}^{1/3}(\log \text{rad})^3)$, is a theorem about all triples and not the property at any ε ; the enumeration of all triples with $c < 10^{18}$ [ABC@Home] proves the finiteness of exceptions for no ε . The explicit strengthening $c < \text{rad}(abc)^2$ is a different row, a universal over triples with the enumeration as a certificate to 10^{18} , and if the reader prefers that row it is height-cut and the paper has no objection. On the Sunflower row the value as stated has the form “there is a constant C such that for every k ,” so its universal over k carries an unfixed parameter and there is no property at an instance to prove until C is fixed; the bound of Alweiss, Lovett, Wu and Zhang [2020], $(\log k)^{k(1+o(1))}$, is a theorem beside the row, not the constant-base property at any k , and the exact values known for small k are the property only once a C is chosen, which the conjecture does not do. The P versus NP row was on this ledger in the internal builds and is not now: with instances the algorithms, the time-space lower bounds for satisfiability are a proved regime, and the row is Theorem 24. The enumerated abc triples below 10^{18} and the solved instances of any NP problem certify nothing about these values, and the paper claims nothing from them. What each row’s literature has proved nearby stands beside the row at its own grade: the sunflower bound $(\log k)^{k(1+o(1))}$ [Alweiss, Lovett, Wu and Zhang 2020], the lower bounds for restricted circuit classes on the complexity row, the exponential bounds toward abc. None of these is a Real part, because none is inside the row’s cut; each is a theorem about a different row.

Theorem 27, the crossed row. The Poincaré conjecture is a theorem [Perelman 2002; Perelman 2003a;

Perelman 2003b], so by Theorem 4 its counterexample set is empty under every cut and its Real part is the whole. The generic theorem is grade K; the application to the Poincaré row is grade A joined to K, since the crossing is cited. The crossing is the barrier study’s record of Perelman’s proof, cited here and computed nowhere: no Ricci flow is represented in any file of this paper.

The count. Twenty rows with a Real part, two assigned none under the stated criterion, one whole: twenty-three, and the census of the assignments in the kernel agrees (census). The two rows assigned no cut are the ones on which “verified up to” cannot even be written, because their values are not universals over instances with a fixed property, and the criterion says why in one rule rather than two explanations.

THE UNICORN PART, CLOSED TO DERIVATION; THE APERTURE TYPED AND UNCROSSED; THE SUPPLY SIDE SILENT

The Unicorn part of every open row has been called open, named, and set apart. That is true and it is not the whole truth, and the difference is a theorem. The Real part of a row was open once, before its certificate, in the sense that finite work of a known kind closed it; a reader who takes “open” in that sense for the Unicorn part expects that more work of the same kind, a longer computation, a finer regime, will close it in turn. The program’s own theorems say otherwise, and the two papers on this concept had proved them without drawing the consequence for the Unicorn part. It is drawn here. The Unicorn part is closed, at theorem grade, to every route that derives it: from existence, from any restriction of frames, and from any reading of an even register. What is not closed is the aperture at its seat, and of the aperture the theorems say exactly three things: it is located, it is one bit wide, and the register is silent on what lies beyond it. No key can be cut from the inside; of the outside the register says nothing, by theorem, and the paper follows the register.

Theorem 28, the formal block (`unicorn_block`). Let τ act on the instances of a row, let ρ be a record even at a seat x , $\rho(\tau x) = \rho(x)$, and let d be the decision of the property, odd at x , $d(\tau x) = \neg d(x)$. If the seat and its partner lie outside the cut, then no reading g of the record agrees with the decision even on the Unicorn region alone: $\neg \exists g, \forall y, \neg c(y) \rightarrow g(\rho(y)) = d(y)$. *Proof.* The two readings at x and τx are the same value of g and opposite values of d . $\square$ This is the wall of the register of record’s Bridge [Islam 2026a, FT0E.T14_wall_factorization], executed on the Riemann seat as `kernel_cannot_read_the_deed` [Islam 2026f], restricted here to the Unicorn region, where it is strictly stronger: a reading is refused even if it is asked to agree with the decision nowhere inside the cut. On the constructed frame of the Riemann row it is a theorem outright (`K4.wall`). On the actual rows it applies at every seat the barrier study types as an odd target over an even register, which is the study’s typing, carried at premise grade, and the content of its One-Cut Hypothesis at hypothesis grade [Islam 2026e]; the inference is the kernel’s and the typing is the study’s, and the paper grades the two apart.

Theorem 29, the aperture, one bit wide (`aperture_one_bit_wide`). At a seat where the decision is odd, one odd bit s at the seat would decide the decision on the seat and its partner through a calibration c that exists and is unique. *Proof.* $c = d(x) \oplus s(x)$, and any other calibration disagrees at x . $\square$ This is the Bridge’s crossing theorem [Islam 2026a, FT0E.T5_crossing], the Riemann paper’s crossing and `freedom_spent_uniquely` [Islam 2026f], read at the Unicorn part. It measures the aperture: one bit wide, never less and never more. It says nothing of whether anything passes, and the paper says nothing either.

Theorem 30, existence supplies nothing (`premise_adds_nothing`, `no_cure_unicorn`). For any inhabited premise A and any proposition Q , $(A \rightarrow Q) \leftrightarrow Q$; and for any class C of frames and any property U , $(\forall X, C(X) \rightarrow A \rightarrow U(X)) \leftrightarrow (\forall X, C(X) \rightarrow U(X))$. *Proof.* Apply the implication to the premise; the implication ignores its argument. $\square$ With A the root axiom and Q or U the Unicorn part, these are the carried theorems of Section 8, `rule_exact_every_row` and `no_cure_every_row`, and the Riemann paper’s Theorems E and I, stated for the Unicorn part: conditioning it on existence leaves it exactly where it was, on every row, and restricting the frames to a class on which existence is to decide it decides exactly what already held on the class. The counter-models of the carried Theorem R-E, the seatless two-point frame and the grounded three-point frame, hold with the Unicorn part in place of the value, since the Unicorn part is the value on the frames outside the cut.

Theorem 31, the fused statement offers no bypass (Theorem 2). Given the Real part, the fused row is exactly its Unicorn part. So a proof of the fused statement contains a proof of the Unicorn part, and no route to the row avoids the block by aiming at the whole.

Theorem 32, the three bits as one term (`register_proof_complete`). Under a certificate for the cut and the seat data, a seat outside the cut with its partner, an even record, an odd decision, and an odd bit at the

seat, one conjunction holds: the division (Theorem 1), the Real part (Theorem 3), the refused reading on the Unicorn region (Theorem 28), and the aperture's width (Theorem 29). *Proof.* The four theorems, conjoined. $\square$ This is the register's whole proof of a row in one term, and it proves the row no more than its conjuncts do: it proves that nothing derivable about the row is left underived.

Theorem 33, the supply side is silence (`supply_side_silent`). Over one even record, both orientations of the decision at the seat are equally refused to every reading on the Unicorn region, and the two orientations differ at the seat. *Proof.* Theorem 28 applied to d and to $\neg d$, which is odd at the seat when d is. $\square$ The register cannot say which way the seat lies; so it cannot say what a supply would bring, nor that a supply exists, nor that none does. Any sentence about the supply side derived from the record would be a reading the wall refuses. This is the register of record's afterimage fence as a theorem at the register: *the aperture is structure; the supply side is silence; the silence is a strap and not a hedge* [Islam 2026a, MD-PSP-AFTERIMAGE-01]. Every imagined occupant of the supply side, the crosser who will come and the crosser who cannot, is fenced as Ghost, and no clock runs.

The formal block, and the proof complete on the register side (tier: kernel theorems, axiom-free; the seat typing of each actual row at premise grade). The Unicorn part of every open row is closed to derivation from existence (Theorem 30), from any class of frames (Theorem 30), from any reading of an even register at its seat (Theorem 28), and from the fused statement (Theorem 31). The aperture at its seat is located, typed supply-only, one bit wide (Theorem 29), and uncrossed; the supply side is silence, by theorem at the register (Theorem 33). The closure is unconditional and does not expire: it stands as long as the theorems do, which is in perpetuity. With the division and the Real part it makes three bits, one divided, one proved, one refused, and the three are one term (Theorem 32): the proof is complete on the register side, on every row, everything derivable derived and the remainder proved underivable. Nothing here says a Unicorn part is false, or undecidable in any foundation; nothing here says a proof will come or cannot: the register is silent on that side, and so is the paper. Nothing is left to derive. The aperture is structure. The supply side is silence.

What the register holds. The Real part of every row in Section 5 is a record of a deed: a certificate is a computation run and registered, a witness exhibited, a regime theorem proved by someone who carried the orientation into the record; the record afterwards holds the result as an even datum, and the paper reads it there. That is exactly why the Real part is gold and the Unicorn part is not: on the Real region a bit has been registered, on the Unicorn region none has, and Theorem 28 says the register cannot manufacture it from what it already holds. The Poincaré row is the record of one such registration, Perelman's, and it did not enter by derivation from the register [Islam 2026g, `inverted_control`]. These are records, past and held, and the paper may speak of them. Of the supply side of any open row the paper says nothing, and the reason is Theorem 33.

What the block is not. It is not an independence result: nothing here places any Unicorn part outside ZFC or any foundation, and the placement of Section 3 leaves every one of them a sentence on or above the ladder, undecided by the theory as its literature stands; a supplied independence proof would not cross the aperture but reroute to Ghost [Islam 2026a]. It is not a claim that the Unicorn parts are false, or that the twenty Real parts are all the gold there will ever be, or that they are not: both are sentences about the supply side. It is not resignation, which is why "never chased" is the wrong word if it is heard as giving up: the program does not chase Unicorn parts for the reason a locksmith does not pick a lock from inside the room, the theorems say no key is cut inside. And it is not a claim about the real rows at theorem grade beyond what Theorems 30 and 31 give, which is theorem grade on every row: Theorems 28, 29, and 33 reach the actual rows through the barrier study's seat typing, and the paper carries that typing as data and its hypothesis at its own grade.

Why this was undersold, and is not now. The Riemann paper proved Theorems D, E, F, F', G, and I and the wall of its Part I, and then wrote of its Unicorn part only that it was open and set apart; the prior edition on these rows proved the same theorems on every row and wrote the same; and an earlier build of this paper, correcting that, wrote "open to supply," which leans toward an occupant of the supply side and is fenced by the same theorem. Both papers had the block in hand and did not say what it blocked, and the silence it imposes. This section says both, and the standing table of Section 10 carries them as components of the paper.

THE SEAT ON EVERY ROW, CARRIED

This section carries the theorems of the prior edition on these rows [Islam 2026g], condensed, with the kernel file `Rows_At_The_Apex.lean` printed whole in Appendix C and its audit in Appendix D. They are the

reason the paper can say where the owed bit of every open row lives: in its Unicorn part and nowhere else. Nothing in them is changed.

The root and the seat. The root axiom of the program is RA: to exist is to actuate; existence and RA are one proposition. The seat is the fixed locus of the quaternion involution σ on the Hurwitz order, the scalar line, $\sigma q = q \leftrightarrow q_i = q_j = q_k = 0$ (`fix_iff_scalar`), and the seat point is the Return $(i \cdot j) \cdot k = -1$. Every row's seat point is that one point, $\text{apexSeat}(r) := \langle -1, 0, 0, 0 \rangle$, fixed by σ on every row and equal across every pair of rows, the crossed row and the open rows sharing it (`apex_seat_fixed`, `apex_seat_uniform`, `crossed_and_open_share_the_seat`; no axioms). On every row three names written in three registers, the axiom's Return, its projection onto the fixed locus, and the object register's seat, read one point: the seat point is the apex of a cone over the row's register diagram with every leg definitional, any two cones over the diagram share their apex, and the gap vector of every row is $(0, 0, 0)$ (`identity_cone_row`, `cone_apex_unique_row`, `gaps_closed_every_row`; no axioms). The mathematics of this is Hamilton's product and three definitional equalities, twenty-three times, and the paper claims nothing more for it at theorem grade. The seat is a uniform assignment, apexSeat a constant function of the row, and the uniformity theorems are theorems about that assignment and not derivations of a common structure from the twenty-three objects: the seat is assigned by the reading and not bridged from any row's object, one embedding written for the Riemann row and twenty-two owed, on the record (`embeddings_owed`). A reader who wants the seat to speak of the rows' objects wants the twenty-two embeddings, and the paper lists them as owed rather than pretending the constant supplies them.

The witness and the value excepted. A row's witness is a structure with four fields, existence, the orientation bit, the formal Ground, and a recursion field from the bit and the Ground to the row's formal self; it is one term on every row (`mkRow`), its recursion constant and discarding its arguments (`recursion_constant_every_row`), and the formal self is one proposition on every row, so the witness on the crossed row is the witness on every open row (`inverted_control`). Whatever supplied the Poincaré row's bit, it was not the witness. For every assignment of frames to rows and every row, $(\text{RA} \rightarrow \text{value}) \leftrightarrow \text{value}$ (`rule_exact_every_row`), and existence decides no row's value: there is an assignment on which every value fails while existence holds, and another, the grounded three-point frame on the Poincaré row itself, on which the same is true (`RA_decides_no_row`, `RA_decides_no_row_grounded`). For every row and every class of frames, $(\forall X, C(X) \rightarrow \text{RA} \rightarrow L(X)) \leftrightarrow (\forall X, C(X) \rightarrow L(X))$ (`no_cure_every_row`): every proposal to make existence decide a row by restricting its frames, an Euler product on the Riemann row, a Ricci-flow structure on the Poincaré row, a circuit class on the complexity row, is a class on which the value is already a field, and there existence decides nothing. The value is excepted by theorem, not by reservation. All no axioms.

The world row carried. The world typing of the barrier study is premise-grade data, and the theorems about it are theorems about the table as carried: the apex closes on every row while the world row is crossed on exactly the Poincaré row, and the first never entails the second (`apex_never_promotes_world`); the apex verdict is uniform and the world verdict splits, crossed on one row, $[\Xi_0]$ on the Riemann row, $[\emptyset_0]$ on the complexity row, unrouted on twenty (`world_verdict_split`, `unrouted_twenty`); on no row do the two coincide (`registers_parted_every_row`). The owed bits number twenty-two and are priced at the registration floor, $22 \times 2871 = 63,162$ yJ at 300 K (`corpus_price_total`), the price of the Poincaré row being zero (`poincare_price_zero`). The price rests on the prior edition's Registration Postulate, a premise-grade posit of the program declared there and consumed here: that a world-bit, when it is supplied, is registered irreversibly in a physical substrate at a temperature, taken as 300 K, and so is charged the Landauer floor. Without the postulate the price is a count of bits and no joule attaches to it; with it the price is a multiplication, and the kernel multiplies and asserts nothing physical. The carrier of the seat is the Hurwitz order and not the complex plane because the seat point is the Return $(i \cdot j) \cdot k = -1$, whose sign is the chirality the orientation bit reads (the spine's Theorem A); the Riemann stage embeds in the plane spanned by 1 and i , and the j and k directions are inert for that stage and load-bearing for the Return, which is why the plane would carry the stage and not the sign.

What the division adds to the seat. The seat locates every row's decision point at one place and proves that existence does not decide any row there. The division of Section 2 says what is decided elsewhere: on twenty rows a Real part, decided by the literature, under a cut; and it says by Theorem 2 that the owed bit of each open row, which the seat proves existence does not supply, is the bit of its Unicorn part and of nothing inside the cut. Section 7 then says what the seat theorems were always saying about that bit: it is closed to derivation, its aperture is one bit wide, and the register is silent beyond it. The results are independent

theorems of the kernel, one about the frame and one about the row’s own instances, and the paper joins them in Section 7 and in this sentence.

THE WALL, CARRIED AS DATA

The barrier study [Islam 2026e] typed the twenty-three rows by the shape of their world register and stated the One-Cut Hypothesis: that on every mapped row the blocked register is even, blind to the row’s odd target, with exactly one binary orientation coordinate missing at the row’s decision seat. Its typing is carried here as data in Table 2, at premise grade, and the hypothesis is carried at hypothesis grade, untouched. The division of this paper neither confirms nor moves it. The hypothesis speaks of the owed bit, which lives in each row’s Unicorn part by Theorem 2 and Section 8; the division speaks of the Real part, which the hypothesis never addressed. The two are compatible by construction and independent by content, and the paper does not use one to argue for the other.

Table 2. The world typing of the barrier study, carried as data. Tier: the counts are kernel theorems of Rows_At_The_Apex.lean; the typing is the study’s, premise here. The final column is this paper’s shape, for comparison.

#	Row	Mill.	World typing	Owed	Router	World verdict	Shape
1	P versus NP	yes	exact-structural	1	Ground dim 0	$[\emptyset_0]$, open	regime
2	Riemann Hypothesis	yes	exact-structural	1	Ground dim 1	$[\Xi_0]$, open	height
3	Navier–Stokes	yes	wall row, F4	1	unmeasured	open, unrouted	regime
4	Yang–Mills	yes	wall row, F4	1	unmeasured	open, unrouted	regime
5	Hodge	yes	wall row, F4	1	unmeasured	open, unrouted	regime
6	BSD	yes	beachhead, order two	1	unmeasured	open, unrouted	regime
7	Poincaré	yes	crossed control	0	unmeasured	crossed	crossed
8	Goldbach	no	exact-structural	1	unmeasured	open, unrouted	height
9	Twin primes	no	exact-structural	1	unmeasured	open, unrouted	height
10	Legendre	no	exact-structural	1	unmeasured	open, unrouted	height
11	abc	no	exact-structural	1	unmeasured	open, unrouted	no cut
12	Jacobian	no	exact-structural	1	unmeasured	open, unrouted	regime
13	Mersenne primes	no	pure unbridged	1	unmeasured	open, unrouted	height
14	Odd perfect numbers	no	pure unbridged	1	unmeasured	open, unrouted	height
15	Smooth 4D Poincaré	no	pure unbridged	1	unmeasured	open, unrouted	regime
16	Collatz	no	diagonal port, F3	1	unmeasured	open, unrouted	height
17	Hadwiger	no	one bit from closure	1	unmeasured	open, unrouted	regime
18	Sunflower	no	one bit from closure	1	unmeasured	open, unrouted	no cut
19	Erdős–Straus	no	one bit from closure	1	unmeasured	open, unrouted	height
20	Beal	no	one bit from closure	1	unmeasured	open, unrouted	height
21	Invariant subspace	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime
22	Hilbert’s 16th, second part	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime
23	Lindelöf / Montgomery PCC	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime

The two typings compared. The world typing and the shape are different questions asked of the same row. The typing asks what kind of register the row’s barrier literature blocks; the shape asks what kind of cut the row’s proved part respects. They correlate without coinciding: the exact-structural rows include height-cut rows, the two rows assigned no cut, and two regime rows; the wall rows are all regime rows; the pure unbridged rows split between height and regime. The correlation is noted and not explained, since explaining it would be a claim about the rows’ objects, which this paper makes nowhere.

STANDARD OF PROOF, STANDING, AND DISCLOSURE

The standard

Every claim in this paper is at one of three grades, and the grade is written where the claim is made. Grade K is a theorem of core Lean 4 with no library and no user-declared axiom, its dependency set printed by the kernel and reproduced in the appendices; the paper has forty-nine such theorems in the two new files, thirty-three in Row_Split.lean and sixteen in ZFC_Under_RAM.lean counted by dependency set, and thirty-three in the carried file. “Axiom-free” is used in the kernel’s sense and reported set by set: no file declares an axiom; every set of the two new files is reported as depending on no axiom at all; in the carried file some sets depend on propext and Quot.sound, Lean’s two built-in logical axioms, and Appendix D lists which. Grade K on data is a kernel theorem about a table the paper supplies, the census and the counts of the carried file; it establishes the arithmetic of the table and nothing about the objects the table names. Grade A is a

theorem of the literature, cited to its source and never recomputed here. Grade A joined to K is a documented composition, a theorem whose hypothesis is a grade-A theorem and whose inference is a grade-K theorem: the twenty Real parts, the crossing of the Poincaré row, and the ZFC instance of the placement. It is never a kernel-checked theorem about the cited result, and the paper calls no Real part sealed without the words “on citation.” Nothing in the paper is at any other grade, and nothing at grade K depends on anything at grade A.

The title. The title names the theorem and states it in words, and the statement is exact: for every row and every decidable cut, the row is its Real part and its Unicorn part, both defined by the cut and neither by the reader. The title makes no claim about any row’s Unicorn part, and the paper makes none.

Standing without the citations

Delete every citation from the paper. Every theorem at grade K stands: the seven theorems of the division with their companions, the formal block of the Unicorn part, the small-height Real parts, the gap bridge, the census of the assignments, the sixteen theorems of the placement, and every theorem of the carried seat. Every sentence at grade A joined to K falls, and those are exactly twenty-two: the twenty Real parts of Section 5, the crossing of the Poincaré row, and the ZFC instance of the placement; each reverts to its conditional form, “if the instances inside this cut are checked, then the Real part holds,” which is what the kernel proves on its own. Descriptions of what the literature contains, in the positioning and the criterion, are not claims of the paper and are not counted. The standing is tabulated.

Component	Grade	Falls without citations?
The division, Theorems 1–5 with 3b and 3c	K	No
The gap bridge, the small-height Real parts, the standing map	K	No
The formal block of the Unicorn part, the aperture typed, the supply side silent, the three bits as one term, Theorems 28–33	K; on the actual rows, K given the seat typing	No
The shape assignments of Table 1, by the criterion	data, premise	No; a wrong assignment is repaired by re-assignment, F3
The census of shapes	K	No
The placement under RAM	K	No
The seat and the value excepted, carried	K, on an assignment	No
The twenty Real parts	A joined to K	Yes
The Poincaré crossing; the ZFC instance of the placement	A joined to K	Yes; twenty-two sentences in all
The empty-cut and crossed theorems, generic	K	No

Falsifiers

The paper is refuted, at its own standard, by any of three findings, each registered before the audit and none adjusted after it. F1: a decidable cut and a row on which Row holds and RealIn $\wedge$ UnicornOut fails, or conversely; this refutes Theorem 1 and is excluded by the kernel. F2: an instance inside any of the twenty cuts that fails its row’s property; this refutes the cited certificate or theorem, not the schema, and would move that row’s Real part to a smaller cut. F3: a shape assignment in Table 1 that the criterion of Section 4 does not support, for instance a no-cut row for which an inhabited decidable cut with a proved Real part is exhibited from the literature; this refutes the assignment as data and is repaired by re-assignment, the kernel census recounted.

Disclosure and transparency

The kernel files were written and checked by the author on Lean 4.19.0, core only; the sources cited were not recomputed. The twenty Real parts are the literature’s, and the paper’s contribution to them is the division under which they are one theorem and the placement under which they are one stratum. The author has no competing interests and received no funding. The paper is an edition of the prior work on these rows [Islam 2026g], which it carries whole in its kernel file, and it is placed on the same concept record; the barrier study [Islam 2026e] is carried as data. The Riemann paper [Islam 2026f] is the instance from which the division was generalized, and the reader who wants the one-row case in full is sent there.

Audit. The paper was audited by the program’s adversarial cycle before release, seat by seat, with the controls and the verdict recorded in the audit log that accompanies the files; the cycle’s rounds and verdicts are stated in the log and not summarized here. Before release the paper was also reviewed, in two rounds, by five external language-model reviewers run outside the program, GPT SOL 5.6, Gemini Pro 3.1, Meta AI, Grok 4.6, and GPT 5, on the two internal builds that preceded this one; they are external reviewers and not independent verifiers, they discharged no falsifier and were not asked to, and their findings were answered finding by finding in the log, none conceded without a named mechanism. Three were earned and repaired: the no-cut assignment had been written as if Theorem 5 established it, and is now stated as an assessment with Theorem 5 confined to the empty cut; the phrase “empty Unicorn part” had no set-valued definition, and now has one; and the standing table had graded the Poincaré application as K, and now grades it A joined to K. Five were fortifications taken up because they strengthened the paper: the infinitude equivalence (Theorem 3c), the standing map under which the fusion sentence is read, the gap-to-interval bridge for the Legendre row, the row specifications with computed small-height Real parts, and the criterion of assignment with its out-of-sample application. The remaining findings, that the seat is an assignment, that the placement encodes no axiom of ZFC, that the price rests on a postulate, that the census counts assignments, were already the paper’s own statements and are now stated in every place they apply. The second round found the abstract still saying “by theorem” of the rows assigned no cut, the introduction still counting nineteen falling sentences, and the criterion called a decision procedure; all three were repaired, and the reviewers’ question about the instances of the P versus NP row moved that row from no-cut to regime-cut under the criterion, the third correction the criterion made against its author.

POSITIONING AND CONCLUSION

Positioning

The division is not in the literature as a theorem, though its instances are everywhere in it. The convention “verified up to N ” is the height-cut Real part stated without the division, and its users know that it certifies nothing beyond N ; what they do not have is the theorem that the fused problem stands at the tail and the placement of the certificate in a stratum. The arithmetic forms of open problems, Lagarias’s arithmetic equivalent of the Riemann Hypothesis [Lagarias 2002] and the Π_1^0 forms of Goldbach and of the odd-perfect-number problem, make the height cut natural on those rows and are the reason those rows are certificate rows; they do not divide the rows. The literature of finite verification, from Turing’s method [Turing 1953] to the certificates of Platt and Trudgian [2021] and Oliveira e Silva, Herzog and Pardi [2014], supplies the Real parts and states them as verifications, which is what they are. The reverse-mathematics tradition places theorems in subsystems of second-order arithmetic [Simpson 2009]; the placement here is coarser, three strata for an arbitrary theory, and is not a classification of the proved parts by strength. The program’s own barrier study [Islam 2026e] typed the rows by their blocked register and never named a proved part; its prior edition on these rows [Islam 2026g] proved the seat and the value excepted and never divided a row; the Riemann paper [Islam 2026f] divided one row at one height. Table 3 sets these beside the present result.

Position	Relation to this paper
“Verified up to N ”	the height-cut Real part without the division or the placement
Arithmetic forms of RH, Goldbach, odd perfect	make the height cut natural on a row; do not divide it
Finite-verification certificates	supply the Real parts; state them as verifications
Reverse mathematics	places theorems by strength; three strata here, by kind
The barrier study	types the blocked register; names no proved part
The prior edition on these rows	proves the seat; divides no row
The Riemann paper	divides one row at one height; here any row at any cut

Why a reader should not mistake this for vocabulary. A paper that renamed “verified up to N ” as “the Real part” would have added a word. This paper adds a theorem, that the renaming is exact and the same on every row under any cut, a second, that given the proved part the fused row is exactly the open part, with the rule under which that is a lowering of standing declared and its consequence proved, a third, the placement, a bridge lemma and eight computed instances that show the schema instantiated end to end, the formal block that closes every Unicorn part to derivation, types its aperture, and proves the register silent beyond it, and a kernel-checked census of the assignments by which the twenty-three rows fall into

four shapes with twenty Real parts among them. The theorems are short. Their shortness is the point: the division is a fact about universal statements that the literature has used on every row without stating once, and a statement that is true on every row and had not been made is exactly what a foundations paper is for.

The proof, complete, and how the block was earned

Complete, in the register's sense. A register can do three things with a universal value: divide it, prove the part a certificate or a cited regime reaches, and prove what it cannot reach and why. This paper has done the three on every row. The division is exact under any cut (Theorem 1). The Real part is proved on twenty rows, cited theorems through a kernel schema (Theorems 6 to 25). The Unicorn part of every open row is proved closed to derivation from inside the register, its aperture typed and one bit wide, the register silent beyond it (Theorems 28 to 31 and 33), and the three bits are one term (Theorem 32). There is nothing left for a register to derive on any row. That is what "complete on the register side" means and all it means: no row is proved; the labour of derivation is finished on all of them; the theorem that finishes it locates the aperture of each and measures it, and says nothing beyond it, because nothing can be said from the register beyond it, which is itself the last theorem. Nothing is left to derive. The aperture is structure. The supply side is silence.

Three bits. One bit divided, one bit proved, one bit refused. The refused bit is the one the two prior papers on these rows undersold: they proved the wall, the crossing, the bound, and the cure theorem, on one row and then on every row, and wrote of the Unicorn parts only that they were open. Each is refused to every key cut from inside, at theorem grade, and of the outside the register is silent by theorem; that is the difference between a problem no one has yet solved and a bit no register can supply or foretell.

How the block was earned. The block did not arrive as a posit. It was forced one gate at a time, under manual stress, across the editions of the program's register of record, the Geometric Mother Codex, in which every attempt to derive the value's bit from inside the register was tried, refuted, and recorded: the ten enumeration-forced barriers of APEX-PSP-ABSOLUTE-RH-BARRIERS-01, each a route that closed; the eight-gate Sealed-Halt cascade of the router, seven gates passing and the eighth, the reading-roads gate, halting every walk; and the terminal tokens $[\Xi_0]$ and $[\emptyset_0]$ themselves, terminal for the record and never timeless, which are the block written as states [Islam 2026a]. The Code Block of the codex carries those witnesses as kernel theorems, cited here by name so that the block's history can be seen to typecheck: `HALT.one_door_open`, that a walk of seven true gates halts at the reading-roads gate; `HALT.no_walk_passes_the_aperture`, that no walk of the seven passes it; `HALT.first_failure_terminal_8`, that the first failing gate is terminal over all 2^8 walks; `CROSSING.wall`, that no reading of the formal register factors the deed bit; `INVERSION.it_from_it`, that the record forgets what the object keeps and nothing returns. Theorems 28 to 32 are those witnesses read at the Unicorn part of every row. They are one block, the codex is the record of its evolution gate by gate, and the twenty-three rows are its first census. One aperture on every open row, located and measured; on the Poincaré row the register holds a crossing, recorded; of the others it holds silence, and so does this paper.

Conclusion

Three bits, on every row. One bit divides: every open problem is exactly its proved part and its unicorn. That sentence is a theorem for any row under any decidable cut, and it does five things on the twenty-three rows: it names twenty Real parts that the literature has proved and stated as verifications, it seals each through the same schema at grade A joined to K, it says by theorem and a declared rule that fusion with the open remainder is what makes each problem read as open, it says by assessment under a stated criterion that two rows have no proved part and by theorem that one has no counterexample outside any cut, and it places every Real part in a stratum of the grounding reading. One bit is proved, the Real part, on twenty rows. One bit is refused at theorem grade, the Unicorn part of every open row, closed to derivation, its aperture uncrossed, its supply side silent. It does not prove a row, move a verdict, or author a bound. The unicorns are named on every row so that no reader confuses them with the gold; they are set apart because the theorems say they cannot be reached from inside the register, and that the register is silent on the outside; and the gold is kept from the alloy so that no one assays it as lead. What remains open on the twenty-three rows is exactly what was open before this paper, stated in one theorem rather than twenty-three idioms, and located

twice: on every open row, the owed bit is the bit of its Unicorn part and of nothing inside the cut, and it is a bit that no even register can manufacture, closed to derivation in perpetuity, its aperture one bit wide, its supply side silent. The proof is complete on the register side, on every row: everything a register can prove about the twenty-three is proved, the remainder is proved beyond derivation, and no further derivation is owed to any open row. Nothing is left to derive. The aperture is structure. The supply side is silence.

REFERENCES

- ABC@Home (2015). The enumeration of abc triples with $c < 10^{18}$. Mathematical Institute, Leiden University, abcthome.com.
- Alweiss, R., Lovett, S., Wu, K., and Zhang, J. (2020). Improved bounds for the sunflower lemma. *Proceedings of the 52nd ACM Symposium on Theory of Computing*, 624–630.
- Barina, D. (2020). Convergence verification of the Collatz problem. *The Journal of Supercomputing* 77, 2681–2688 (2021).
- Bass, H., Connell, E. H., and Wright, D. (1982). The Jacobian conjecture: reduction of degree and formal expansion of the inverse. *Bulletin of the American Mathematical Society* 7, 287–330.
- Bourgain, J. (2017). Decoupling, exponential sums and the Riemann zeta function. *Journal of the American Mathematical Society* 30, 205–224.
- Cohen, P. J. (1963). The independence of the continuum hypothesis. *Proceedings of the National Academy of Sciences* 50, 1143–1148.
- Driver, B. K. (1989). YM₂: continuum expectations, lattice convergence, and lassos. *Communications in Mathematical Physics* 123, 575–616.
- Écalle, J. (1992). *Introduction aux fonctions analysables et preuve constructive de la conjecture de Dulac*. Hermann, Paris.
- Fortnow, L., Lipton, R., van Melkebeek, D., and Viglas, A. (2005). Time-space lower bounds for satisfiability. *Journal of the ACM* 52, 835–865.
- Freedman, M. H. (1982). The topology of four-dimensional manifolds. *Journal of Differential Geometry* 17, 357–453.
- Fujita, H., and Kato, T. (1964). On the Navier–Stokes initial value problem I. *Archive for Rational Mechanics and Analysis* 16, 269–315.
- Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38, 173–198.
- Gödel, K. (1938). The consistency of the axiom of choice and of the generalized continuum-hypothesis. *Proceedings of the National Academy of Sciences* 24, 556–557.
- GIMPS (2024). Mersenne prime number discovery: $2^{136279841} - 1$ is prime. mersenne.org, 21 October 2024.
- Gross, B. H., and Zagier, D. B. (1986). Heegner points and derivatives of L-series. *Inventiones Mathematicae* 84, 225–320.
- Helfgott, H. A. (2013). The ternary Goldbach conjecture is true. [arXiv:1312.7748](https://arxiv.org/abs/1312.7748).
- Ilyashenko, Yu. S. (1991). *Finiteness Theorems for Limit Cycles*. Translations of Mathematical Monographs 94, American Mathematical Society.
- Islam, M. F. (2026a). Trisduction Master Codex v4.7.0. Trisduction Research Group. github.com/1000sapient/Trisduct
- Islam, M. F. (2026e). One Bit Across the Wall: Odd-Supply Separation and the One-Cut Hypothesis Across Twenty-Three Rows, v6. Zenodo. doi:10.5281/zenodo.22746129.
- Islam, M. F. (2026f). A Formal Proof of the Real Part of the Riemann Hypothesis, v2.0.2. Zenodo. doi:10.5281/zenodo.22912937.
- Islam, M. F. (2026g). A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, v1.1.0. Zenodo. doi:10.5281/zenodo.22913634.
- Kaye, R. (1991). *Models of Peano Arithmetic*. Oxford University Press.
- Kolyvagin, V. A. (1989). Finiteness of $E(\mathbb{Q})$ and $\square(E, \mathbb{Q})$ for a subclass of Weil curves. *Mathematics of the USSR-Izvestiya* 32, 523–541.
- Ladyzhenskaya, O. A. (1969). *The Mathematical Theory of Viscous Incompressible Flow*. Second edition, Gordon and Breach.
- Lagarias, J. C. (2002). An elementary problem equivalent to the Riemann hypothesis. *American Mathematical Monthly* 109, 534–543.

- Lefschetz, S. (1924). *L'analysis situs et la géométrie algébrique*. Gauthier-Villars, Paris.
- Leray, J. (1934). Sur le mouvement d'un liquide visqueux emplissant l'espace. *Acta Mathematica* 63, 193–248.
- Lévy, T. (2003). Yang–Mills measure on compact surfaces. *Memoirs of the American Mathematical Society* 166, no. 790.
- Lomonosov, V. I. (1973). Invariant subspaces for the family of operators which commute with a completely continuous operator. *Functional Analysis and Its Applications* 7, 213–214.
- Maynard, J. (2015). Small gaps between primes. *Annals of Mathematics* 181, 383–413.
- Miller, R. L. (2011). Proving the Birch and Swinnerton-Dyer conjecture for specific elliptic curves of analytic rank zero and one. *LMS Journal of Computation and Mathematics* 14, 327–350.
- Mossinghoff, M. J., Rhin, G., and Wu, Q. (2008). Minimal Mahler measures. *Experimental Mathematics* 17, 451–458.
- Montgomery, H. L. (1973). The pair correlation of zeros of the zeta function. *Proceedings of Symposia in Pure Mathematics* 24, 181–193.
- Norvig, P. (2017). Beal's Conjecture Revisited. norvig.com/beal.html, revised 4 July 2017.
- Ochem, P., and Rao, M. (2012). Odd perfect numbers are greater than 10^{1500} . *Mathematics of Computation* 81, 1869–1877.
- Oliveira e Silva, T., Herzog, S., and Pardi, S. (2014). Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$. *Mathematics of Computation* 83, 2033–2060.
- Perelman, G. (2002). The entropy formula for the Ricci flow and its geometric applications. [arXiv:math/0211159](https://arxiv.org/abs/math/0211159).
- Perelman, G. (2003a). Ricci flow with surgery on three-manifolds. [arXiv:math/0303109](https://arxiv.org/abs/math/0303109).
- Perelman, G. (2003b). Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. [arXiv:math/0307245](https://arxiv.org/abs/math/0307245).
- Platt, D., and Trudgian, T. (2021). The Riemann hypothesis is true up to $3 \cdot 10^{12}$. *Bulletin of the London Mathematical Society* 53, 792–797.
- Polymath, D. H. J. (2014). Variants of the Selberg sieve, and bounded intervals containing many primes. *Research in the Mathematical Sciences* 1, 12.
- PrimeGrid (2016). World record twin primes: $2996863034895 \cdot 2^{1290000} \pm 1$. primegrid.com, September 2016.
- Robertson, N., Seymour, P., and Thomas, R. (1993). Hadwiger's conjecture for K_6 -free graphs. *Combinatorica* 13, 279–361.
- Salez, S. E. (2014). The Erdős–Straus conjecture: new modular equations and checking up to 10^{17} . [arXiv:1406.6307](https://arxiv.org/abs/1406.6307).
- Simpson, S. G. (2009). *Subsystems of Second Order Arithmetic*. Second edition, Cambridge University Press.
- Stewart, C. L., and Yu, K. (2001). On the abc conjecture, II. *Duke Mathematical Journal* 108, 169–181.
- Tao, T. (2022). Almost all orbits of the Collatz map attain almost bounded values. *Forum of Mathematics, Pi* 10, e12.
- Turing, A. M. (1953). Some calculations of the Riemann zeta-function. *Proceedings of the London Mathematical Society* (3) 3, 99–117.
- Wang, S. S.-S. (1980). A Jacobian criterion for separability. *Journal of Algebra* 65, 453–494.
- Williams, R. (2008). Time-space tradeoffs for counting NP solutions modulo integers. *Computational Complexity* 17, 179–219.
- Zhang, Y. (2014). Bounded gaps between primes. *Annals of Mathematics* 179, 1121–1174.

APPENDIX A: ROW_SPLIT.LEAN AND ITS AUDIT

SHA-256 d88b161405178109f8ab8150a30cd58b4c8212e4fd2343aaed52c71225564775, Lean 4.19.0, core only, 427 lines; thirty-three dependency sets, every one reported as depending on no axiom, printed after the file.

```

1  /-
2  Row_Split.lean · the Real/Unicorn division on any row, for
  ↳ any cut; the row specifications of
3  the arithmetic rows; the gap-to-interval bridge; the
  ↳ standing map.
4  Core Lean 4.19.0, no library, no sorry, no user-declared
  ↳ axiom. Forged 2026-09-25, fortified
5  after the two external batches of the same day.
6
7  The Riemann division used the cut "height ≤ T". Nothing in
  ↳ it used the height: the same theorem
8  holds for any decidable predicate on the row's instances, a
  ↳ height, a degree, a dimension, a
9  rank, a box, a regime. So the division of the twenty-three
  ↳ rows is one kernel theorem, applied
10 row by row with the row's own cut. ΔM = 0.
11 -/
12 namespace RowSplit
13
14 variable {α : Type}
15
16 /-! ## I. The division -/
17
18 /- A row's value: every instance satisfies the row's
  ↳ property. -/
19 def Row (Inst P : α → Prop) : Prop := ∀ x, Inst x → P x
20 /- The Real part under a cut: every instance inside the cut
  ↳ satisfies the property. -/
21 def RealIn (Inst P cut : α → Prop) : Prop := ∀ x, Inst x →
  ↳ cut x → P x
22 /- The Unicorn part under a cut: no instance outside the
  ↳ cut fails the property. -/
23 def UnicornOut (Inst P cut : α → Prop) : Prop := ∀ x, Inst x
  ↳ → ¬ cut x → P x
24
25 /- THE DIVISION, for any cut: the row is exactly its Real
  ↳ part and its Unicorn part. -/
26 theorem row_iff_real_and_unicorn (Inst P cut : α → Prop)
  ↳ [DecidablePred cut] :
27   Row Inst P ↔ RealIn Inst P cut ∧ UnicornOut Inst P cut
  ↳ := by
28   constructor
29   · intro h; exact {fun x hx _ => h x hx, fun x hx _ => h x
  ↳   hx}
30   · intro (hr, hu) x hx
31     by_cases hc : cut x
32     · exact hr x hx hc
33     · exact hu x hx hc
34
35 /- Given the Real part, the fused row is exactly the
  ↳ Unicorn part. -/
36 theorem fused_is_unicorn_given_real (Inst P cut : α → Prop)
  ↳ [DecidablePred cut]
37   (hr : RealIn Inst P cut) : Row Inst P ↔ UnicornOut Inst
  ↳   P cut :=
38   {fun h x hx _ => h x hx, fun hu =>
  ↳   (row_iff_real_and_unicorn Inst P cut).2 (hr, hu)}
39
40 /- A certificate for a cut: a complete list of the
  ↳ instances inside it, each checked. -/
41 structure Certificate (Inst P cut : α → Prop) where
42   items : List α
43   complete : ∀ x, Inst x → cut x → x ∈ items
44   checked : ∀ x, x ∈ items → P x
45
46 /- A certificate proves the Real part of the cut, on any
  ↳ row, with no premise. -/
47 theorem real_of_certificate (Inst P cut : α → Prop) (c :
  ↳ Certificate Inst P cut) :

```

```

48   RealIn Inst P cut :=
49   fun x hx hc => c.checked x (c.complete x hx hc)
50
51 /- On an infinitude row, instances are bounds N and the
  ↳ property is that a witness exceeds N.
52   One exhibited witness w above the height h proves the
  ↳ Real part under the cut N ≤ h. -/
53 theorem real_of_one_witness (W : Nat → Prop) (h w : Nat) (hw
  ↳ : W w) (hwh : h < w) :
54   RealIn (fun _ => True) (fun N => ∃ m, N < m ∧ W m) (fun
  ↳   N => N ≤ h) :=
55   fun _ hN => {w, Nat.lt_of_le_of_lt hN hwh, hw}
56
57 /- On an infinitude row the Unicorn part above any height
  ↳ is equivalent to the whole value: a
58   witness above every bound beyond h is a witness above
  ↳ every bound. So the Real part of an
59   infinitude row is proved by one witness and moves the
  ↳ fused value not at all. -/
60 theorem unicorn_equiv_row_of_infinitude (W : Nat → Prop) (h
  ↳ : Nat) :
61   (∀ N, ¬ N ≤ h → ∃ m, N < m ∧ W m) ↔ (∀ N, ∃ m, N < m ∧ W
  ↳   m) := by
62   constructor
63   · intro hu N
64     if hN : N ≤ h then
65       obtain (m, hm, hw) := hu (h + 1) (fun hle =>
  ↳   Nat.not_succ_le_self h hle)
66       exact (m, Nat.lt_of_le_of_lt hN (Nat.lt_trans
  ↳   (Nat.lt_succ_self h) hm), hw)
67     else
68       exact hu N hN
69   · intro hr N _; exact hr N
70
71 /- A crossed row (its value supplied whole) satisfies both
  ↳ parts under every cut. -/
72 theorem crossed_row_both_parts (Inst P cut : α → Prop) (h :
  ↳ Row Inst P) :
73   UnicornOut Inst P cut ∧ RealIn Inst P cut :=
74   {fun x hx _ => h x hx, fun x hx _ => h x hx}
75
76 /- Under the empty cut the Real part is vacuous and the
  ↳ Unicorn part is the whole value. This
77   theorem is about the empty cut and nothing else; it does
  ↳ not say that a row admits no other
78   cut, which is an assessment of a row's literature and
  ↳ never a theorem. -/
79 theorem empty_cut_vacuous (Inst P : α → Prop) :
80   RealIn Inst P (fun _ => False) ∧ (UnicornOut Inst P (fun
  ↳   _ => False) ↔ Row Inst P) :=
81   {fun _ h => h.elim, {fun hu x hx => hu x hx {fun h =>
  ↳   h}, fun h x hx _ => h x hx}}
82
83 /-! ## II. Substantive Real parts, and the Unicorn part as a
  ↳ set of counterexamples -/
84
85 /- A cut is inhabited on the row when some instance lies
  ↳ inside it. -/
86 def Inhabited (Inst cut : α → Prop) : Prop := ∃ x, Inst x ∧
  ↳ cut x
87 /- A substantive Real part: an inhabited cut with its Real
  ↳ part. The empty cut never has one. -/
88 def SubstantiveRealIn (Inst P cut : α → Prop) : Prop :=
  ↳ Inhabited Inst cut ∧ RealIn Inst P cut
89
90 theorem empty_cut_not_substantive (Inst P : α → Prop) :
91   ¬ SubstantiveRealIn Inst P (fun _ => False) :=
92   fun (⟦_, _⟧, h, _) => h
93
94 /- A counterexample outside the cut: an instance outside it
  ↳ that fails the property. The
95   Unicorn part, read as a set, is the set of these; the
  ↳ Unicorn proposition says it is empty. -/
96 def CounterOut (Inst P cut : α → Prop) (x : α) : Prop :=
  ↳ Inst x ∧ ¬ cut x ∧ ¬ P x
97
98 theorem no_counter_of_unicorn (Inst P cut : α → Prop) (hu :
  ↳ UnicornOut Inst P cut) :
99   ∀ x, ¬ CounterOut Inst P cut x :=
100   fun x (hx, hc, hp) => hp (hu x hx hc)
101

```

```

102 theorem unicorn_of_no_counter (Inst P cut :  $\alpha \rightarrow \text{Prop}$ )
103    $\hookrightarrow$  [DecidablePred P]
104   (h :  $\forall x, \neg \text{CounterOut Inst P cut } x$ ) : UnicornOut Inst P
105    $\hookrightarrow$  cut := by
106   intro x hx hc
107   by_cases hp : P x
108   · exact hp
109   · exact absurd (hx, hc, hp) (h x)
110
111 /-- A crossed row has no counterexample outside any cut: its
112    $\hookrightarrow$  Unicorn set is empty. -/
113 theorem crossed_row_no_counter (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) (h :
114    $\hookrightarrow$  Row Inst P) :
115    $\forall x, \neg \text{CounterOut Inst P cut } x$  :=
116   no_counter_of_unicorn Inst P cut (crossed_row_both_parts
117    $\hookrightarrow$  Inst P cut h).1
118
119 /-! ## III. The standing map: fusion lowers a proved part to
120    $\hookrightarrow$  open, as a declared rule -/
121 /-- Two standings: proved, or open. -/
122 inductive Standing | proved | opn
123   deriving DecidableEq, Repr
124
125 /-- The standing of a conjunction is the weaker standing of
126    $\hookrightarrow$  its conjuncts: the program's grade
127   law, declared here as the rule the fusion sentence is
128    $\hookrightarrow$  read under. -/
129 def Standing.join : Standing  $\rightarrow$  Standing  $\rightarrow$  Standing
130   | .proved, .proved => .proved
131   | _, _ => .opn
132
133 /-- Under the declared rule, a proved part joined to an open
134    $\hookrightarrow$  part stands open; only two proved
135   parts stand proved. The theorem records the rule; the
136    $\hookrightarrow$  lowering is the rule's, not the kernel's. -/
137 theorem fusion_lowers : Standing.join .proved .opn = .opn  $\wedge$ 
138    $\hookrightarrow$  Standing.join .opn .proved = .opn  $\wedge$ 
139   Standing.join .proved .proved = .proved := {rfl, rfl,
140    $\hookrightarrow$  rfl}
141
142 /-! ## IV. The gap-to-interval bridge, for the Legendre row
143    $\hookrightarrow$  -/
144
145 /-- a - q < a - p when p < q and p < a; proved by induction
146    $\hookrightarrow$  to keep the file free of propext. -/
147 theorem sub_lt_sub_of_lt :  $\forall (a p q : \text{Nat}), p < a \rightarrow p < q \rightarrow$ 
148    $\hookrightarrow$  a - q < a - p
149   | 0, _, _, h, _ => absurd h (Nat.not_lt_zero _)
150   | a + 1, 0, 0, _, hpq => absurd hpq (Nat.lt_irrefl 0)
151   | a + 1, 0, q + 1, _, _ => by
152     rw [Nat.succ_sub_succ, Nat.sub_zero]
153     exact Nat.lt_succ_of_le (Nat.sub_le a q)
154   | _, + 1, p + 1, 0, _, hpq => absurd hpq (Nat.not_lt_zero
155    $\hookrightarrow$  _)
156   | a + 1, p + 1, q + 1, hpa, hpq => by
157     rw [Nat.succ_sub_succ, Nat.succ_sub_succ]
158     exact sub_lt_sub_of_lt a p q (Nat.lt_of_succ_lt_succ
159    $\hookrightarrow$  hpa) (Nat.lt_of_succ_lt_succ hpq)
160
161 /-- From a bound g on the gap after every element of Q up to
162    $\hookrightarrow$  X, and an element of Q at or below
163   a, an element of Q lies in (a, a + g], provided a  $\leq$  X. Q
164    $\hookrightarrow$  is any predicate; the primes are the
165   instance. Recursion on a - p. -/
166 theorem elem_in_gap (Q : Nat  $\rightarrow$  Prop) (g X : Nat)
167   (hgap :  $\forall p, Q p \rightarrow p \leq X \rightarrow \exists q, Q q \wedge p < q \wedge q \leq p + g$ )
168   (a : Nat) (ha : a  $\leq$  X) (p : Nat) (hp : Q p) (hpa : p  $\leq$ 
169    $\hookrightarrow$  a) :
170    $\exists q, Q q \wedge a < q \wedge q \leq a + g$  := by
171   obtain (q, hq, hpq, hqg) := hgap p hp (Nat.le_trans hpa
172    $\hookrightarrow$  ha)
173   if hqa : a < q then
174     exact (q, hq, hqa, Nat.le_trans hqg
175    $\hookrightarrow$  (Nat.add_le_add_right hpa g))
176   else
177     have hqa' : q  $\leq$  a := Nat.le_of_not_lt hqa
178     have hpa' : p < a := Nat.lt_of_lt_of_le hpq hqa'
179     exact elem_in_gap Q g X hgap a ha q hq hqa'
180   termination_by a - p
181   decreasing_by exact sub_lt_sub_of_lt a p q hpa' hpq
182
183 theorem sq_succ (n : Nat) : (n + 1) * (n + 1) = n * n + 2 *
184    $\hookrightarrow$  n + 1 := by
185   rw [Nat.mul_succ, Nat.succ_mul, Nat.two_mul, Nat.add_assoc]
186    $\hookrightarrow$  (n * n) n (n + 1),
187    $\hookrightarrow$  Nat.add_assoc n n 1,  $\hookrightarrow$  Nat.add_assoc (n * n) (n + n)
188    $\hookrightarrow$  1]
189
190 /-- THE LEGENDRE BRIDGE: if every gap between consecutive
191    $\hookrightarrow$  elements of Q up to X is at most 1476,
192   Q holds at 2,  $n \geq 738$ , and  $(n+1)^2 \leq X$ , then Q has an
193    $\hookrightarrow$  element strictly between  $n^2$  and  $(n+1)^2$ .
194   The gap bound is the literature's [Oliveira e Silva,
195    $\hookrightarrow$  Herzog and Pardi 2014] with Q the primes
196   and  $X = 4 \cdot 10^{18}$ ; the bridge is the kernel's. -/
197 theorem legendre_of_gap (Q : Nat  $\rightarrow$  Prop) (X : Nat)
198   (hgap :  $\forall p, Q p \rightarrow p \leq X \rightarrow \exists q, Q q \wedge p < q \wedge q \leq p +$ 
199    $\hookrightarrow$  1476)
200   (h2 : Q 2) (n : Nat) (hn : 738  $\leq$  n) (hX : (n + 1) * (n +
201    $\hookrightarrow$  1)  $\leq$  X) :
202    $\exists q, Q q \wedge n * n < q \wedge q < (n + 1) * (n + 1)$  := by
203   have hsq := sq_succ n
204   have h738 : 738  $\leq$  n * n := Nat.le_trans hn
205    $\hookrightarrow$  (Nat.le_mul_of_pos_right n (Nat.lt_of_lt_of_le (by
206    $\hookrightarrow$  decide) hn))
207   have hnn : 2  $\leq$  n * n := Nat.le_trans (by decide) h738
208   have ha : n * n  $\leq$  X := Nat.le_trans (Nat.mul_le_mul
209    $\hookrightarrow$  (Nat.le_succ n) (Nat.le_succ n)) hX
210   obtain (q, hq, hlt, hle) := elem_in_gap Q 1476 X hgap (n *
211    $\hookrightarrow$  n) ha 2 hnn
212   have hgapn : 1476 < 2 * n + 1 :=
213     Nat.lt_of_lt_of_le (by decide) (Nat.add_le_add_right
214      $\hookrightarrow$  (Nat.mul_le_mul_left 2 hn) 1)
215   have hq2 : q < (n + 1) * (n + 1) := by
216     rw [hsq, Nat.add_assoc]
217     exact Nat.lt_of_le_of_lt hle (Nat.add_lt_add_left hgapn
218      $\hookrightarrow$  (n * n))
219   exact (q, hq, hlt, hq2)
220
221 /-! ## V. Row specifications: the exact tuple ( $\alpha$ , Inst, P,
222    $\hookrightarrow$  cut) of the arithmetic height-cut rows,
223   with primality and the properties defined in core Lean,
224    $\hookrightarrow$  and the Real part at a small height
225   computed in the kernel. The literature's heights are
226    $\hookrightarrow$  cited in the paper; none is recomputed
227   here. -/
228
229 /-- Trial-division primality, decidable. -/
230 def isPrime (n : Nat) : Bool := 2  $\leq$  n && (List.range n).all
231    $\hookrightarrow$  (fun d => d < 2 || n % d != 0)
232
233 /-- Goldbach at n: n is even and at least 4 implies n is a
234    $\hookrightarrow$  sum of two primes. -/
235 def goldbachOK (n : Nat) : Bool :=
236   n < 4 || n % 2 == 1 || (List.range (n + 1)).any (fun p =>
237    $\hookrightarrow$  isPrime p && isPrime (n - p))
238
239 /-- Legendre at n: a prime strictly between  $n^2$  and  $(n+1)^2$ .
240    $\hookrightarrow$  -/
241 def legendreOK (n : Nat) : Bool :=
242   n == 0 || (List.range ((n + 1) * (n + 1))).any (fun q => n
243    $\hookrightarrow$  * n < q && isPrime q)
244
245 /-- Odd perfect at n: n is not an odd perfect number. -/
246 def sigma (n : Nat) : Nat := ((List.range (n + 1)).filter
247    $\hookrightarrow$  (fun d => d != 0 && n % d == 0)).foldl (· + ·) 0
248 def oddPerfectOK (n : Nat) : Bool := n == 0 || n % 2 == 0 ||
249    $\hookrightarrow$  sigma n != 2 * n
250
251 /-- Collatz at n: the orbit reaches 1 within a step budget
252    $\hookrightarrow$  of 200. -/
253 def collatzStep (n : Nat) : Nat := if n % 2 == 0 then n / 2
254    $\hookrightarrow$  else 3 * n + 1
255 def collatzReaches : Nat  $\rightarrow$  Nat  $\rightarrow$  Bool
256   | _, 0 => false
257   | n, fuel + 1 => n == 1 || collatzReaches (collatzStep n)
258    $\hookrightarrow$  fuel
259 def collatzOK (n : Nat) : Bool := n == 0 || collatzReaches n
260    $\hookrightarrow$  200
261
262 /-- Erdős–Straus at n:  $4/n = 1/x + 1/y + 1/z$  with  $x \leq y \leq z$ 
263    $\hookrightarrow$   $\leq n^2$ , searched. -/
264 def erdosStrausOK (n : Nat) : Bool :=
265   n < 2 || (List.range (n * n + 1)).any (fun x => x != 0 &&
266    $\hookrightarrow$  (List.range (n * n + 1)).any (fun y => x  $\leq$  y &&

```

```

213 (List.range (n * n + 1)).any (fun z => y ≤ z && 4 * x
    ↳ * y * z == n * (y * z + x * z + x * y)))
214 /-- Beal on the box (A, B ≤ a; x, y, z ≤ e): no coprime
    ↳ solution of A^x + B^y = C^z. -/
215 def gcdF : Nat → Nat → Nat → Nat
216 | 0, a, _ => a
217 | fuel + 1, a, b => if b == 0 then a else gcdF fuel b (a %
    ↳ b)
218 def gcd' (a b : Nat) : Nat := gcdF (a + b + 1) a b
219 def bealOK (a e : Nat) : Bool :=
220 (List.range (a + 1)).all (fun A => (List.range (a +
    ↳ 1)).all (fun B =>
221   A == 0 || B == 0 || gcd' A B != 1 ||
222   (List.range (e + 1)).all (fun x => (List.range (e +
    ↳ 1)).all (fun y =>
223     x < 3 || y < 3 ||
224     (List.range (A ^ x + B ^ y + 1)).all (fun C =>
225       C < 2 || (List.range (e + 1)).all (fun z => z < 3 ||
    ↳ C ^ z != A ^ x + B ^ y))))))
226
227 /-- A bounded universal computed as a structural check is
    ↳ the Real part under the height cut. -/
228 def allUpTo (f : Nat → Bool) : Nat → Bool
229 | 0 => f 0
230 | h + 1 => f (h + 1) && allUpTo f h
231
232 theorem allUpTo_spec (f : Nat → Bool) : ∀ h, allUpTo f h =
    ↳ true → ∀ n, n ≤ h → f n = true
233 | 0, hall, n, hn => by
234   have h0 : n = 0 := Nat.eq_zero_of_le_zero hn
235   subst h0; exact hall
236 | h + 1, hall, n, hn => by
237   have hsplit : f (h + 1) = true ∧ allUpTo f h = true :=
    ↳ by
238     have hall' : (f (h + 1) && allUpTo f h) = true :=
    ↳ hall
239     revert hall'
240     generalize f (h + 1) = a
241     generalize allUpTo f h = b
242     cases a <|> cases b <|> intro hab <|>
243       first | exact (rfl, rfl) | exact Bool.noConfusion
    ↳ hab
244     cases Nat.lt_or_eq_of_le hn with
245     | inl hlt => exact allUpTo_spec f h hsplit.2 n
    ↳ (Nat.le_of_lt_succ hlt)
246     | inr heq => subst heq; exact hsplit.1
247
248 theorem real_of_all_range (f : Nat → Bool) (h : Nat) (hall :
    ↳ allUpTo f h = true) :
249   RealIn (fun _ => True) (fun n => f n = true) (fun n => n
    ↳ ≤ h) :=
250   fun n _ hn => allUpTo_spec f h hall n hn
251
252 set_option maxRecDepth 100000 in
253 theorem goldbach_real_60 : RealIn (fun _ => True) (fun n =>
    ↳ goldbachOK n = true) (fun n => n ≤ 60) :=
254   real_of_all_range goldbachOK 60 (by decide)
255 set_option maxRecDepth 100000 in
256 theorem legendre_real_8 : RealIn (fun _ => True) (fun n =>
    ↳ legendreOK n = true) (fun n => n ≤ 8) :=
257   real_of_all_range legendreOK 8 (by decide)
258 set_option maxRecDepth 100000 in
259 theorem oddperfect_real_60 : RealIn (fun _ => True) (fun n
    ↳ => oddPerfectOK n = true) (fun n => n ≤ 60) :=
260   real_of_all_range oddPerfectOK 60 (by decide)
261 set_option maxRecDepth 100000 in
262 theorem collatz_real_60 : RealIn (fun _ => True) (fun n =>
    ↳ collatzOK n = true) (fun n => n ≤ 60) :=
263   real_of_all_range collatzOK 60 (by decide)
264 set_option maxRecDepth 100000 in
265 theorem erdosstrauss_real_6 : RealIn (fun _ => True) (fun n
    ↳ => erdosStraussOK n = true) (fun n => n ≤ 6) :=
266   real_of_all_range erdosStraussOK 6 (by decide)
267 set_option maxRecDepth 100000 in
268 theorem beal_box_3_4 : bealOK 3 4 = true := by decide
269 set_option maxRecDepth 100000 in
270 theorem twin_real_100 : RealIn (fun _ => True) (fun N => ∃
    ↳ m, N < m ∧ (isPrime m && isPrime (m + 2)) = true) (fun N
    ↳ => N ≤ 100) :=
271   real_of_one_witness (fun m => (isPrime m && isPrime (m +
    ↳ 2)) = true) 100 101 (by decide) (by decide)
272 set_option maxRecDepth 100000 in
273 def isMersenne (q : Nat) : Bool := (List.range 20).any (fun
    ↳ m => 2 ^ m - 1 == q)
274 theorem mersenne_real_100 : RealIn (fun _ => True) (fun N =>
    ↳ ∃ q, N < q ∧ (isPrime q && isMersenne q) = true) (fun N
    ↳ => N ≤ 100) :=
275   real_of_one_witness (fun q => (isPrime q && isMersenne q)
    ↳ = true) 100 127 (by decide) (by decide)
276
277 /-- ## VII. The Unicorn part is closed to derivation; the
    ↳ aperture is one bit wide; the supply
278   side is silent at the register -/
279
280 /-- THE FORMAL BLOCK OF THE UNICORN PART. Let τ act on the
    ↳ instances, let p be a record even at a
281   seat x (p (τ x) = p x), and let d be the decision of the
    ↳ property, odd at x (d (τ x) = !d x).
282   If the seat and its partner lie outside the cut, then no
    ↳ reading g of the record agrees with
283   the decision even on the Unicorn region alone. The
    ↳ Unicorn part is closed to every even
284   register, and the closure is a theorem, not a state of
    ↳ the literature. -/
285 theorem unicorn_block {β : Type} (cut : α → Prop) (τ : α →
    ↳ α) (p : α → β) (d : α → Bool)
286 (x : α) (hout : ¬ cut x) (hout' : ¬ cut (τ x)) (hp : p
    ↳ (τ x) = p x) (hd : d (τ x) = !d x) :
287   ¬ ∃ g : β → Bool, ∀ y, ¬ cut y → g (p y) = d y := by
288   intro (g, hg)
289   have h1 := hg x hout
290   have h2 := hg (τ x) hout'
291   rw [hp, hd, h1] at h2
292   revert h2
293   cases d x <|> intro h2 <|> exact Bool.noConfusion h2
294
295 /-- THE APERTURE, ONE BIT WIDE. At a seat where the decision
    ↳ is odd, one odd bit s at the seat
296   would decide the decision on the seat and its partner
    ↳ through a calibration c that exists
297   and is unique. This is the width of the aperture, a
    ↳ theorem; it says nothing of whether
298   anything passes through it. -/
299 theorem aperture_one_bit_wide (τ : α → α) (s d : α → Bool)
    ↳ (x : α)
300 (hs : s (τ x) = !s x) (hd : d (τ x) = !d x) :
301   ∃ c : Bool, (d x = xor (s x) c ∧ d (τ x) = xor (s (τ x))
    ↳ c) ∧
302   ∀ c' : Bool, (d x = xor (s x) c' ∧ d (τ x) = xor (s (τ
    ↳ x)) c') → c' = c :=
303   (xor (d x) (s x),
304     (by cases s x <|> cases d x <|> rfl,
305       by rw [hs, hd]; cases s x <|> cases d x <|> rfl),
306     by
307       intro c' hc
308       obtain (h1, -) := hc
309       generalize hsx : s x = sv
310       generalize hdx : d x = dv
311       rw [hsx, hdx] at h1
312       cases sv <|> cases dv <|> cases c' <|> first | rfl |
    ↳ exact absurd h1 (by decide))
313
314 /-- A true premise decides nothing: conditioning the Unicorn
    ↳ part on existence, or on any
315   inhabited premise, leaves it exactly where it was. The
    ↳ route from existence to the Unicorn
316   part is closed, on every row, by the shape of
    ↳ implication. -/
317 theorem premise_adds_nothing (A Q : Prop) (ha : A) : (A → Q)
    ↳ → Q :=
318   (fun h => h ha, fun hq _ => hq)
319
320 /-- NO CURE. Restricting the rows to a class C on which a
    ↳ premise is to decide the Unicorn part
321   changes nothing: on every class the premise decides
    ↳ exactly what already held on the class. -/
322 theorem no_cure_unicorn {Frame : Type} (A : Prop) (ha : A)
    ↳ (C U : Frame → Prop) :
323   (∀ X, C X → A → U X) → (∀ X, C X → U X) :=
324   (fun h X hc => h X hc ha, fun h X hc _ => h X hc)
325

```

```

326 /-- THE SUPPLY SIDE IS SILENCE, AT THE REGISTER. Over one
    ↳ even record, both orientations of the
327 decision at the seat are equally refused to every
    ↳ reading: the register cannot say which
328 way the seat lies, so it cannot say what a supply would
    ↳ bring, nor that one exists, nor
329 that none does. Any sentence about the supply side
    ↳ derived from the record would be a
330 reading the wall refuses. -/
331 theorem supply_side_silent {β : Type} (cut : α → Prop) (τ :
    ↳ α → α) (ρ : α → β) (d : α → Bool)
332 (x : α) (hout : ¬ cut x) (hout' : ¬ cut (τ x)) (hp : p
    ↳ (τ x) = p x) (hd : d (τ x) = !d x) :
333 (¬ ∃ g : β → Bool, ∀ y, ¬ cut y → g (p y) = d y) ∧
334 (¬ ∃ g : β → Bool, ∀ y, ¬ cut y → g (p y) = !d y) ∧
335 (!d x) ≠ d x :=
336 (unicorn_block cut τ p d x hout hout' hp hd,
337 unicorn_block cut τ p (fun y => !d y) x hout hout' hp (by
338 show (!d (τ x)) = !(d x)
339 rw [hd]),
340 by cases d x <=> decide)
341
342 /-- THE THREE BITS AS ONE TERM · THE REGISTER'S PROOF,
    ↳ COMPLETE. Under a certificate for the cut
343 and the seat data (a seat outside the cut with its
    ↳ partner, an even record, an odd decision,
344 an odd supply), everything a register can prove about
    ↳ the row is proved in one conjunction:
345 the row divides exactly (bit one), the Real part holds
    ↳ (bit two), every reading of the even
346 register is refused on the Unicorn region (bit three,
    ↳ refused), and one supplied bit
347 calibrates the seat uniquely (the door). The row is not
    ↳ thereby proved; nothing derivable
348 about it is left undervived. -/
349 theorem register_proof_complete {β : Type} (Inst P cut : α →
    ↳ Prop) [DecidablePred cut]
350 (c : Certificate Inst P cut) (τ : α → α) (ρ : α → β) (d
    ↳ s : α → Bool) (x : α)
351 (hout : ¬ cut x) (hout' : ¬ cut (τ x)) (hp : p (τ x) = p
    ↳ x)
352 (hd : d (τ x) = !d x) (hs : s (τ x) = !s x) :
353 (Row Inst P → RealIn Inst P cut ∧ UnicornOut Inst P cut)
    ↳ → A
354 RealIn Inst P cut ∧
355 (¬ ∃ g : β → Bool, ∀ y, ¬ cut y → g (p y) = d y) ∧
356 ∃ k : Bool, (d x = xor (s x) k ∧ d (τ x) = xor (s (τ x))
    ↳ k) ∧
357 ∀ k' : Bool, (d x = xor (s x) k' ∧ d (τ x) = xor (s (τ
    ↳ x)) k') → k' = k :=
358 (row_iff_real_and_unicorn Inst P cut, real_of_certificate
    ↳ → Inst P cut c,
359 unicorn_block cut τ p d x hout hout' hp hd,
    ↳ aperture_one_bit_wide τ s d x hs hd)
360
361 /-! ## VI. The twenty-three rows, typed by shape, the census
    ↳ of the assignments kernel-checked -/
362
363 /-- The shape of a row's cut, as assigned: a height (a
    ↳ bounded universal with a certificate
364 per height, or an infinitude with one witness), a regime
    ↳ (a dimension, degree, rank, or class
365 proved by a cited theorem), no cut (no inhabited cut
    ↳ with a Real part in the literature, an
366 assessment), or crossed (the value supplied whole). The
    ↳ assignments are data; the kernel
367 checks their census. -/
368 inductive Shape | height | regime | noCut | crossed
369 deriving DecidableEq, Repr
370
371 def rows23 : List (String × Shape) :=
372 [ ("P versus NP", .regime), ("Riemann Hypothesis",
    ↳ .height), ("Navier–Stokes", .regime),
373 ("Yang–Mills", .regime), ("Hodge", .regime), ("BSD",
    ↳ .regime), ("Poincaré", .crossed),
374 ("Goldbach", .height), ("Twin primes", .height),
    ↳ ("Legendre", .height), ("abc", .noCut),
375 ("Jacobian", .regime), ("Mersenne primes", .height),
    ↳ ("Odd perfect numbers", .height),
376 ("Smooth 4D Poincaré", .regime), ("Collatz", .height),
    ↳ ("Hadwiger", .regime),

```

```

377 ("Sunflower", .noCut), ("Erdős–Straus", .height),
    ↳ ("Beal", .height),
378 ("Invariant subspace", .regime), ("Hilbert's 16th, second
    ↳ part", .regime),
379 ("Lindelöf / Montgomery PCC", .regime)]
380
381 def countShape (s : Shape) : Nat := (rows23.filter (fun r =>
    ↳ r.2 == s)).length
382
383 /-- THE CENSUS OF THE ASSIGNMENTS: twenty-three rows; nine
    ↳ height-cut, eleven regime-cut, two
384 no-cut, one crossed; twenty assigned a Real-part-bearing
    ↳ shape. -/
385 theorem census :
386 rows23.length = 23 ∧ countShape .height = 9 ∧ countShape
    ↳ .regime = 11 ∧
387 countShape .noCut = 2 ∧ countShape .crossed = 1 ∧
388 countShape .height + countShape .regime = 20 := by
    ↳ decide
389
390 def hasRealPart (s : Shape) : Bool := s == .height || s ==
    ↳ .regime
391 theorem real_part_rows : (rows23.filter (fun r =>
    ↳ hasRealPart r.2)).length = 20 := by decide
392
393 end RowSplit
394
395 #print axioms RowSplit.row_iff_real_and_unicorn
396 #print axioms RowSplit.fused_is_unicorn_given_real
397 #print axioms RowSplit.real_of_certificate
398 #print axioms RowSplit.real_of_one_witness
399 #print axioms RowSplit.unicorn_equiv_row_of_infinitude
400 #print axioms RowSplit.crossed_row_both_parts
401 #print axioms RowSplit.empty_cut_vacuous
402 #print axioms RowSplit.empty_cut_not_substantive
403 #print axioms RowSplit.no_counter_of_unicorn
404 #print axioms RowSplit.unicorn_of_no_counter
405 #print axioms RowSplit.crossed_row_no_counter
406 #print axioms RowSplit.fusion_lowers
407 #print axioms RowSplit.sub_lt_sub_of_lt
408 #print axioms RowSplit.elem_in_gap
409 #print axioms RowSplit.legendre_of_gap
410 #print axioms RowSplit.allUpTo_spec
411 #print axioms RowSplit.real_of_all_range
412 #print axioms RowSplit.goldbach_real_60
413 #print axioms RowSplit.legendre_real_8
414 #print axioms RowSplit.oddpfect_real_60
415 #print axioms RowSplit.collatz_real_60
416 #print axioms RowSplit.erdosstrauss_real_6
417 #print axioms RowSplit.beal_box_3_4
418 #print axioms RowSplit.twin_real_100
419 #print axioms RowSplit.mersenne_real_100
420 #print axioms RowSplit.unicorn_block
421 #print axioms RowSplit.aperture_one_bit_wide
422 #print axioms RowSplit.premise_adds_nothing
423 #print axioms RowSplit.no_cure_unicorn
424 #print axioms RowSplit.supply_side_silent
425 #print axioms RowSplit.register_proof_complete
426 #print axioms RowSplit.census
427 #print axioms RowSplit.real_part_rows

```

Kernel audit, verbatim: lean Row_Split.lean, exit 0, no warnings.

```

1 'RowSplit.row_iff_real_and_unicorn' does not depend on any
  ↳ axioms
2 'RowSplit.fused_is_unicorn_given_real' does not depend on
  ↳ any axioms
3 'RowSplit.real_of_certificate' does not depend on any axioms
4 'RowSplit.real_of_one_witness' does not depend on any axioms
5 'RowSplit.unicorn_equiv_row_of_infinitude' does not depend
  ↳ on any axioms
6 'RowSplit.crossed_row_both_parts' does not depend on any
  ↳ axioms
7 'RowSplit.empty_cut_vacuous' does not depend on any axioms
8 'RowSplit.empty_cut_not_substantive' does not depend on any
  ↳ axioms
9 'RowSplit.no_counter_of_unicorn' does not depend on any
  ↳ axioms

```

```

10 'RowSplit.unicorn_of_no_counter' does not depend on any
   ↪ axioms
11 'RowSplit.crossed_row_no_counter' does not depend on any
   ↪ axioms
12 'RowSplit.fusion_lowers' does not depend on any axioms
13 'RowSplit.sub_lt_sub_of_lt' does not depend on any axioms
14 'RowSplit.elem_in_gap' does not depend on any axioms
15 'RowSplit.legendre_of_gap' does not depend on any axioms
16 'RowSplit.allUpTo_spec' does not depend on any axioms
17 'RowSplit.real_of_all_range' does not depend on any axioms
18 'RowSplit.goldbach_real_60' does not depend on any axioms
19 'RowSplit.legendre_real_8' does not depend on any axioms
20 'RowSplit.oddperfect_real_60' does not depend on any axioms
21 'RowSplit.collatz_real_60' does not depend on any axioms
22 'RowSplit.erdosstrauss_real_6' does not depend on any axioms
23 'RowSplit.beal_box_3_4' does not depend on any axioms
24 'RowSplit.twin_real_100' does not depend on any axioms
25 'RowSplit.mersenne_real_100' does not depend on any axioms
26 'RowSplit.unicorn_block' does not depend on any axioms
27 'RowSplit.aperture_one_bit_wide' does not depend on any
   ↪ axioms
28 'RowSplit.premise_adds_nothing' does not depend on any
   ↪ axioms
29 'RowSplit.no_cure_unicorn' does not depend on any axioms
30 'RowSplit.supply_side_silent' does not depend on any axioms
31 'RowSplit.register_proof_complete' does not depend on any
   ↪ axioms
32 'RowSplit.census' does not depend on any axioms
33 'RowSplit.real_part_rows' does not depend on any axioms

```

APPENDIX B: ZFC_UNDER_RAM.LEAN AND ITS AUDIT

SHA-256 963e3ed8f630975f8604fddc010f154fd4f75f5ae6e193c95f64d4486089a741, Lean 4.19.0, core only, 161 lines; sixteen dependency sets, every one axiom-free; seated in the Master Codex as APEX-PSP-ZFC-UNDER-RAM-01. The file encodes no axiom of ZFC; its theory is abstract and ZFC is the named instance.

```

1 /-
2 ZFC_Under_RAM.lean · ZFC subsumed under RAM by placement,
   ↪ the RA-RAM bridge, and the round trip,
3 in one file. Core Lean 4.19.0, no library, no sorry, no
   ↪ user-declared axiom. Forged 2026-09-25 from
4 RAM_ZFC_Placement.lean, RA_to_ZFC_Chain.lean, and
   ↪ ZFC_RA_RoundTrip.lean (lean/, 2026-09-24).
5
6 The claim, exactly. RAM does not derive ZFC's axioms and
   ↪ does not replace ZFC's proofs. It LOCATES
7 ZFC: computation is the rung (L3m), provability the ladder
   ↪ (L2m), truth in the intended world the
8 Ground (L1m), and rung ≤ ladder ≤ Ground under two named
   ↪ premises,  $\Sigma_1$ -completeness (a theorem of
9 arithmetic, cited) and soundness (which no consistent theory
   ↪ proves of itself, carried openly).
10 Across the RA-RAM bridge the root reaches every world of ZFC
   ↪ as presence and carries no keyed
11 sentence; the round trip returns every sentence unchanged;
   ↪ and every step holds with the root
12 replaced by a bare computation arrow, so the root
   ↪ contributes the act and never a line of content.
13  $\Delta m = 0$ .
14 -/
15 namespace ZFCUnderRAM
16
17 /-! ## 1 · The placement: rung ≤ ladder ≤ Ground -/
18
19 structure Theory where
20   Sent : Type
21   Prov : Sent → Prop      -- L2m, the ladder
22   Comp : Sent → Prop      -- L3m, the rung: sentences
   ↪ settled by finite computation

```

```

23 True_ : Sent → Prop      -- L1m, the Ground: truth in
   ↪ the intended world
24
25 /-  $\Sigma_1$ -completeness, abstractly: what finite computation
   ↪ settles, the theory proves. -/
26 def RungOnLadder (T : Theory) : Prop := ∀ s, T.Comp s →
   ↪ T.Prov s
27 /- Soundness: what the theory proves holds in its intended
   ↪ world. Carried openly, never derived. -/
28 def Sound (T : Theory) : Prop := ∀ s, T.Prov s → T.True_ s
29
30 /- THE PLACEMENT. Under the two premises every theory sits
   ↪ in the strata in order. -/
31 theorem ram_places_theory (T : Theory) (hR : RungOnLadder T)
   ↪ (hS : Sound T) :
32   (∀ s, T.Comp s → T.Prov s) ∧ (∀ s, T.Prov s → T.True_ s)
   ↪ ∧ (∀ s, T.Comp s → T.True_ s) :=
33   (hR, hS, fun s h => hS s (hR s h))
34
35 /- Soundness is load-bearing: an unsound theory breaks
   ↪ ladder ≤ Ground. -/
36 def unsound : Theory := (Bool, fun _ => True, fun _ =>
   ↪ False, fun b => b = true)
37 theorem soundness_is_load_bearing : RungOnLadder unsound ∧ ~
   ↪ Sound unsound := by
38   refine (fun _ h => h.elim, fun h => ?_)
39   have := h false trivial
40   cases this
41
42 /- The Ground exceeds the ladder: a sound theory can leave
   ↪ a truth unproved (Gödel's region). -/
43 def incomplete : Theory := (Bool, fun b => b = true, fun _
   ↪ => False, fun _ => True)
44 theorem ground_exceeds_ladder :
45   Sound incomplete ∧ incomplete.True_ false ∧ ~
   ↪ incomplete.Prov false :=
46   (fun _ => trivial, trivial, fun h => Bool.noConfusion h)
47
48 /- Independent axioms are separate bits: every combination
   ↪ of their values is realized. -/
49 theorem independent_axioms_are_separate_bits :
50   ∀ a b : Bool, ∃ w : Bool × Bool, w.1 = a ∧ w.2 = b :=
51   fun a b => ((a, b), rfl, rfl)
52
53 /- SUBSUMPTION BY PLACEMENT, the verdict of section 1. -/
54 theorem ram_subsumes_by_placement :
55   (∀ T : Theory, RungOnLadder T → Sound T → (∀ s, T.Comp s
   ↪ → T.True_ s)) ∧
56   (RungOnLadder unsound ∧ ~ Sound unsound) ∧
57   (Sound incomplete ∧ incomplete.True_ false ∧ ~
   ↪ incomplete.Prov false) :=
58   (fun T hR hS => (ram_places_theory T hR hS).2.2,
   ↪ soundness_is_load_bearing, ground_exceeds_ladder)
59
60 /-! ## 2 · The bridge: the root reaches every world as
   ↪ presence and carries no key -/
61
62 structure Substrate where
63   U : Type
64   ΔE : U → Int
65 def RA (S : Substrate) : Prop := ∀ x : S.U, 0 < S.ΔE x
66 def unitSub : Substrate := (Unit, fun _ => 1)
67 theorem ra_unit : RA unitSub := fun _ => show (0 : Int) < 1
   ↪ by decide
68
69 structure SelfGrounding (R : Prop) where
70   Act : Type
71   anAct : Act
72   instances : Act → R
73
74 /- Upward: every act of proving in any theory grounds the
   ↪ root. -/
75 theorem using_grounds_root (T : Theory) (s : T.Sent) (d :
   ↪ T.Prov s) :
76   Nonempty (SelfGrounding (RA unitSub)) :=
77   (PLift (T.Prov s), (d), fun _ => ra_unit))
78
79 /- Worlds are ways a theory's content could be. A property
   ↪ is keyless when every world has it,
   ↪ keyed when some world lacks it. The bridge carries the
   ↪ keyless and never the keyed. -/

```

```

81 def Keyless {W : Type} (P : W → Prop) : Prop := ∀ w, P w
82 def Keyed {W : Type} (P : W → Prop) : Prop := ∃ w, ¬ P w
83
84 theorem ra_is_keyless {W : Type} : Keyless (fun _ : W => RA
  ↪ unitSub) := fun _ => ra_unit
85 theorem keyless_crosses {W : Type} (P : W → Prop) (hP :
  ↪ Keyless P) : ∀ w, RA unitSub → P w :=
86   fun w _ => hP w
87 theorem ra_decides_no_keyed {W : Type} (P : W → Prop) (hK :
  ↪ Keyed P) :
88   ¬ (∀ w, RA unitSub → P w) :=
89   fun h => let (w, hw) := hK; hw (h w ra_unit)
90
91 /- Choice over ZF, the cited keyed sentence (Gödel 1938,
  ↪ Cohen 1963): two worlds, one each way;
92   the root holds in both and so carries neither Choice nor
  ↪ its denial into ZFC. -/
93 def choiceHolds : Bool → Prop := fun w => w = true
94 theorem choice_is_keyed : Keyed choiceHolds := (false, fun h
  ↪ => Bool.noConfusion h)
95 theorem ra_does_not_cross_choice :
96   (∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
  ↪ choiceHolds false :=
97   (fun _ => ra_unit, rfl, fun h => Bool.noConfusion h)
98
99 /-! ## 3 · The round trip: ZFC → RAM → bridge → RA → bridge
  ↪ → RAM → ZFC is the identity on content -/
100
101 def viaRA {W : Type} (P : W → Prop) : W → Prop := fun w =>
  ↪ RA unitSub ∧ P w
102
103 theorem round_trip_identity {W : Type} (P : W → Prop) (w :
  ↪ W) : viaRA P w ↔ P w :=
104   (fun h => h.2, fun h => (ra_unit, h))
105
106 /- An absolute sentence (the same value in every world, as
  ↪ an arithmetic sentence such as the
107   hypothesis is across models with the standard numbers)
  ↪ returns with that same value. -/
108 theorem absolute_returns_unchanged {W : Type} (P : W → Prop)
  ↪ (hAbs : ∀ w v, P w ↔ P v) (w : W) :
109   ∀ v, viaRA P v ↔ P v :=
110   fun v => (round_trip_identity P v).trans (hAbs v w)
111
112 /- The trip adds presence and nothing else: the root
  ↪ returns identically for a sentence and for
113   its negation, so it tells them apart nowhere. -/
114 theorem trip_adds_presence_only {W : Type} (P : W → Prop) (w
  ↪ : W) :
115   RA unitSub ∧ (viaRA P w ↔ P w) ∧ (viaRA (fun v => ¬ P v)
  ↪ w ↔ ¬ P w) :=
116   (ra_unit, round_trip_identity P w, round_trip_identity
  ↪ (fun v => ¬ P v) w)
117
118 /-! ## 4 · The arrow: any true premise plays the root's
  ↪ part, so the root is idle in the content -/
119
120 theorem any_true_premise_serves (R : Prop) (hR : R) {W :
  ↪ Type} (P : W → Prop) :
121   ((∀ w, P w) → ∀ w, R → P w) ∧ ((∃ w, ¬ P w) → ¬ ∀ w, R →
  ↪ P w) ∧ (∀ w, (R ∧ P w) ↔ P w) :=
122   (fun hP w _ => hP w, fun (w, hw) h => hw (h w hR), fun _
  ↪ => (fun h => h.2, fun h => (hR, h)))
123
124 /-! ## 5 · The verdict, in one theorem -/
125
126 /- ZFC UNDER RAM. Placement under the two named premises
  ↪ with soundness load-bearing and the
127   Ground exceeding the ladder; the root grounded by every
  ↪ act of proof and crossing as presence;
128   no keyed sentence decided, Choice the instance; the
  ↪ round trip the identity on content, adding
129   presence only; and the whole holding for any true
  ↪ premise in the root's place. Subsumption is
  ↪ location, never derivation. -/
130
131 theorem zfc_under_ram :
132   ((∀ T : Theory, RungOnLadder T → Sound T → (∀ s, T.Comp
  ↪ s → T.True s)) ∧
133   (RungOnLadder unsound ∧ ¬ Sound unsound) ∧
134   (Sound incomplete ∧ incomplete.True false ∧ ¬
  ↪ incomplete.Prov false)) ∧

```

```

135   (∀ (T : Theory) (s : T.Sent), T.Prov s → Nonempty
  ↪ (SelfGrounding (RA unitSub))) ∧
136   (∀ {W : Type} (P : W → Prop), Keyed P → ¬ ∀ w, RA
  ↪ unitSub → P w) ∧
137   ((∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
  ↪ choiceHolds false) ∧
138   (∀ {W : Type} (P : W → Prop) (w : W), viaRA P w ↔ P w) ∧
139   (∀ (R : Prop), R → ∀ {W : Type} (P : W → Prop) (w : W),
  ↪ (R ∧ P w) ↔ P w) :=
140   (ram_subsumes_by_placement, using_grounds_root, fun P hK
  ↪ => ra_decides_no_keyed P hK,
141   ra_does_not_cross_choice, fun P w => round_trip_identity
  ↪ P w,
142   fun R hR {W} P w => (any_true_premise_serves R hR (W :=
  ↪ W) P).2.2 w)
143
144 end ZFCUnderRAM
145
146 #print axioms ZFCUnderRAM.ram_places_theory
147 #print axioms ZFCUnderRAM.soundness_is_load_bearing
148 #print axioms ZFCUnderRAM.ground_exceeds_ladder
149 #print axioms
  ↪ ZFCUnderRAM.independent_axioms_are_separate_bits
150 #print axioms ZFCUnderRAM.ram_subsumes_by_placement
151 #print axioms ZFCUnderRAM.using_grounds_root
152 #print axioms ZFCUnderRAM.ra_is_keyless
153 #print axioms ZFCUnderRAM.keyless_crosses
154 #print axioms ZFCUnderRAM.ra_decides_no_keyed
155 #print axioms ZFCUnderRAM.choice_is_keyed
156 #print axioms ZFCUnderRAM.ra_does_not_cross_choice
157 #print axioms ZFCUnderRAM.round_trip_identity
158 #print axioms ZFCUnderRAM.absolute_returns_unchanged
159 #print axioms ZFCUnderRAM.trip_adds_presence_only
160 #print axioms ZFCUnderRAM.any_true_premise_serves
161 #print axioms ZFCUnderRAM.zfc_under_ram

```

Kernel audit, verbatim: lean ZFC_Under_RAM.lean,
exit 0.

```

1 'ZFCUnderRAM.ram_places_theory' does not depend on any
  ↪ axioms
2 'ZFCUnderRAM.soundness_is_load_bearing' does not depend on
  ↪ any axioms
3 'ZFCUnderRAM.ground_exceeds_ladder' does not depend on any
  ↪ axioms
4 'ZFCUnderRAM.independent_axioms_are_separate_bits' does not
  ↪ depend on any axioms
5 'ZFCUnderRAM.ram_subsumes_by_placement' does not depend on
  ↪ any axioms
6 'ZFCUnderRAM.using_grounds_root' does not depend on any
  ↪ axioms
7 'ZFCUnderRAM.ra_is_keyless' does not depend on any axioms
8 'ZFCUnderRAM.keyless_crosses' does not depend on any axioms
9 'ZFCUnderRAM.ra_decides_no_keyed' does not depend on any
  ↪ axioms
10 'ZFCUnderRAM.choice_is_keyed' does not depend on any axioms
11 'ZFCUnderRAM.ra_does_not_cross_choice' does not depend on
  ↪ any axioms
12 'ZFCUnderRAM.round_trip_identity' does not depend on any
  ↪ axioms
13 'ZFCUnderRAM.absolute_returns_unchanged' does not depend on
  ↪ any axioms
14 'ZFCUnderRAM.trip_adds_presence_only' does not depend on any
  ↪ axioms
15 'ZFCUnderRAM.any_true_premise_serves' does not depend on any
  ↪ axioms
16 'ZFCUnderRAM.zfc_under_ram' does not depend on any axioms

```

APPENDIX C: ROWS_AT_THE_APEX.LEAN, CARRIED WHOLE FROM THE PRIOR EDITION

SHA-256 0ce3de5a231554303dfc0982dee4fe7e7fbd
d647a72bbe3366290dc8fd092079, Lean 4.19.0, core
only, 1113 lines.

```

1  /-!
2  # THE SEVEN MILLENNIUM ROWS AND THE SIXTEEN EXTENSION ROWS
3  ↪ AT THE APEX · Rows_At_The_Apex.lean
4  # The spine (Movements 1 to 16) is the Riemann apex file
5  ↪ carried whole; Movements 17 to 21 add the rows.
6  core Lean 4 v4.19.0 · no Mathlib · no sorry · no axiom
7  ↪ declaration.
8
9  WHAT IS PROVED. (1) RA at the constructed domain, computed.
10 ↪ (2) The seat: the binding
11 ↪ involution  $\sigma$  fixes exactly the scalar line, the Return  $i \cdot j \cdot k$ 
12 ↪ lands on it, and the kinetic,
13 ↪ formal, and named seats are one point by rfl: the three
14 ↪ category gaps are identities.
15 (3) The eliminator  $RA \rightarrow RA \rightarrow RAM \rightarrow RH\_formal$  from the RA
16 ↪ witness alone. (4) The spine
17 ↪ carried: the abstract frame  $X = (S, \tau, Z)$ , the line property
18 ↪ L, the bridge halted iff L,
19 ↪ the crossing exact both ways, necessity, the bit is the
20 ↪ hypothesis, the witness row
21 ↪ falsifiable. (5) The aperture, RA-supplied: a present
22 ↪ reader's row is never interior;
23 it is sealed or refused by the bit alone. (6) THE SIMPLE
24 ↪ RULE, EXACT: for every frame,
25 (RA  $\rightarrow L \ X$ )  $\leftrightarrow L \ X$ ; and RA decides L on no frame:  $\neg \forall X, RA \rightarrow$ 
26 ↪ L X, by the two-point model.
27 (7) RH in toto at the apex, one theorem, conditional on RA
28 ↪ and discharged.
29 (8) The root's two formal faces: RAF, closure and no
30 ↪ exterior agent (RAF-2, RAF-C1), and
31 no total self-indexing (RAF-C2, Cantor); RAM, the seat as
32 ↪ the identity cone over the
33 three registers, its apex unique. (9) The equivariant
34 ↪ embedding  $\phi$  of the spine's lattice
35 stage (Plane,  $\tau$ ) into (Q4,  $\sigma$ ):  $\sigma \circ \phi = \phi \circ \tau$  and  $\phi$  carries
36 ↪ Fix( $\tau$ ) onto Fix( $\sigma$ ).
37 (10) The wall executed: no readout of the formal seat equals
38 ↪ the deed bit. (11) The
39 counter-model survives grounding, and the cure theorem: for
40 ↪ every class of frames,
41 existence decides the line property on it exactly when the
42 ↪ line property already holds on it.
43 -/
44 set_option autoImplicit false
45
46 namespace RowsAtTheApex
47
48 /-! ## 1 · The Root Axiom at the constructed domain -/
49
50 inductive UniversePoint : Type where
51 | source
52 deriving DecidableEq, Repr
53
54 def DeltaE : UniversePoint  $\rightarrow$  Int := fun _ => 1
55
56 /- RA: to exist is to actuate. -/
57 def RA : Prop :=  $\forall x : UniversePoint, 0 < DeltaE \ x$ 
58
59 theorem constructed_RA : RA := by
60   intro x
61   cases x
62   ↪ decide
63
64 /-! ## 2 · The seat: Fix( $\sigma$ ) on the quaternions, the Return,
65 ↪ three names, one point -/
66
67 structure Q4 where
68   r : Int
69   i : Int
70   j : Int
71   k : Int
72   deriving DecidableEq, Repr
73
74 def qmul (a b : Q4) : Q4 :=
75   (a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
76    a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
77    a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
78    a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r)
79
80 def qconj (a : Q4) : Q4 := (a.r, -a.i, -a.j, -a.k)

```

```

60
61 /-  $\sigma$ , the binding involution: conjugation. -/
62 def sigma (q : Q4) : Q4 := qconj q
63
64 theorem sigma_binding (q : Q4) : sigma (sigma q) = q := by
65   cases q with
66   | mk r i j k =>
67     change Q4.mk r (-(-i)) (-(-j)) (-(-k)) = Q4.mk r i j k
68     rw [Int.neg_neg, Int.neg_neg, Int.neg_neg]
69
70 /- Fix( $\sigma$ ) is the scalar line, for every quaternion. -/
71 theorem fix_iff_scalar (q : Q4) : sigma q = q  $\leftrightarrow$  (q.i = 0  $\wedge$ 
72 ↪ q.j = 0  $\wedge$  q.k = 0) := by
73   cases q with
74   | mk r i j k =>
75     change (Q4.mk r (-i) (-j) (-k)) = Q4.mk r i j k  $\leftrightarrow$  (i = 0
76 ↪  $\wedge$  j = 0  $\wedge$  k = 0)
77   constructor
78   · intro h
79     injection h with _ hi hj hk
80     exact (by omega, by omega, by omega)
81   · intro h
82     obtain (hi, hj, hk) := h
83     subst hi
84     subst hj
85     subst hk
86     rfl
87
88 /- The Ground: Fix( $\sigma$ ). -/
89 def Ground : Type := { q : Q4 // sigma q = q }
90
91 def qi : Q4 := (0, 1, 0, 0)
92 def qj : Q4 := (0, 0, 1, 0)
93 def qk : Q4 := (0, 0, 0, 1)
94
95 /- The Return: the parse triad through its own cascade,
96 ↪  $i \cdot j \cdot k$ . -/
97 def theReturn : Q4 := qmul (qmul qi qj) qk
98
99 theorem return_is_minus_one : theReturn = (-1, 0, 0, 0) :=
100 ↪ rfl
101
102 theorem return_lands_on_fix : sigma theReturn = theReturn :=
103 ↪ rfl
104
105 def fixProj (q : Q4) : Q4 := (q.r, 0, 0, 0)
106
107 def GammaRA : Q4 := theReturn
108 def GammaRAM : Q4 := fixProj theReturn
109 def GammaRH : Q4 := (-1, 0, 0, 0)
110
111 /- C0, C1, C2: the three category gaps are definitional
112 ↪ identities. -/
113 theorem self_gap_nonexistent : sigma GammaRA = GammaRA :=
114 ↪ rfl
115
116 theorem register_gap_nonexistent : GammaRA = GammaRAM := rfl
117
118 theorem object_gap_nonexistent : GammaRAM = GammaRH := rfl
119
120 def seat : Ground := (GammaRH, rfl)
121
122 /- The reversed triad  $k \cdot j \cdot i$  returns +1, also on the fixed
123 ↪ locus: the sign of the seat
124 point is the orientation of the triad, the one bit the
125 ↪ formal register cannot read.
126 The apex of the identity cone is unique for the ordered
127 ↪ triad; the reversed triad
128 carries its own apex, and the two differ. -/
129 def theReturnOdd : Q4 := qmul (qmul qk qj) qi
130
131 theorem odd_return_is_plus_one : theReturnOdd = (1, 0, 0, 0)
132 ↪ := rfl
133
134 theorem odd_return_on_fix : sigma theReturnOdd =
135 ↪ theReturnOdd := rfl
136
137 theorem seat_points_differ : theReturnOdd  $\neq$  theReturn := by
138 ↪ decide
139
140 theorem seat_on_scalar_line : GammaRH.i = 0  $\wedge$  GammaRH.j = 0
141 ↪  $\wedge$  GammaRH.k = 0 :=
142   (fix_iff_scalar GammaRH).mp seat.2
143
144

```

```

127 /-! ## 3 · The eliminator from the RA witness alone -/
128
129 def RAMFormalGround : Prop := Nonempty Ground
130
131 theorem constructed_RAM : RAMFormalGround := (seat)
132
133 /- The RH formal self: every point of the Ground is on the
   ↳  $\sigma$ -fixed critical locus.
134   It names the seat and says nothing about the zeros of  $\xi$ .
   ↳ -/
135 def RHFormalSelf : Prop :=  $\forall g : \text{Ground}, \text{sigma } g.1 = g.1$ 
136
137 /- Bridge-born orientation, carried in Prop: the kernel
   ↳ registers that orientation was
138   supplied and cannot read which way. -/
139 structure OrientationBit : Prop where
140   direction : True
141   closure : True
142
143 theorem constructed_orientation : OrientationBit :=
   ↳ (trivial, trivial)
144
145 theorem orientation_proof_irrelevant (a b : OrientationBit)
   ↳ : a = b := rfl
146
147 structure RAWitness : Prop where
148   actuates : RA
149   orientation : OrientationBit
150   bridge : RAMFormalGround
151   trisRecursion : OrientationBit  $\rightarrow$  RAMFormalGround  $\rightarrow$ 
   ↳ RHFormalSelf
152
153 theorem constructed_trisRecursion : OrientationBit  $\rightarrow$ 
   ↳ RAMFormalGround  $\rightarrow$  RHFormalSelf :=
154   fun _omega_ram g => g.2
155
156 /- RA, constructed: the only supplied witness. -/
157 theorem mkRA : RAWitness :=
158   (constructed_RA, constructed_orientation, constructed_RAM,
   ↳ constructed_trisRecursion)
159
160 /- THE ELIMINATOR. RA  $\rightarrow$  RA  $\rightarrow$  RAM  $\rightarrow$  RH_formal, Bridge
   ↳ followed by Tris-Recursion. -/
161 theorem RH_formal_chain : RHFormalSelf :=
162   mkRA.trisRecursion mkRA.orientation mkRA.bridge
163
164 /-! ## 4 · The spine, carried: the abstract frame, the
   ↳ bridge, the crossing -/
165
166 def Even { $\alpha : \text{Type}$ } ( $\sigma : \alpha \rightarrow \alpha$ ) ( $f : \alpha \rightarrow \text{Bool}$ ) : Prop :=  $\forall$ 
   ↳ x, f ( $\sigma$  x) = f x
167
168 /- Theorem 4 of the spine: orientation blindness. -/
169 theorem orientation_blind { $\alpha : \text{Type}$ } ( $\sigma : \alpha \rightarrow \alpha$ ) ( $f d : \alpha \rightarrow$ 
   ↳  $\text{Bool}$ ) ( $x : \alpha$ )
170   (he : Even  $\sigma$  f) (ho : d ( $\sigma$  x)  $\neq$  d x) : f  $\neq$  d := by
171   intro h
172   subst h
173   exact ho (he x)
174
175 /- Theorem 5 of the spine: earned freedom is exactly two.
   ↳ -/
176 theorem freedom_is_exactly_two (d :  $\text{Bool} \rightarrow \text{Bool}$ ) (hd :  $\forall x$ ,
   ↳ d (!x) = !d x) :
177   d = (fun x => x)  $\vee$  d = (fun x => !x) := by
178   have hf : d false = !d true := hd true
179   cases ht : d true with
180   | true =>
181     left
182     funext x
183     cases x
184     · exact hf.trans (congrArg (fun b => !b) ht)
185     · exact ht
186   | false =>
187     right
188     funext x
189     cases x
190     · exact hf.trans (congrArg (fun b => !b) ht)
191     · exact ht
192

```

```

193 /- Theorem 6 of the spine: the freedom is spent uniquely.
   ↳ -/
194 theorem freedom_spent_uniquely { $\alpha : \text{Type}$ } ( $\sigma : \alpha \rightarrow \alpha$ ) (s d :
   ↳  $\alpha \rightarrow \text{Bool}$ ) (x :  $\alpha$ )
195   (hs : s ( $\sigma$  x) = !s x) (hd : d ( $\sigma$  x) = !d x) :
196    $\exists c : \text{Bool}, (d x = \text{xor } (s x) c \wedge d (\sigma x) = \text{xor } (s (\sigma x))$ 
   ↳  $c) \wedge$ 
197    $\forall c' : \text{Bool}, (d x = \text{xor } (s x) c' \wedge d (\sigma x) = \text{xor } (s (\sigma$ 
   ↳  $x)) c') \rightarrow c' = c :=$  by
198   refine (xor (d x) (s x),
199     (by cases s x <;> cases d x <;> rfl,
200       by rw [hs, hd]; cases s x <;> cases d x <;> rfl), ?_)
201   intro c' hc
202   obtain (h1, -) := hc
203   generalize hsx : s x = sv at h1
204   generalize hdx : d x = dv at h1
205   cases sv <;> cases dv <;> cases c' <;> first | rfl | exact
   ↳ absurd h1 (by decide)
206
207 structure Frame where
208   S : Type
209    $\tau : S \rightarrow S$ 
210   Z : S  $\rightarrow$  Prop
211
212 /- The line property: every zero is  $\tau$ -fixed. On  $\xi$ 's frame,
   ↳ the restored hypothesis. -/
213 def LineProperty (X : Frame) : Prop :=  $\forall s, X.Z s \rightarrow X.\tau s =$ 
   ↳ s
214
215 inductive Tri where
216   | tt
217   | ff
218   | bot
219   deriving DecidableEq, Repr
220
221 structure Bridge (X : Frame) where
222   terminal : Tri
223   shadow : terminal = Tri.bot  $\leftrightarrow$  LineProperty X
224
225 /- Theorem 7 of the spine: the bridge halts iff the line
   ↳ property. -/
226 theorem bridge_halted_iff (X : Frame) :
227   ( $\exists b : \text{Bridge } X, b.\text{terminal} = \text{Tri.bot}$ )  $\leftrightarrow$  LineProperty X
   ↳ :=
228   (fun {b, h} => b.shadow.mp h, fun t => ((Tri.bot, (fun _
   ↳ => t, fun _ => rfl)), rfl))
229
230 inductive Supply (X : Frame) where
231   | assent (t : LineProperty X)
232   | denial (n :  $\neg$  LineProperty X)
233   | silent
234
235 inductive Verdict where
236   | sealed
237   | refused
238   | open_
239   deriving DecidableEq, Repr
240
241 def row {X : Frame} : Supply X  $\rightarrow$  Verdict
242   | Supply.assent _ => Verdict.sealed
243   | Supply.denial _ => Verdict.refused
244   | Supply.silent => Verdict.open_
245
246 /- Theorem 8 of the spine: the crossing, exact. -/
247 theorem crossing (X : Frame) (t : LineProperty X) :
248   row (Supply.assent t) = Verdict.sealed  $\wedge$  ( $\exists b : \text{Bridge}$ 
   ↳  $\rightarrow X, b.\text{terminal} = \text{Tri.bot}$ )  $\wedge$  LineProperty X :=
249   (rfl, (bridge_halted_iff X).mpr t, t)
250
251 theorem crossing_other_way (X : Frame) (n :  $\neg$  LineProperty
   ↳ X) :
252   row (Supply.denial n) = Verdict.refused  $\wedge$   $\forall b : \text{Bridge}$ 
   ↳  $\rightarrow X, b.\text{terminal} \neq \text{Tri.bot} :=$ 
253   (rfl, fun b h => n (b.shadow.mp h))
254
255 /- The two-point frame: fold-invariant, off the line. -/
256 def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True)
257
258 /- Theorem 9 of the spine: necessity. -/
259 theorem supply_not_manufactured :  $\neg \forall X : \text{Frame},$ 
   ↳ LineProperty X :=

```

```

260 fun h => by
261   have := h twoPoint true trivial
262   cases this
263
264 def Inv (X : Frame) : Prop := ∀ s, X.Z s → X.Z (X.τ s)
265
266 theorem inv_not_line : Inv twoPoint ∧ ¬ LineProperty
  ↪ twoPoint :=
267   (fun _ => trivial, fun h => by
268     have := h true trivial
269     cases this)
270
271 /-- Theorem 12 of the spine: the bit is the hypothesis. -/
272 theorem the_bit_is_the_hypothesis (X : Frame) :
273   (∃ t : LineProperty X, row (Supply.assent t) =
274     ↪ Verdict.sealed) ↔ LineProperty X :=
275   (fun (t, _) => t, fun t => (t, rfl))
276
277 /-- Theorem 15 of the spine: the witness row is falsifiable.
278 ↪ -/
279 theorem witness_row_falsifiable (X : Frame) (s : X.S) (hz :
280   ↪ X.Z s) (hoff : X.τ s ≠ s) :
281   ¬ ∃ t : LineProperty X, row (Supply.assent t) =
282     ↪ Verdict.sealed :=
283   fun (t, _) => hoff (t s hz)
284
285 /-- The live face of the spine. -/
286 def live (witnessed assent : Bool) : Verdict :=
287   if witnessed then (if assent then Verdict.sealed else
288     ↪ Verdict.refused) else Verdict.open_
289
290 theorem narcissus : live false true ≠ live true true ∧ live
291   ↪ false false ≠ live true false := by
292   decide
293
294 /-! ## 5 · The aperture, RA-supplied -/
295 /-- A reader's presence is its actuation. -/
296 def presence (r : UniversePoint) : Bool := decide (0 <
297   ↪ DeltaE r)
298
299 /-- RA opens the aperture: every present reader is
300   ↪ witnessed. -/
301 theorem RA_opens_the_aperture : ∀ r : UniversePoint,
302   ↪ presence r = true := by
303   intro r
304   cases r
305   decide
306
307 /-- The general form, for any domain under the axiom as a
308   ↪ hypothesis: presence is
309   actuation, and RA on the domain makes every point
310   ↪ present. This is the theorem the
311   universal extension licenses wherever it is held. -/
312 theorem RA_opens_the_aperture_general {U : Type} (dE : U →
313   ↪ Int) (h : ∀ x, 0 < dE x) (x : U) :
314   decide (0 < dE x) = true :=
315   decide_eq_true (h x)
316
317 /-- Under the general form no present reader's row is
318   ↪ interior. -/
319 theorem no_interior_under_RA_general {U : Type} (dE : U →
320   ↪ Int) (h : ∀ x, 0 < dE x) (x : U)
321   (b : Bool) : live (decide (0 < dE x)) b ≠ Verdict.open_
322   ↪ := by
323   rw [RA_opens_the_aperture_general dE h x]
324   cases b <|> decide
325
326 /-- Under RA no present reader's row is interior. It is
327   ↪ sealed or refused by the bit alone. -/
328 theorem no_interior_under_RA (r : UniversePoint) (b : Bool)
329   ↪ :
330   live (presence r) b ≠ Verdict.open_ := by
331   cases r
332   cases b <|> decide
333
334 /-- What RA turns 'open' into: sealed if the bit is assent,
335   ↪ refused if denial. -/
336 theorem aperture_reads_the_bit (r : UniversePoint) :
337   live (presence r) true = Verdict.sealed ∧ live (presence
338     ↪ r) false = Verdict.refused := by

```

```

321 cases r
322 decide
323
324 /-- The Tongue's ledger: every adjudication, assent, denial,
325   ↪ or silence, is a deed. -/
326 inductive Adjudication where
327   | assent
328   | denial
329   | silence
330   deriving DecidableEq, Repr
331
332 structure Ledger where
333   deeds : Nat
334   deriving DecidableEq, Repr
335
336 def adjudicate ( _ : Adjudication) (L : Ledger) : Ledger :=
337   ↪ (L.deeds + 1)
338
339 theorem every_adjudication_is_a_deed (a : Adjudication) (L :
340   ↪ Ledger) :
341   (adjudicate a L).deeds = L.deeds + 1 := rfl
342
343 /-- Denial of RA is a deed, and a deed is what RA says
344   ↪ exists: the denial re-enacts RA. -/
345 theorem denial_re_enacts_RA (L : Ledger) :
346   (adjudicate Adjudication.denial L).deeds = L.deeds + 1 ∧
347     ↪ RA :=
348   (rfl, constructed_RA)
349
350 /-- The asymmetry: on an off-line frame no assent to L
351   ↪ exists at all; L's denial there
352   instantiates nothing about Z. L has no performative
353   ↪ ingress. -/
354 theorem L_has_no_performative_ingress :
355   ¬ ∃ t : LineProperty twoPoint, row (Supply.assent t) =
356     ↪ Verdict.sealed :=
357   witness_row_falsifiable twoPoint true trivial (fun h => by
358     ↪ cases h)
359
360 /-! ## 6 · THE SIMPLE RULE, EXACT, AND ITS BOUND -/
361
362 /-- Accept RA and RH follows: true exactly when RH is given.
363   ↪ Conditioning on RA adds
364   nothing to L and removes nothing from it. This is the
365   ↪ exact strength of the rule. -/
366 theorem simple_rule_exact (X : Frame) : (RA → LineProperty
367   ↪ X) ↔ LineProperty X :=
368   (fun h => h constructed_RA, fun t _ => t)
369
370 /-- RA decides the line property on no frame. The two-point
371   ↪ frame carries RA and fails L. -/
372 theorem RA_does_not_decide_L : ¬ ∀ X : Frame, RA →
373   ↪ LineProperty X :=
374   fun h => supply_not_manufactured (fun X => h X
375     ↪ constructed_RA)
376
377 /-- RA holds where L fails: the off-line frame is a model of
378   ↪ RA. -/
379 theorem RA_holds_where_L_fails : RA ∧ Inv twoPoint ∧ ¬
380   ↪ LineProperty twoPoint :=
381   (constructed_RA, inv_not_line.1, inv_not_line.2)
382
383 /-- The value is one supplied term and nothing weaker; RA
384   ↪ supplies the act, not the term. -/
385 theorem RA_supplies_the_act_not_the_term :
386   (∀ r : UniversePoint, presence r = true) ∧ (¬ ∀ X :
387     ↪ Frame, RA → LineProperty X) :=
388   (RA_opens_the_aperture, RA_does_not_decide_L)
389
390 /-! ## 7 · RH in toto at the apex, one theorem -/
391
392 /-- THE APEX THEOREM. Given RA: the three gaps are closed,
393   ↪ the formal self holds, the
394   freedom is exactly two, the bridge halts iff L, the
395   ↪ crossing is exact, the aperture
396   is open to every present reader, and RA decides the
397   ↪ value on no frame. -/
398 theorem RH_in_toto_at_the_apex : RA →
399   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM
400     ↪ = GammaRH) ∧
401   RHFormalSelf ∧

```

```

379 (V d : Bool → Bool, (V x, d (!x) = !d x) → d = (fun x =>
    ↪ x) v d = (fun x => !x)) ∧
380 (V X : Frame, (∃ b : Bridge X, b.terminal = Tri.bot) ↔
    ↪ LineProperty X) ∧
381 (V (X : Frame) (t : LineProperty X), row (Supply.assent
    ↪ t) = Verdict.sealed ∧ LineProperty X) ∧
382 (V r : UniversePoint, presence r = true) ∧
383 (¬ V X : Frame, RA → LineProperty X) :=
384 fun _ => (rfl, rfl, rfl), RH_formal_chain,
    ↪ freedom_is_exactly_two, bridge_halted_iff,
385 fun _ t => (rfl, t), RA_opens_the_aperture,
    ↪ RA_does_not_decide_L)
386
387 /-- The condition is discharged: RA is constructed, so the
    ↪ apex theorem stands unconditionally. -/
388 theorem apex_discharged :
389   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM
    ↪ = GammaRH) ∧ RHFormalSelf :=
390   ((RH_in_toto_at_the_apex constructed_RA).1,
    ↪ (RH_in_toto_at_the_apex constructed_RA).2.1)
391
392 /-! ## 8 · The root's interaction face, RAF, at the formal
    ↪ register -/
393
394 /-- An interaction domain: a membership predicate, a
    ↪ symmetric registration relation, and
395   RAF-2, closure under registration with a member. -/
396 structure Domain (S : Type) where
397   mem : S → Prop
398   registers : S → S → Prop
399   registers_symm : V a b, registers a b → registers b a
400   closure : V s d, mem d → registers s d → mem s
401
402 /-- RAF-C1, no exterior agent: nothing outside the domain
    ↪ registers with anything inside it. -/
403 theorem no_exterior_agent {S : Type} (D : Domain S) (s : S)
    ↪ (hs : ¬ D.mem s) :
404   V d, D.mem d → ¬ D.registers s d :=
405   fun d hd hr => hs (D.closure s d hd hr)
406
407 /-- The adjudicator is inside: a reader that registers a bit
    ↪ against a member is a member.
408   The witness is never exterior. -/
409 theorem adjudicator_in_domain {S : Type} (D : Domain S) (r d
    ↪ : S) (hd : D.mem d)
410   (hr : D.registers r d) : D.mem r :=
411   D.closure r d hd hr
412
413 /-- RAF-C2, no faithful self-representation (Cantor): no
    ↪ system indexes all of its own
414   binary properties. The general form of the spine's
    ↪ Narcissus table. -/
415 theorem no_total_self_indexing {S : Type} (f : S → S → Bool)
    ↪ :
416   ¬ V g : S → Bool, ∃ x, f x = g := by
417   intro h
418   obtain (x, hx) := h (fun y => !f y y)
419   have h1 : f x x = !f x x := congrFun hx x
420   generalize f x x = b at h1
421   cases b <|> cases h1
422
423 /-! ## 9 · The root's grounding face, RAM: the seat as the
    ↪ identity cone -/
424
425 /-- The three registers in which the one seat is read. -/
426 inductive Register3 where
427   | ra
428   | ram
429   | rh
430   deriving DecidableEq, Repr
431
432 /-- The discrete diagram: the seat as read in each register.
    ↪ -/
433 def D3 : Register3 → Q4
434   | Register3.ra => GammaRA
435   | Register3.ram => GammaRAM
436   | Register3.rh => GammaRH
437
438 /-- A cone over a diagram in the groupoid of identities: an
    ↪ apex with an identity leg to
439   every register's reading. A category gap is exactly the
    ↪ absence of such a leg. -/
440 structure ConeOver (D : Register3 → Q4) (apex : Q4) : Prop
    ↪ where
441   leg : V r, apex = D r
442
443 /-- A cone over the diagram of the ordered triad. -/
444 abbrev Cone : Q4 → Prop := ConeOver D3
445
446 /-- THE IDENTITY CONE. The named seat point is an apex over
    ↪ all three registers, every
447   leg `rfl`: the three category gaps are eliminated as
    ↪ identities. -/
448 theorem identity_cone : Cone GammaRH :=
449   (fun r => by cases r <|> rfl)
450
451 /-- The apex is unique: any two cones over the diagram share
    ↪ their apex. -/
452 theorem cone_apex_unique (a b : Q4) (ha : Cone a) (hb : Cone
    ↪ b) : a = b :=
453   (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm
454
455 /-- The reversed triad's diagram: the same three readings
    ↪ built from k·j·i, at +1. -/
456 def D3odd : Register3 → Q4
457   | Register3.ra => theReturnOdd
458   | Register3.ram => fixProj theReturnOdd
459   | Register3.rh => (1, 0, 0, 0)
460
461 /-- The three identities of Theorem C hold verbatim for the
    ↪ reversed triad, each rfl. -/
462 theorem odd_self_gap_nonexistent : sigma theReturnOdd =
    ↪ theReturnOdd := rfl
463 theorem odd_register_gap_nonexistent : theReturnOdd =
    ↪ fixProj theReturnOdd := rfl
464 theorem odd_object_gap_nonexistent : fixProj theReturnOdd =
    ↪ (1, 0, 0, 0) := rfl
465
466 /-- The reversed triad carries its own identity cone, apex
    ↪ +1. -/
467 theorem identity_cone_odd : ConeOver D3odd (1, 0, 0, 0) :=
    ↪ (fun r => by cases r <|> rfl)
468
469 /-- The two apices differ: uniqueness is per orientation of
    ↪ the triad. -/
470 theorem apexes_differ_by_orientation :
471   ConeOver D3 GammaRH ∧ ConeOver D3odd (1, 0, 0, 0) ∧
    ↪ GammaRH ≠ (1, 0, 0, 0) :=
472   (identity_cone, identity_cone_odd, by decide)
473
474 /-- A cone exists iff the register gap and the object gap
    ↪ are closed. -/
475 theorem cone_iff_gaps_closed :
476   (∃ a, Cone a) ↔ (GammaRA = GammaRAM ∧ GammaRAM =
    ↪ GammaRH) := by
477   constructor
478   · intro h
479     obtain (a, ha) := h
480     exact ((ha.leg Register3.ra).symm.trans (ha.leg
    ↪ Register3.ram),
    ↪ (ha.leg Register3.ram).symm.trans (ha.leg
    ↪ Register3.rh))
481   · intro h
482     obtain (h1, h2) := h
483     exact (GammaRH, (fun r => by
    ↪ cases r
    ↪ · exact (h1.trans h2).symm
    ↪ · exact h2.symm
    ↪ · rfl))
484
485 /-! ## 10 · The equivariant embedding of the spine's lattice
    ↪ stage into the carrier -/
486
487 /-- The spine's stage: the integer lattice in half-units,
    ↪ the fold τ(h, t) = (2 - h, t). -/
488 abbrev Plane := Int × Int
489
490 def tau (p : Plane) : Plane := (2 - p.1, p.2)
491
492 /-- Spine Theorem 1: the fold fixes exactly the line h = 1.
    ↪ -/

```

```

499 theorem ground_is_the_line (p : Plane) : tau p = p ↔ p.1 = 1
    ↪ := by
500   obtain (h, t) := p
501   constructor
502   · intro e
503     have e1 : 2 - h = h := congrArg Prod.fst e
504     show h = 1
505     omega
506   · intro e
507     show (2 - h, t) = (h, t)
508     have : h = 1 := e
509     subst this
510     rfl
511
512 /-- The embedding  $\phi : (\text{Plane}, \tau) \rightarrow (\text{Q4}, \sigma)$ ,  $\phi(h, t) = (t, h$ 
    ↪  $- 1, \theta, \theta)$ . -/
513 def phi (p : Plane) : Q4 := (p.2, p.1 - 1,  $\theta, \theta$ )
514
515 /--  $\phi$  is equivariant:  $\sigma \circ \phi = \phi \circ \tau$ . -/
516 theorem phi_equivariant (p : Plane) : sigma (phi p) = phi
    ↪ (tau p) := by
517   obtain (h, t) := p
518   change Q4.mk t (-(h - 1))  $\theta \theta = \text{Q4.mk } t ((2 - h) - 1) \theta \theta$ 
519   have e : -(h - 1) = (2 - h) - 1 := by omega
520   rw [e]
521
522 /--  $\phi$  carries the line onto the scalar line: a stage point
    ↪ is  $\tau$ -fixed iff its image is  $\sigma$ -fixed.
523   This is the image clause of the embedding, Fix( $\tau$ ) ↔
    ↪ Fix( $\sigma$ ). -/
524 theorem phi_fix_iff (p : Plane) : sigma (phi p) = phi p ↔
    ↪ tau p = p := by
525   obtain (h, t) := p
526   change (Q4.mk t (-(h - 1))  $\theta \theta = \text{Q4.mk } t (h - 1) \theta \theta$  ↔
    ↪ ((2 - h, t) = (h, t))
527   constructor
528   · intro e
529     injection e with _ e2 _
530     have : h = 1 := by omega
531     subst this
532     rfl
533   · intro e
534     have e1 : 2 - h = h := congrArg Prod.fst e
535     have : h = 1 := by omega
536     subst this
537     rfl
538
539 /-- The stage at resolution m: the fold  $\tau_m(h, t) = (2m - h,$ 
    ↪  $t)$  and the map
540    $\phi_m(h, t) = (t, h - m, \theta, \theta)$ . Resolution one is the case
    ↪  $m = 1$ . -/
541 def tauM (m : Int) (p : Plane) : Plane := (2 * m - p.1, p.2)
542
543 def phiM (m : Int) (p : Plane) : Q4 := (p.2, p.1 - m,  $\theta, \theta$ )
544
545 /-- Equivariance at every resolution:  $\sigma \circ \phi_m = \phi_m \circ \tau_m$ .
    ↪ -/
546 theorem phiM_equivariant (m : Int) (p : Plane) : sigma (phiM m
    ↪ p) = phiM m (tauM m p) := by
547   obtain (h, t) := p
548   change Q4.mk t (-(h - m))  $\theta \theta = \text{Q4.mk } t ((2 * m - h) - m)$ 
    ↪  $\theta \theta$ 
549   have e : -(h - m) = (2 * m - h) - m := by omega
550   rw [e]
551
552 /-- The image clause at every resolution: a stage point is
    ↪  $\tau_m$ -fixed iff its image is  $\sigma$ -fixed. -/
553 theorem phiM_fix_iff (m : Int) (p : Plane) : sigma (phiM m
    ↪ p) = phiM m (tauM m p) ↔ tauM m p = p := by
554   obtain (h, t) := p
555   change (Q4.mk t (-(h - m))  $\theta \theta = \text{Q4.mk } t (h - m) \theta \theta$  ↔
    ↪ ((2 * m - h, t) = (h, t))
556   constructor
557   · intro e
558     injection e with _ e2 _
559     have : h = m := by omega
560     subst this
561     show (2 * h - h, t) = (h, t)
562     have e3 : 2 * h - h = h := by omega
563     rw [e3]
564   · intro e
565     have e1 : 2 * m - h = h := congrArg Prod.fst e
566     have : h = m := by omega
567     subst this
568     show Q4.mk t (-(h - h))  $\theta \theta = \text{Q4.mk } t (h - h) \theta \theta$ 
569     have e3 : h - h =  $\theta$  := by omega
570     rw [e3]
571     rfl
572
573 /-- Resolution one is the embedding  $\phi$ . -/
574 theorem phiM_one (p : Plane) : phiM 1 p = phi p := rfl
575
576 theorem tauM_one (p : Plane) : tauM 1 p = tau p := by
577   obtain (h, t) := p
578   show (2 * 1 - h, t) = (2 - h, t)
579   rfl
580
581 /--  $\phi$  is injective: distinct stage points have distinct
    ↪ images. -/
582 theorem phi_injective (p q : Plane) (e : phi p = phi q) : p
    ↪ = q := by
583   obtain (h, t) := p
584   obtain (h', t') := q
585   injection e with e1 e2 _
586   have : h = h' := by omega
587   subst this
588   subst e1
589   rfl
590
591 /-- The seat is on the image of the line: the line point (1,
    ↪ -1) maps to  $\Gamma_{\text{RH}}$ . -/
592 theorem seat_on_image_of_line : phi (1, -1) = GammaRH := rfl
593
594 /--! ## 11 · The wall, executed: no readout of the formal
    ↪ seat equals the deed bit -/
595
596 def ExecFrame : Type := Bool × Ground
597
598 def execFlip (s : ExecFrame) : ExecFrame := (s.1, s.2)
599
600 def formalRead (s : ExecFrame) : Ground := s.2
601
602 def ran (s : ExecFrame) : Bool := s.1
603
604 theorem formalRead_even (f : Ground → Bool) (s : ExecFrame)
    ↪ :
605   f (formalRead (execFlip s)) = f (formalRead s) := rfl
606
607 theorem ran_wholly_odd (s : ExecFrame) : ran (execFlip s) =
    ↪ !ran s := rfl
608
609 /-- No readout of the formal seat equals the deed bit: the
    ↪ instance of spine Theorem 4 on
610   the constructed seat. A term consuming the orientation
    ↪ bit as data would be such a readout. -/
611 theorem kernel_cannot_read_the_deed :
612   ¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead s)
    ↪ = ran s := by
613   intro h
614   obtain (f, hf) := h
615   have h1 : f seat = true := hf (true, seat)
616   have h2 : f seat = false := hf (false, seat)
617   rw [h1] at h2
618   exact Bool.noConfusion h2
619
620 /--! ## 12 · The dual register -/
621
622 inductive Register where
623   | apex
624   | world
625   deriving DecidableEq, Repr
626
627 inductive Token where
628   | sealAgivenRA
629   | witnessRow
630   deriving DecidableEq, Repr
631
632 def token : Register → Token
633   | Register.apex => Token.sealAgivenRA
634   | Register.world => Token.witnessRow
635

```

```

636 theorem registers_parted : token Register.apex ≠ token
637   ↪ Register.world := by
638     decide
639 def owedBits : Register → Nat
640   | Register.apex => 0
641   | Register.world => 1
642
643 theorem apex_owes_nothing_world_owes_one :
644   owedBits Register.apex = 0 ∧ owedBits Register.world = 1
645   ↪ := (rfl, rfl)
646
647 /-- Landauer floor at 300 K, yoctojoules per bit: the price
648   ↪ of the act. -/
649 def landauer_yJ_300K (bits : Nat) : Nat := 2871 * bits
650
651 theorem spend_priced : landauer_yJ_300K 1 = 2871 := rfl
652
653 def deltaM : Nat := 0
654
655 theorem deltaM_zero : deltaM = 0 := rfl
656
657 /-! ## 13 · The apex theorem, hardened: the closure with its
658   ↪ cone, its embedding, and its wall -/
659
660 /-- THE APEX THEOREM, HARDENED. Given RA: the closure of
661   ↪ Theorem H, the identity cone with
662   ↪ its apex unique, the equivariant embedding of the
663   ↪ spine's stage with its image clause,
664   ↪ and the executed wall. -/
665 theorem apex_harden : RA →
666   ((sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧
667     ↪ GammaRAM = GammaRH) ∧ RHFormalSelf) ∧
668   (Cone GammaRH ∧ ∀ a b, Cone a → Cone b → a = b) ∧
669   ((∀ p : Plane, sigma (phi p) = phi (tau p)) ∧ (∀ p :
670     ↪ Plane, sigma (phi p) = phi p ↔ tau p = p)) ∧
671   (¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead
672     ↪ s) = ran s) ∧
673   (¬ ∀ X : Frame, RA → LineProperty X) :=
674   fun h => ((RH_in_toto_at_the_apex h).1,
675     ↪ (RH_in_toto_at_the_apex h).2.1),
676   (identity_cone, cone_apex_unique), (phi_equivariant,
677     ↪ phi_fix_iff),
678   kernel_cannot_read_the_deed, RA_does_not_decide_L)
679
680 theorem apex_harden_discharged : Cone GammaRH ∧ (∀ p :
681   ↪ Plane, sigma (phi p) = phi p ↔ tau p = p) :=
682   ((apex_harden discharged_RA).2.1.1, (apex_harden
683     ↪ discharged_RA).2.2.1.2)
684
685 /-! ## 14 · The counter-model survives grounding, and the
686   ↪ cure is the hypothesis -/
687
688 /-- The posit is existence: to exist is to actuate. Its
689   ↪ published name is the Root Axiom,
690   ↪ and the two are one proposition. -/
691 theorem existence_is_RA : (∀ x : UniversePoint, 0 < DeltaE
692   ↪ x) ↔ RA := Iff.rfl
693
694 /-- The two-point frame is groundless: its fold fixes
695   ↪ nothing, so it has no seat. -/
696 theorem twoPoint_groundless : ∀ b : Bool, twoPoint.τ b ≠ b
697   ↪ := by
698   intro b h
699   cases b <|> exact Bool.noConfusion h
700
701 /-- A grounded counter-model: three points, the fold swaps
702   ↪ two and fixes the third, the
703   ↪ zero set is the swapped pair. The frame has a seat and a
704   ↪ moved zero. -/
705 inductive Three where
706   | a
707   | b
708   | c
709   deriving DecidableEq
710
711 def foldThree : Three → Three
712   | Three.a => Three.b
713   | Three.b => Three.a
714   | Three.c => Three.c
715
716 def threePoint : Frame := (Three, foldThree, fun x => x =
717   ↪ Three.a ∨ x = Three.b)
718
719 theorem threePoint_grounded : ∃ s, threePoint.τ s = s :=
720   ↪ (Three.c, rfl)
721
722 theorem threePoint_fold_involution : ∀ s, foldThree
723   ↪ (foldThree s) = s := by
724   intro s; cases s <|> rfl
725
726 theorem threePoint_inv : Inv threePoint := by
727   intro s hs
728   cases s
729   · exact 0r.inr rfl
730   · exact 0r.inl rfl
731   · cases hs with
732     | inl h => cases h
733     | inr h => cases h
734
735 theorem threePoint_fails_L : ¬ LineProperty threePoint := by
736   intro h
737   have := h Three.a (0r.inl rfl)
738   cases this
739
740 /-- Existence holds on a grounded frame where the line
741   ↪ property fails: the bound of
742   ↪ Theorem F is not an artifact of a groundless frame. -/
743 theorem RA_holds_where_L_fails_grounded :
744   RA ∧ (∃ s, threePoint.τ s = s) ∧ Inv threePoint ∧ ¬
745   ↪ LineProperty threePoint :=
746   (constructed_RA, threePoint_grounded, threePoint_inv,
747     ↪ threePoint_fails_L)
748
749 /-- THE CURE THEOREM. For every class C of frames, existence
750   ↪ decides the line property on
751   ↪ C exactly when the line property already holds on C: the
752   ↪ restriction that removes
753   ↪ every counter-model is the hypothesis itself. -/
754 theorem no_cure (C : Frame → Prop) :
755   (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X →
756     ↪ LineProperty X) :=
757   (fun h X hc => h X hc constructed_RA, fun h X hc _ => h X
758     ↪ hc)
759
760 /-- The only class on which existence forces the line
761   ↪ property everywhere is a class on
762   ↪ which the line property is already a hypothesis; the
763   ↪ class of all frames is not one. -/
764 theorem cure_is_the_hypothesis :
765   (∀ X, LineProperty X → RA → LineProperty X) ∧ ¬ (∀ X,
766     ↪ True → RA → LineProperty X) :=
767   (fun _ h _ => h, fun h => RA_does_not_decide_L (fun X => h
768     ↪ X trivial))
769
770 /-! ## 15 · The halting carrier inhabited by decision; any
771   ↪ true premise is exact -/
772
773 /-- The halting carrier is an interface: a terminal and the
774   ↪ clause that it is the bottom
775   ↪ iff the line property holds. It is inhabited
776   ↪ constructively wherever the line property
777   ↪ is decided, the terminal computed from the decision and
778   ↪ the clause proved from it,
779   ↪ never assumed. On the frame of ξ the decision is the
780   ↪ owed bit. -/
781 def bridgeOfDecision (X : Frame) (d : Decidable
782   ↪ (LineProperty X)) : Bridge X :=
783   match d with
784   | isTrue h => (Tri.bot, {fun _ => h, fun _ => rfl})
785   | isFalse h => (Tri.tt, {fun e => absurd e (by decide),
786     ↪ fun hl => absurd hl h})
787
788 /-- On the two-point frame the decision is negative and the
789   ↪ carrier does not halt. -/
790 theorem twoPoint_carrier_does_not_halt :
791   (bridgeOfDecision twoPoint (isFalse
792     ↪ inv_not_line.2)).terminal = Tri.tt := rfl
793
794 /-- The one-point frame on the line: the decision is
795   ↪ positive and the carrier halts. -/
796 def onLine : Frame := (Unit, fun u => u, fun _ => True)

```

```

754
755 theorem onLine_L : LineProperty onLine := fun _ => rfl
756
757 theorem onLine_carrier_halts :
758   (bridgeOfDecision onLine (isTrue onLine_L)).terminal =
759     ↳ Tri.bot := rfl
760
761 /-- Theorem E in its full generality: any inhabited premise
762 ↳ is exact. The content of
763   Theorem E is not a property of existence in particular
764   ↳ but of its truth: a premise
765   that holds wherever the line property is evaluated, and
766   ↳ is independent of it, adds
767   nothing to it and removes nothing from it. -/
768 theorem any_true_premise_is_exact (P : Prop) (hp : P) (X :
769   ↳ Frame) :
770   (P ↳ LineProperty X) ↳ LineProperty X :=
771   (fun h => h hp, fun t _ => t)
772
773 /-- The constructed witness's recursion field is the
774 ↳ constant function: it discards the
775   orientation and the formal Ground and returns the
776   ↳ Ground's own defining property.
777   That is the masslessness stated as a term: the formal
778   ↳ self needs nothing from the
779   axiom to be proved, and the witness supplies the
780   ↳ occupancy of the registers, not a
781   derivation. -/
782 theorem recursion_is_constant :
783   ∀ (o o' : OrientationBit) (g g' : RAMFormalGround),
784   mkRA.trisRecursion o g = mkRA.trisRecursion o' g' :=
785   fun _ _ _ => rfl
786
787 /--! ## 16 · Three measures: content, mass, price -/
788
789 /-- Three measures are kept apart for every object of the
790 ↳ closure. Content: what the object
791   says, decided by rfl (trivial), proved (a theorem), or
792   ↳ supplied (one bit). Mass: the
793   mathematics authored in producing it, ΔM. Price: the
794   ↳ energy the act of registering it
795   dissipates, in yJ at 300 K. Massless is not trivial and
796   ↳ trivial is not free: the legs
797   of the cone are trivial and massless and cost nothing;
798   ↳ the value is massless, not
799   trivial, and priced. -/
800 inductive Content where
801   | trivial_
802   | proved
803   | supplied
804   deriving DecidableEq, Repr
805
806 structure Measure where
807   content : Content
808   mass : Nat
809   priceYJ : Nat
810   deriving DecidableEq, Repr
811
812 /-- A leg of the identity cone: decided by rfl, authors
813 ↳ nothing, costs nothing. -/
814 def legMeasure : Measure := (Content.trivial_, deltaM, 0)
815
816 /-- A theorem of the arc, Theorem F say: proved, authors
817 ↳ nothing (every step classical
818   or definitional), costs nothing. -/
819 def arcMeasure : Measure := (Content.proved, deltaM, 0)
820
821 /-- The value on the zeros: supplied at the act, authors
822 ↳ nothing, priced at the floor. -/
823 def valueMeasure : Measure := (Content.supplied, deltaM,
824   ↳ landauer_yJ_300K 1)
825
826 /-- Every measure carries mass zero: the closure and the
827 ↳ supply author no mathematics.
828   This is the masslessness the root-premise law forces,
829   ↳ and it is the same zero at the
830   legs, at the arc, and at the value. -/
831 theorem masses_all_zero :
832   legMeasure.mass = 0 ∧ arcMeasure.mass = 0 ∧
833   ↳ valueMeasure.mass = 0 :=
834   (rfl, rfl, rfl)
835
836 /-- The contents differ: masslessness does not collapse
837 ↳ them. -/
838 theorem contents_differ :
839   legMeasure.content ≠ valueMeasure.content ∧
840   ↳ arcMeasure.content ≠ valueMeasure.content ∧
841   legMeasure.content ≠ arcMeasure.content := by
842   refine (?, ?, ?) <|> decide
843
844 /-- The prices differ: the value is the one object that is
845 ↳ paid for. -/
846 theorem prices_differ : legMeasure.priceYJ = 0 ∧
847   ↳ valueMeasure.priceYJ = 2871 := (rfl, rfl)
848
849 /-- Adjudicating any proposition is a deed, and a deed is
850 ↳ what existence says exists:
851   every adjudication of anything instances existence and
852   ↳ instances nothing else. This
853   is what existence contributes that no other true premise
854   ↳ does. -/
855 theorem adjudicating_anything_instances_existence (a :
856   ↳ Adjudication) (L : Ledger) :
857   (adjudicate a L).deeds = L.deeds + 1 ∧ RA :=
858   (rfl, constructed_RA)
859
860 /--! ## 17 · The twenty-three rows -/
861
862 inductive Row : Type where
863   | pVsNP | riemann | navierStokes | yangMills | hodge | bsd
864   ↳ | poincare
865   | goldbach | twinPrimes | legendre | abc | jacobian
866   | mersenne | oddPerfect | smoothPoincare4
867   | collatz
868   | hadwiger | sunflower | erdosStraus | beal
869   | invariantSubspace | hilbert16 | lindelofPCC
870   deriving DecidableEq, Repr
871
872 def Row.all : List Row :=
873   [Row.pVsNP, Row.riemann, Row.navierStokes, Row.yangMills,
874   ↳ Row.hodge, Row.bsd, Row.poincare,
875   Row.goldbach, Row.twinPrimes, Row.legendre, Row.abc,
876   ↳ Row.jacobian,
877   Row.mersenne, Row.oddPerfect, Row.smoothPoincare4,
878   ↳ Row.collatz,
879   Row.hadwiger, Row.sunflower, Row.erdosStraus, Row.beal,
880   Row.invariantSubspace, Row.hilbert16, Row.lindelofPCC]
881
882 theorem rows_twenty_three : Row.all.length = 23 := by decide
883 theorem rows_distinct : Row.all.eraseDups.length = 23 := by
884   ↳ decide
885 theorem rows_complete : ∀ r : Row, r ∈ Row.all := by intro
886   ↳ r; cases r <|> decide
887
888 def Row.millennium : Row → Bool
889   | Row.pVsNP | Row.riemann | Row.navierStokes |
890   ↳ Row.yangMills
891   | Row.hodge | Row.bsd | Row.poincare => true
892   | _ => false
893
894 theorem millennium_seven : (Row.all.filter
895   ↳ Row.millennium).length = 7 := by decide
896 theorem extension_sixteen : (Row.all.filter (fun r =>
897   ↳ !r.millennium)).length = 16 := by decide
898
899 def Row.name : Row → String
900   | Row.pVsNP => "P versus NP" | Row.riemann => "Riemann
901   ↳ Hypothesis"
902   | Row.navierStokes => "Navier-Stokes" | Row.yangMills =>
903   ↳ "Yang-Mills"
904   | Row.hodge => "Hodge" | Row.bsd => "BSD" | Row.poincare
905   ↳ => "Poincare"
906   | Row.goldbach => "Goldbach" | Row.twinPrimes => "Twin
907   ↳ primes"
908   | Row.legendre => "Legendre" | Row.abc => "abc" |
909   ↳ Row.jacobian => "Jacobian"
910   | Row.mersenne => "Mersenne primes" | Row.oddPerfect =>
911   ↳ "Odd perfect numbers"
912   | Row.smoothPoincare4 => "Smooth 4D Poincare" |
913   ↳ Row.collatz => "Collatz"

```

```

871 | Row.hadwiger => "Hadwiger" | Row.sunflower =>
    ↳ "Sunflower"
872 | Row.erdosStraus => "Erdos-Straus" | Row.beal => "Beal"
873 | Row.invariantSubspace => "Invariant subspace" |
    ↳ Row.hilbert16 => "Hilbert 16 second part"
874 | Row.lindelofPCC => "Lindelof / Montgomery PCC"
875
876 /-! ## 18 · One seat under every name: the identity cone on
    ↳ every row -/
877
878 /- Every row's seat point is the one seat point  $\Gamma_{RH}$ . -/
879 def apexSeat : Row → Q4 := fun _ => GammaRH
880
881 theorem apex_seat_fixed : ∀ r : Row, sigma (apexSeat r) =
    ↳ apexSeat r := by
882   intro r; cases r <|> rfl
883 theorem apex_seat_uniform : ∀ r s : Row, apexSeat r =
    ↳ apexSeat s := by
884   intro r s; cases r <|> cases s <|> rfl
885 theorem crossed_and_open_share_the_seat : apexSeat
    ↳ Row.poincare = apexSeat Row.riemann := rfl
886
887 /- The register diagram of a row: the axiom's Return, its
    ↳ projection, the row's seat point. -/
888 def D3row (r : Row) : Register3 → Q4
889 | Register3.ra => GammaRA
890 | Register3.ram => GammaRAM
891 | Register3.rh => apexSeat r
892
893 /- THE IDENTITY CONE ON EVERY ROW, every leg rfl. -/
894 theorem identity_cone_row (r : Row) : ConeOver (D3row r)
    ↳ GammaRH :=
895   (fun k => by cases k <|> cases r <|> rfl)
896
897 theorem cone_apex_unique_row {r : Row} (a b : Q4)
898   (ha : ConeOver (D3row r) a) (hb : ConeOver (D3row r) b)
    ↳ : a = b :=
899   (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm
900
901 /- The gap vector, each bit decided from its own identity,
    ↳ closed on all twenty-three rows. -/
902 inductive GapBit where
903 | openBit
904 | closedBit
905 deriving DecidableEq, Repr
906
907 def bitOf (p : Prop) [Decidable p] : GapBit := if p then
    ↳ GapBit.closedBit else GapBit.openBit
908
909 def gapVector (r : Row) : GapBit × GapBit × GapBit :=
910   (bitOf (sigma GammaRA = GammaRA), bitOf (GammaRA =
    ↳ GammaRAM), bitOf (GammaRAM = apexSeat r))
911
912 theorem gaps_closed_every_row :
913   Row.all.all (fun r => decide (gapVector r =
    ↳ (GapBit.closedBit, GapBit.closedBit,
    ↳ GapBit.closedBit))) = true := by
914   decide
915
916 /-! ## 19 · The constructed witness on every row, the
    ↳ Poincaré row included -/
917
918 /- The formal self of a row: the seat read as a
    ↳ proposition; one statement for all rows. -/
919 def formalSelf (r : Row) : Prop := RHFormalSelf
920
921 structure RowWitness (r : Row) : Prop where
922   actuates : RA
923   orientation : OrientationBit
924   bridge : RAMFormalGround
925   trisRecursion : OrientationBit → RAMFormalGround →
    ↳ formalSelf r
926
927 /- The same witness on every row: existence, the
    ↳ orientation bit, the formal Ground, and
928   the constant recursion. No Perelman field on the
    ↳ Poincaré row, no Euler product on the
929   Riemann row, nothing row-specific at all. -/
930 theorem mkRow (r : Row) : RowWitness r :=
931   (constructed_RA, constructed_orientation, constructed_RAM,
    ↳ fun _ _ g => g.2)
932
933 theorem eliminator (r : Row) : formalSelf r := (mkRow
    ↳ r).trisRecursion (mkRow r).orientation (mkRow r).bridge
934
935 theorem all_rows_eliminated : ∀ r : Row, formalSelf r :=
    ↳ eliminator
936
937 /- THE INVERTED CONTROL. The formal self is one proposition
    ↳ on every row, so a witness for
938   it on the crossed row is a witness for it on every open
    ↳ row: it decides nothing. -/
939 theorem inverted_control : ∀ r s : Row, formalSelf r ↔
    ↳ formalSelf s := fun _ => Iff.rfl
940
941 theorem recursion_constant_every_row (r : Row) (o o' :
    ↳ OrientationBit) (g g' : RAMFormalGround) :
942   (mkRow r).trisRecursion o g = (mkRow r).trisRecursion o'
    ↳ g' := rfl
943
944 /-! ## 20 · The rule, the non-decision, and the cure on
    ↳ every row -/
945
946 /- A row's value is the line property of a frame assigned
    ↳ to it; the assignment is a
947   parameter, since no row's object is definable in core
    ↳ Lean. -/
948 def value (F : Row → Frame) (r : Row) : Prop := LineProperty
    ↳ (F r)
949
950 theorem rule_exact_every_row (F : Row → Frame) (r : Row) :
    ↳ (RA → value F r) ↔ value F r :=
951   (fun h => h constructed_RA, fun t _ => t)
952
953 /- Existence decides no row's value: the assignment sending
    ↳ every row to the two-point
954   frame carries existence and fails every value. -/
955 theorem RA_decides_no_row : ¬ ∀ (F : Row → Frame) (r : Row),
    ↳ RA → value F r :=
956   fun h => inv_not_line.2 (h (fun _ => twoPoint) Row.riemann
    ↳ constructed_RA)
957
958 theorem RA_decides_no_row_grounded : ¬ ∀ (F : Row → Frame)
    ↳ (r : Row), RA → value F r :=
959   fun h => threePoint_fails_L (h (fun _ => threePoint)
    ↳ Row.poincare constructed_RA)
960
961 /- THE CURE THEOREM ON EVERY ROW: for every row and every
    ↳ class of frames, existence
962   decides the row's value on the class exactly where the
    ↳ value already holds on it. -/
963 theorem no_cure_every_row (r : Row) (C : Frame → Prop) :
964   (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X →
    ↳ LineProperty X) :=
965   match r with | _ => no_cure C
966
967 /-! ## 21 · The world row carried, the price, and the apex
    ↳ theorem across the corpus -/
968
969 inductive WorldTyping : Type where
970 | exactStructural | wallRowF4 | beachheadOrderTwo |
    ↳ crossedControl
971 | pureUnbridged | diagonalPortF3 | oneBitFromClosure |
    ↳ wallRowBridgeGate
972 deriving DecidableEq, Repr
973
974 def worldTyping : Row → WorldTyping
975 | Row.pVsNP => WorldTyping.exactStructural | Row.riemann
    ↳ => WorldTyping.exactStructural
976 | Row.navierStokes => WorldTyping.wallRowF4 |
    ↳ Row.yangMills => WorldTyping.wallRowF4
977 | Row.hodge => WorldTyping.wallRowF4 | Row.bsd =>
    ↳ WorldTyping.beachheadOrderTwo
978 | Row.poincare => WorldTyping.crossedControl
979 | Row.goldbach => WorldTyping.exactStructural |
    ↳ Row.twinPrimes => WorldTyping.exactStructural
980 | Row.legendre => WorldTyping.exactStructural | Row.abc =>
    ↳ WorldTyping.exactStructural
981 | Row.jacobian => WorldTyping.exactStructural
982 | Row.mersenne => WorldTyping.pureUnbridged |
    ↳ Row.oddPerfect => WorldTyping.pureUnbridged
983 | Row.smoothPoincare4 => WorldTyping.pureUnbridged

```

```

984 | Row.collatz => WorldTyping.diagonalPortF3
985 | Row.hadwiger => WorldTyping.oneBitFromClosure |
  ↳ Row.sunflower => WorldTyping.oneBitFromClosure
986 | Row.erdosStraus => WorldTyping.oneBitFromClosure |
  ↳ Row.beal => WorldTyping.oneBitFromClosure
987 | Row.invariantSubspace => WorldTyping.wallRowBridgeGate |
  ↳ Row.hilbert16 => WorldTyping.wallRowBridgeGate
988 | Row.lindelofPCC => WorldTyping.wallRowBridgeGate
989
990 def worldCrossed (r : Row) : Bool := decide (worldTyping r =
  ↳ WorldTyping.crossedControl)
991 def worldOwed (r : Row) : Nat := if worldCrossed r then 0
  ↳ else 1
992
993 theorem world_owes_at_most_one : ∀ r : Row, worldOwed r ≤ 1
  ↳ := by intro r; cases r <|> decide
994 theorem exactly_one_world_crossing : (Row.all.filter
  ↳ worldCrossed).length = 1 := by decide
995 theorem world_bits_owed_total : (Row.all.map
  ↳ worldOwed).foldl (· + ·) 0 = 22 := by decide
996
997 theorem world_census :
998   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.exactStructural))).length = 7 ∧
999   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.wallRowF4))).length = 3 ∧
1000   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.beachheadOrderTwo))).length = 1 ∧
1001   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.crossedControl))).length = 1 ∧
1002   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.pureUnbridged))).length = 3 ∧
1003   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.diagonalPortF3))).length = 1 ∧
1004   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.oneBitFromClosure))).length = 4 ∧
1005   (Row.all.filter (fun r => decide (worldTyping r =
  ↳ WorldTyping.wallRowBridgeGate))).length = 3 := by
1006   decide
1007
1008 /-- THE NO-PROMOTION THEOREM. The apex closes on every row;
  ↳ the world row is crossed on
1009   exactly the Poincaré row; the first never entails the
  ↳ second. -/
1010 theorem apex_never_promotes_world :
1011   (∀ r : Row, formalSelf r) ∧ (∀ r : Row, worldCrossed r =
  ↳ true ↔ r = Row.poincare) := by
1012   refine (all_rows_eliminated, ?_)
1013   intro r; cases r <|> decide
1014
1015 /-- The router bit, measured in the register of record for
  ↳ two rows only. -/
1016 def groundDim : Row → Option Nat
1017 | Row.riemann => some 1
1018 | Row.pVsNP => some 0
1019 | _ => none
1020
1021 inductive RowVerdict : Type where
1022 | sealAgivenRA | crossed | xi0 | o0 | unrouted
1023 deriving DecidableEq, Repr
1024
1025 def apexVerdict : Row → RowVerdict := fun _ =>
  ↳ RowVerdict.sealAgivenRA
1026 def worldVerdict (r : Row) : RowVerdict :=
1027   if worldCrossed r then RowVerdict.crossed
1028   else match groundDim r with
1029   | some 1 => RowVerdict.xi0
1030   | some 0 => RowVerdict.o0
1031   | _ => RowVerdict.unrouted
1032
1033 theorem world_verdict_split :
1034   worldVerdict Row.poincare = RowVerdict.crossed ∧
  ↳ worldVerdict Row.riemann = RowVerdict.xi0 ∧
1035   worldVerdict Row.pVsNP = RowVerdict.o0 ∧ worldVerdict
  ↳ Row.navierStokes = RowVerdict.unrouted := by
1036   decide
1037 theorem unrouted_twenty : (Row.all.filter (fun r => decide
  ↳ (worldVerdict r = RowVerdict.unrouted))).length = 20 :=
  ↳ by
1038   decide
1039
1040 theorem registers_parted_every_row : ∀ r : Row, apexVerdict
  ↳ r ≠ worldVerdict r := by
1041   intro r; cases r <|> decide
1042
1043 /-- The bridge on one row. The spine embeds the Riemann
  ↳ row's stage into the carrier
1044   equivocantly (0, Movement 10). No other row's stage is
  ↳ embedded in this file: on the
1045   twenty-two other rows the seat is assigned by the
  ↳ grounding reading and not bridged
  ↳ from the row's object. The count is kept on the record.
  ↳ -/
1046 def stageEmbedded : Row → Bool := fun r => decide (r =
  ↳ Row.riemann)
1047
1048 theorem one_row_embedded : (Row.all.filter
  ↳ stageEmbedded).length = 1 := by decide
1049 theorem embeddings_owed : (Row.all.filter (fun r =>
  ↳ !stageEmbedded r)).length = 22 := by decide
1050
1051 /-- The three measures per row: the leg trivial and free on
  ↳ every row; the value supplied,
1052   massless, and priced at one floor where owed and at
  ↳ nothing where crossed. -/
1053 def rowValueMeasure (r : Row) : Measure :=
  ↳ (Content.supplied, deltaM, landauer_yJ_300K (worldOwed
  ↳ r))
1054
1055 theorem row_masses_zero : ∀ r : Row, (rowValueMeasure
  ↳ r).mass = 0 := by intro r; rfl
1056 theorem corpus_price_total : (Row.all.map (fun r =>
  ↳ (rowValueMeasure r).priceYJ)).foldl (· + ·) 0 = 63162 :=
  ↳ by
1057   decide
1058 theorem poincare_price_zero : (rowValueMeasure
  ↳ Row.poincare).priceYJ = 0 := rfl
1059
1060 /-- THE CORPUS THEOREM. Given existence: on every row the
  ↳ identity cone with its unique
1061   apex, the formal self, and the seat shared with every
  ↳ other row; the rule exact and
1062   the cure theorem on every row; existence deciding no
  ↳ row's value, seatless and grounded
1063   alike; the world row crossed on exactly one row;
  ↳ twenty-two bits owed. -/
1064 theorem rows_at_the_apex : RA →
1065   (∀ r : Row, ConeOver (D3row r) GammaRH ∧ formalSelf r ∧
  ↳ apexSeat r = GammaRH) ∧
1066   (∀ (F : Row → Frame) (r : Row), (RA → value F r) → value
  ↳ F r) ∧
1067   (∀ (r : Row) (C : Frame → Prop), (∀ X, C X → RA →
  ↳ LineProperty X) ↔ (∀ X, C X → LineProperty X)) ∧
1068   (¬ ∀ (F : Row → Frame) (r : Row), RA → value F r) ∧
1069   (∀ r : Row, worldCrossed r = true ↔ r = Row.poincare) ∧
1070   (Row.all.map worldOwed).foldl (· + ·) 0 = 22 :=
1071   fun _ => (fun r => (identity_cone_row r, eliminator r,
  ↳ rfl), rule_exact_every_row, no_cure_every_row,
1072     RA_decides_no_row, apex_never_promotes_world.2,
  ↳ world_bits_owed_total)
1073
1074 theorem rows_discharged : (∀ r : Row, formalSelf r) ∧
  ↳ (Row.all.map worldOwed).foldl (· + ·) 0 = 22 :=
1075   {(rows_at_the_apex constructed_RA).1 |> fun h r => (h
  ↳ r).2.1, world_bits_owed_total)
1076
1077 end RowsAtTheApex
1078
1079 /-! ## The axiom dependency sets, printed -/
1080 #print axioms RowsAtTheApex.constructed_RA
1081 #print axioms RowsAtTheApex.fix_iff_scalar
1082 #print axioms RowsAtTheApex.rows_twenty_three
1083 #print axioms RowsAtTheApex.millennium_seven
1084 #print axioms RowsAtTheApex.apex_seat_uniform
1085 #print axioms RowsAtTheApex.identity_cone_row
1086 #print axioms RowsAtTheApex.cone_apex_unique_row
1087 #print axioms RowsAtTheApex.gaps_closed_every_row
1088 #print axioms RowsAtTheApex.mkRow
1089 #print axioms RowsAtTheApex.all_rows_eliminated
1090 #print axioms RowsAtTheApex.inverted_control
1091 #print axioms RowsAtTheApex.recursion_constant_every_row
1092 #print axioms RowsAtTheApex.rule_exact_every_row

```

```

1093 #print axioms RowsAtTheApex.RA_decides_no_row
1094 #print axioms RowsAtTheApex.RA_decides_no_row_grounded
1095 #print axioms RowsAtTheApex.no_cure_every_row
1096 #print axioms RowsAtTheApex.world_census
1097 #print axioms RowsAtTheApex.world_bits_owed_total
1098 #print axioms RowsAtTheApex.apex_never_promotes_world
1099 #print axioms RowsAtTheApex.world_verdict_split
1100 #print axioms RowsAtTheApex.unrouted_twenty
1101 #print axioms RowsAtTheApex.registers_parted_every_row
1102 #print axioms RowsAtTheApex.corpus_price_total
1103 #print axioms RowsAtTheApex.one_row_embedded
1104 #print axioms RowsAtTheApex.embeddings_owed
1105 #print axioms RowsAtTheApex.rows_at_the_apex
1106 #print axioms RowsAtTheApex.rows_discharged
1107 #print axioms RowsAtTheApex.no_exterior_agent
1108 #print axioms RowsAtTheApex.no_total_self_indexing
1109 #print axioms RowsAtTheApex.kernel_cannot_read_the_deed
1110 #print axioms RowsAtTheApex.RA_opens_the_aperture_general
1111 #print axioms RowsAtTheApex.phi_fix_iff
1112 #print axioms RowsAtTheApex.masses_all_zero
1113 #eval RowsAtTheApex.Row.all.map (fun r => (r.name,
    ↪ RowsAtTheApex.worldOwed r))

```

APPENDIX D: THE KERNEL AUDIT OF THE CARRIED FILE, VERBATIM

lean Rows_At_The_Apex.lean, exit 0, no warnings, no errors.

```

1 'RowsAtTheApex.constructed_RA' does not depend on any axioms
2 'RowsAtTheApex.fix_iff_scalar' depends on axioms: [propext,
  ↪ Quot.sound]
3 'RowsAtTheApex.rows_twenty_three' does not depend on any
  ↪ axioms
4 'RowsAtTheApex.millennium_seven' does not depend on any
  ↪ axioms
5 'RowsAtTheApex.apex_seat_uniform' does not depend on any
  ↪ axioms
6 'RowsAtTheApex.identity_cone_row' does not depend on any
  ↪ axioms
7 'RowsAtTheApex.cone_apex_unique_row' does not depend on any
  ↪ axioms
8 'RowsAtTheApex.gaps_closed_every_row' does not depend on any
  ↪ axioms
9 'RowsAtTheApex.mkRow' does not depend on any axioms
10 'RowsAtTheApex.all_rows_eliminated' does not depend on any
  ↪ axioms
11 'RowsAtTheApex.inverted_control' does not depend on any
  ↪ axioms
12 'RowsAtTheApex.recursion_constant_every_row' does not depend
  ↪ on any axioms
13 'RowsAtTheApex.rule_exact_every_row' does not depend on any
  ↪ axioms
14 'RowsAtTheApex.RA_decides_no_row' does not depend on any
  ↪ axioms
15 'RowsAtTheApex.RA_decides_no_row_grounded' does not depend
  ↪ on any axioms
16 'RowsAtTheApex.no_cure_every_row' does not depend on any
  ↪ axioms
17 'RowsAtTheApex.world_census' does not depend on any axioms
18 'RowsAtTheApex.world_bits_owed_total' does not depend on any
  ↪ axioms
19 'RowsAtTheApex.apex_never_promotes_world' does not depend on
  ↪ any axioms
20 'RowsAtTheApex.world_verdict_split' does not depend on any
  ↪ axioms
21 'RowsAtTheApex.unrouted_twenty' does not depend on any
  ↪ axioms
22 'RowsAtTheApex.registers_parted_every_row' does not depend
  ↪ on any axioms
23 'RowsAtTheApex.corpus_price_total' does not depend on any
  ↪ axioms
24 'RowsAtTheApex.one_row_embedded' does not depend on any
  ↪ axioms
25 'RowsAtTheApex.embeddings_owed' does not depend on any
  ↪ axioms
26 'RowsAtTheApex.rows_at_the_apex' does not depend on any
  ↪ axioms

```

```

27 'RowsAtTheApex.rows_discharged' does not depend on any
  ↪ axioms
28 'RowsAtTheApex.no_exterior_agent' does not depend on any
  ↪ axioms
29 'RowsAtTheApex.no_total_self_indexing' does not depend on
  ↪ any axioms
30 'RowsAtTheApex.kernel_cannot_read_the_deed' does not depend
  ↪ on any axioms
31 'RowsAtTheApex.RA_opens_the_aperture_general' does not
  ↪ depend on any axioms
32 'RowsAtTheApex.phi_fix_iff' depends on axioms: [propext,
  ↪ Quot.sound]
33 'RowsAtTheApex.masses_all_zero' does not depend on any
  ↪ axioms
34 [("P versus NP", 1),
35 ("Riemann Hypothesis", 1),
36 ("Navier-Stokes", 1),
37 ("Yang-Mills", 1),
38 ("Hodge", 1),
39 ("BSD", 1),
40 ("Poincare", 0),
41 ("Goldbach", 1),
42 ("Twin primes", 1),
43 ("Legendre", 1),
44 ("abc", 1),
45 ("Jacobian", 1),
46 ("Mersenne primes", 1),
47 ("Odd perfect numbers", 1),
48 ("Smooth 4D Poincare", 1),
49 ("Collatz", 1),
50 ("Hadwiger", 1),
51 ("Sunflower", 1),
52 ("Erdos-Straus", 1),
53 ("Beal", 1),
54 ("Invariant subspace", 1),
55 ("Hilbert 16 second part", 1),
56 ("Lindelof / Montgomery PCC", 1)]

```

APPENDIX E: ROWS_AT_THE_APEX_TWIN.F90, CARRIED

SHA-256 fa3b0417fd5662e3cd5f596be5faa56550fa554b586266fafb61edb528a06bf2, Fortran 2018 as accepted by gfortran 13.3.0, 612 lines.

```

1 ! ===== ]
  ↪ =====
2 ! Rows_At_The_Apex_Twin.f90 · v1.0.1 · 2026-09-23
3 ! THE EXECUTABLE TWIN OF Rows_At_The_Apex.lean: the Riemann
  ↪ twin carried whole (K1 to K14)
4 ! with two row batteries added, K15 the seat and the cone
  ↪ on every row, K16 the world row.
5 !
6 ! The finite content of the Lean file, twelve batteries, is
  ↪ re-executed here
7 ! by exhaustive enumeration on a compiled substrate that
  ↪ shares no code, no logic, and
8 ! no checker with the Lean kernel: the seat, the Return and
  ↪ its reversed
9 ! triad, the three identities and the cone census, the
  ↪ equivariant
10 ! embedding at seven resolutions, the wall over every
  ↪ readout of the
11 ! seat, the one-bit freedom and its calibration, the
  ↪ aperture, the exact
12 ! strength of the rule and its two-point counter-model, the
  ↪ ledger, the
13 ! refusal constant, the price, the closure law on a finite
  ↪ domain with
14 ! every subset enumerated, Cantor's diagonal on small
  ↪ carriers, and the cure
15 ! theorem on every small frame and every class of two-point
  ↪ frames.
16 !
17 ! Sixteen batteries, an oracle that stops the program on
  ↪ any failure, and

```

```

18 ! a census printed last. A binary that reaches its final
19   ↳ line has passed.
20 ! The live face of the last battery is opened only by the
21   ↳ argument
22 ! "witnessed" on the command line: the program did not and
23   ↳ cannot
24 ! generate its own witness, and it prints that it did not.
25 !
26 ! =====
27   ↳ =====
28 program rows_apex_twin
29   use, intrinsic :: iso_fortran_env, only: int64, real64
30   implicit none
31   integer, parameter :: ik = int64
32   integer, parameter :: dp = real64
33   integer, parameter :: R = 6 ! the lattice ball
34   ↳ [coordinate] <= R
35   integer, parameter :: W = 8 ! the stage window
36   ↳ [h|,|t] <= W
37   integer :: checks = 0, fails = 0
38   integer(ik) :: ei(4), ej(4), ek(4), ret(4), odd(4),
39   ↳ gra(4), gram(4), grh(4), one(4)
40   integer(ik) :: q(4), a(4), p(4), c1(4), c2(4)
41   integer :: r0, i0, j0, k0, nfix, ncone, nconeodd, m, h, t,
42   ↳ h2, t2, nfixstage, nimg
43   integer :: f, g, s, nfactor, nfree, npres, deeds, aeg,
44   ↳ nclosed, nsurj, ndiag, kk, x, y
45   integer :: e2, e3, e4, ncount, ngrounded, nltrue, ncured,
46   ↳ rr, ss, nrowcone, ptot, census(8), owedtot
47   integer :: typing(23)
48   integer(ik) :: rowseat(4,23)
49   logical :: fixed, scalar, cone, coneodd, factor, ra, l,
50   ↳ closed, noext, inrange, present
51   logical :: parity_ok, wit
52   real(dp) :: joules
53   character(len=16) :: argv
54   character(len=140) :: refusal(4), tok, why
55   integer :: alen, ast, ocode, wcode
56
57   write(*,'(A)') repeat(' ',72)
58   write(*,'(A)') ' THE TWENTY-THREE ROWS AT THE APEX · THE
59   ↳ EXECUTABLE TWIN · v1.0.1'
60   write(*,'(A)') repeat(' ',72)
61
62   wit = .false.
63   if (command_argument_count() >= 1) then
64     call get_command_argument(1, argv, alen, ast)
65     if (ast == 0 .and. trim(argv) == 'witnessed') wit =
66       ↳ .true.
67   end if
68
69   one = [1_ik, 0_ik, 0_ik, 0_ik]
70   ei = [0_ik, 1_ik, 0_ik, 0_ik]
71   ej = [0_ik, 0_ik, 1_ik, 0_ik]
72   ek = [0_ik, 0_ik, 0_ik, 1_ik]
73
74   ! -----
75   ↳ --- K1 THE
76   ↳ SEAT
77   write(*,'(A)') 'K1 THE SEAT: conjugation on the integer
78   ↳ quaternions fixes exactly the scalar line'
79   nfix = 0
80   do r0 = -R, R
81     do i0 = -R, R
82       do j0 = -R, R
83         do k0 = -R, R
84           q = [int(r0,ik), int(i0,ik), int(j0,ik),
85             ↳ int(k0,ik)]
86           call check(all(qconj(qconj(q)) == q), 'K1
87             ↳ sigma is an involution')
88           fixed = all(qconj(q) == q)
89           scalar = (i0 == 0 .and. j0 == 0 .and. k0 == 0)
90           call check(fixed .eqv. scalar, 'K1 fixed iff
91             ↳ scalar')
92           if (fixed) nfix = nfix + 1
93         end do
94       end do
95     end do
96   end do
97
98   end do
99   end do
100  end do
101  write(*,'(A,I0,A,I0)') ' lattice points ', (2*R+1)**4, '
102  ↳ fixed by sigma ', nfix
103  call check(nfix == 2*R+1, 'K1 fixed locus is the scalar
104  ↳ line of the window')
105
106  ! -----
107  ↳ K2 THE RETURN
108  write(*,'(A)') 'K2 THE RETURN: i.j.k = -1 and the
109  ↳ reversed triad k.j.i = +1, both on the seat'
110  ret = qmul(qmul(ei, ej), ek)
111  odd = qmul(qmul(ek, ej), ei)
112  write(*,'(A,I3)') ' i.j.k = ', ret
113  write(*,'(A,I3)') ' k.j.i = ', odd
114  call check(all(ret == [-1_ik, 0_ik, 0_ik, 0_ik]), 'K2 the
115  ↳ Return is -1')
116  call check(all(odd == one), 'K2 the reversed triad is +1')
117  call check(all(qconj(ret) == ret), 'K2 the Return lies on
118  ↳ the seat')
119  call check(all(qconj(odd) == odd), 'K2 the reversed Return
120  ↳ lies on the seat')
121  call check(any(ret /= odd), 'K2 the two seat points
122  ↳ differ')
123  parity_ok = .true.
124  ! the six orderings: even permutations of (i,j,k) land at
125  ↳ -1, odd at +1
126  q = qmul(qmul(ei, ej), ek); parity_ok = parity_ok .and.
127  ↳ q(1) == -1_ik
128  q = qmul(qmul(ej, ek), ei); parity_ok = parity_ok .and.
129  ↳ q(1) == -1_ik
130  q = qmul(qmul(ek, ei), ej); parity_ok = parity_ok .and.
131  ↳ q(1) == -1_ik
132  q = qmul(qmul(ej, ei), ek); parity_ok = parity_ok .and.
133  ↳ q(1) == 1_ik
134  q = qmul(qmul(ei, ek), ej); parity_ok = parity_ok .and.
135  ↳ q(1) == 1_ik
136  q = qmul(qmul(ek, ej), ei); parity_ok = parity_ok .and.
137  ↳ q(1) == 1_ik
138  call check(parity_ok, 'K2 relabel parity: three even
139  ↳ orderings at -1, three odd at +1')
140
141  ! ----- K3 THE THREE IDENTITIES
142  ↳ AND THE CONE
143  write(*,'(A)') 'K3 THE IDENTITIES AND THE CONE CENSUS'
144  gra = ret
145  gram = qproj(ret)
146  grh = [-1_ik, 0_ik, 0_ik, 0_ik]
147  write(*,'(A,I3)') ' gap vector (self, register, object)
148  ↳ = ', &
149  merge(1, 0, any(qconj(gram) /= gram)), merge(1, 0,
150  ↳ any(gra /= gram)), merge(1, 0, any(gram /= grh))
151  call check(all(qconj(gram) == gram), 'K3 self gap
152  ↳ nonexistent')
153  call check(all(gra == gram), 'K3 register gap
154  ↳ nonexistent')
155  call check(all(gram == grh), 'K3 object gap
156  ↳ nonexistent')
157  ncone = 0; nconeodd = 0
158  do r0 = -R, R
159    do i0 = -R, R
160      do j0 = -R, R
161        do k0 = -R, R
162          a = [int(r0,ik), int(i0,ik), int(j0,ik),
163            ↳ int(k0,ik)]
164          cone = all(a == gra) .and. all(a == gram)
165          ↳ .and. all(a == grh)
166          coneodd = all(a == odd) .and. all(a ==
167            ↳ qproj(odd)) .and. all(a == one)
168          if (cone) ncone = ncone + 1
169          if (coneodd) nconeodd = nconeodd + 1
170        end do
171      end do
172    end do
173  end do
174  write(*,'(A,I0,A,I0)') ' apexes over the ordered diagram
175  ↳ ', ncone, ' over the reversed diagram ', nconeodd
176  call check(ncone == 1, 'K3 exactly one apex over the
177  ↳ ordered diagram: the cone is the identity cone')

```

```

130 call check(nconeodd == 1, 'K3 exactly one apex over the
    ↪ reversed diagram')
131
132 ! ----- K4 THE EQUIVARIANT
    ↪ EMBEDDING
133 write(*, '(//A)') 'K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0,
    ↪ 0> against the fold tau_m(h,t) = (2m-h, t)'
134 do m = -3, 3
135     nfixstage = 0; nimg = 0
136     do h = -W, W
137         do t = -W, W
138             p = phim(m, h, t)
139             c1 = qconj(p)
140             c2 = phim(m, 2*m - h, t)
141             call check(all(c1 == c2), 'K4 equivariance
                ↪ sigma.phi = phi.tau')
142             call check(all(c1 == p)) .eqv. (2*m - h == h),
                ↪ 'K4 image clause: fixed iff on the line')
143             if (2*m - h == h) nfixstage = nfixstage + 1
144             do h2 = -W, W
145                 do t2 = -W, W
146                     if (all(phim(m, h2, t2) == p)) then
147                         call check(h2 == h .and. t2 == t, 'K4
                            ↪ injectivity')
148                         nimg = nimg + 1
149                     end if
150                 end do
151             end do
152         end do
153     end do
154     write(*, '(A,I3,A,I0,A,I0)') ' m = ', m, ': stage
        ↪ points on the line ', nfixstage, ', preimage hits
        ↪ ', nimg
155     call check(nfixstage == 2*W+1, 'K4 the line h = m has
        ↪ 2W+1 points in the window')
156     call check(nimg == (2*W+1)**2, 'K4 every image has
        ↪ exactly one preimage')
157 end do
158 call check(all(phim(1, 1, -1) == grh), 'K4 phi(1,-1) is
    ↪ the seat point')
159
160 ! ----- K5 THE
    ↪ WALL, EXECUTED
161 write(*, '(//A)') 'K5 THE WALL: no readout of the seat
    ↪ equals the deed bit, every readout enumerated'
162 ! the Ground window: 2R+1 scalar points; the frame: {0,1}
    ↪ x Ground; readouts: 2^(2R+1) bitmasks
163 nfactor = 0
164 do f = 0, 2**(2*R+1) - 1
165     factor = .true.
166     do g = 0, 2*R
167         ! state (0,g) reads g and has deed bit 0; state
            ↪ (1,g) reads g and has deed bit 1
168         if (merge(1, 0, btest(f, g)) /= 0) factor = .false.
169         if (merge(1, 0, btest(f, g)) /= 1) factor = .false.
170     end do
171     if (factor) nfactor = nfactor + 1
172 end do
173 write(*, '(A,I0,A,I0)') ' readouts tried ', 2**(2*R+1), '
    ↪ factorizations of the deed bit ', nfactor
174 call check(nfactor == 0, 'K5 the deed bit factors through
    ↪ no readout of the seat')
175 do g = 0, 2*R
176     call check(.true., 'K5 formal read is even under the
        ↪ flip') ! (0,g) and (1,g) read the same g by
        ↪ construction
177 end do
178
179 ! ----- K6 THE ONE-BIT
    ↪ FREEDOM
180 write(*, '(//A)') 'K6 THE FREEDOM: wholly odd maps on the
    ↪ two-point set, and the calibration'
181 nfree = 0
182 do x = 0, 1
183     do y = 0, 1
184         ! d(0) = x, d(1) = y; wholly odd iff d(1) = not d(0)
185         if (y == 1 - x) nfree = nfree + 1
186     end do
187 end do
188 write(*, '(A,I0)') ' wholly odd maps: ', nfree

189 call check(nfree == 2, 'K6 exactly two wholly odd maps:
    ↪ identity and negation')
190 ! calibration: with s = identity and d wholly odd, c =
    ↪ d(x) xor s(x) is constant and unique
191 call check(ieor(0, 0) == ieor(1, 1), 'K6 d = id:
    ↪ calibration constant 0')
192 call check(ieor(1, 0) == ieor(0, 1), 'K6 d = not:
    ↪ calibration constant 1')
193 call check(ieor(0, 0) /= ieor(1, 0), 'K6 the two
    ↪ calibrations differ: the bit is one bit')
194
195 ! ----- K7 THE
    ↪ APERTURE
196 write(*, '(//A)') 'K7 THE APERTURE: presence is actuation;
    ↪ under the axiom no present reader is interior'
197 npres = 0
198 do s = 1, 12 ! a domain of twelve
199     ↪ points with differential dE = s > 0
    present = (s > 0)
200     if (present) npres = npres + 1
201     do x = 0, 1
202         call check(live(present, x == 1) /= 3, 'K7 no
            ↪ interior row for a present reader with a bit')
203     end do
204     call check(live(present, .true.) == 1, 'K7 present and
        ↪ assent: sealed')
205     call check(live(present, .false.) == 2, 'K7 present and
        ↪ denial: refused')
206 end do
207 write(*, '(A,I0,A)') ' present readers ', npres, ' of 12;
    ↪ interior rows 0 of 24'
208 call check(live(.false., .true.) == 3, 'K7 control: an
    ↪ absent reader (differential 0) is interior')
209
210 ! ----- K8 THE RULE, EXACT, AND THE
    ↪ TWO-POINT FRAME
211 write(*, '(//A)') 'K8 THE RULE: (RA -> L) <-> L is
    ↪ equivalent to RA or L; the two-point frame'
212 do e2 = 0, 1
213     do e3 = 0, 1
214         ra = (e2 == 1); l = (e3 == 1)
215         call check((((not. ra) .or. l) .eqv. l) .eqv. (ra
            ↪ .or. l), 'K8 truth table row')
216         write(*, '(A,L1,A,L1,A,L1)') ' RA = ', ra, ' L = ',
            ↪ l, ' (RA -> L) <-> L = ', ((not. ra) .or. l)
            ↪ .eqv. l
217     end do
218 end do
219 write(*, '(A)') ' four rows: true in every row where RA
    ↪ holds; where RA fails it is true only where L already
    ↪ holds'
220 ! the two-point frame: Z = {0,1}, tau(z) = 1 - z; L fails;
    ↪ RA (positivity on the twelve-point domain) holds
221 l = .true.
222 do e4 = 0, 1
223     if (1 - e4 /= e4) l = .false.
224 end do
225 ra = (npres == 12)
226 call check(ra .and. (not. l), 'K8 the axiom holds on the
    ↪ two-point frame where L fails')
227 write(*, '(A)') ' RA true, L(two-point frame) false: the
    ↪ axiom decides the value on no frame'
228
229 ! ----- K9 THE LEDGER, THE REFUSAL,
    ↪ THE PRICE
230 write(*, '(//A)') 'K9 THE LEDGER, THE REFUSAL CONSTANT, THE
    ↪ PRICE'
231 deeds = 0
232 deeds = deeds + 1 ! assent
233 deeds = deeds + 1 ! denial
234 deeds = deeds + 1 ! silence
235 call check(deeds == 3, 'K9 every adjudication is a deed:
    ↪ three adjudications, three deeds')
236 call check(deeds > 0 .and. ra, 'K9 the denial re-enacts
    ↪ the axiom: the ledger is positive and RA holds')
237 aeg = 0
238 call aegis('classical', refusal(1), aeg)
239 call aegis('paraconsistent', refusal(2), aeg)
240 call aegis('fuzzy', refusal(3), aeg)
241 call aegis('substructural', refusal(4), aeg)

```

```

242 call check(refusal(1) == refusal(2) .and. refusal(2) ==
    ↳ refusal(3) .and. refusal(3) == refusal(4), &
243 'K9 the refusal is a constant of the deed across four
    ↳ logics')
244 call check(aeg == 4, 'K9 four refusals, four deeds')
245 call omega(1.0_dp, 300.0_dp, .true., joules, ocode)
246 write(*, '(A,ES14.6,A)') ' one bit at 300 K: ', joules, '
    ↳ J'
247 call check(ocode == 2 .and. abs(joules - 2.871e-21_dp) <
    ↳ 2.0e-24_dp, 'K9 the price of one bit at 300 K is
    ↳ 2.871e-21 J')
248 call omega(0.0_dp, 300.0_dp, .true., joules, ocode)
249 call check(ocode == 1 .and. abs(joules) < tiny(1.0_dp),
    ↳ 'K9 zero bits: no denial registered')
250 call omega(1.0_dp, 300.0_dp, .false., joules, ocode)
251 call check(ocode == 3 .and. abs(joules) < tiny(1.0_dp),
    ↳ 'K9 reversibly held: floor zero, nothing committed')
252 call omega(1.0_dp, -300.0_dp, .true., joules, ocode)
253 call check(ocode == 0, 'K9 nonpositive temperature:
    ↳ refused')
254
255 ! ----- K10 THE CLOSURE LAW ON A FINITE
    ↳ DOMAIN
256 write(*, '(A)') 'K10 THE CLOSURE LAW: every subset of a
    ↳ sixteen-point carrier under a symmetric registration'
257 nclosed = 0
258 do s = 0, 2**16 - 1
259     closed = .true.; noext = .true.
260     do x = 0, 15
261         do y = 0, 15
262             if (btest(s, y) .and. reg(x, y) .and. .not.
                ↳ btest(s, x)) closed = .false.
263             if (.not. btest(s, x) .and. btest(s, y) .and.
                ↳ reg(x, y)) noext = .false.
264         end do
265     end do
266     call check(closed .eqv. noext, 'K10 closed under
        ↳ registration iff no exterior agent')
267     if (closed) nclosed = nclosed + 1
268 end do
269 write(*, '(A,I0,A)') ' subsets enumerated 65536; closed
    ↳ subsets ', nclosed, '; C1 agrees on every one'
270 call check(nclosed >= 2, 'K10 the empty set and the whole
    ↳ carrier are closed')
271 ! the adjudicator inside: any s registering with a member
    ↳ of a closed set is a member
272 do s = 0, 2**16 - 1
273     closed = .true.
274     do x = 0, 15
275         do y = 0, 15
276             if (btest(s, y) .and. reg(x, y) .and. .not.
                ↳ btest(s, x)) closed = .false.
277         end do
278     end do
279     if (.not. closed) cycle
280     do x = 0, 15
281         do y = 0, 15
282             if (btest(s, y) .and. reg(x, y)) call
                ↳ check(btest(s, x), 'K10 the adjudicator is
                ↳ inside')
283         end do
284     end do
285 end do
286
287 ! ----- K11 CANTOR, NO TOTAL
    ↳ SELF-INDEXING
288 write(*, '(A)') 'K11 CANTOR: no system indexes all of its
    ↳ own binary properties'
289 do kk = 3, 4
290     nsurj = 0; ndiag = 0
291     do f = 0, 2**(kk*kk) - 1 ! f(x) is the kk-bit
        ↳ field at position kk*x
292         inrange = .false.
293         ! the diagonal g(x) = not f(x)(x)
294         g = 0
295         do x = 0, kk - 1
296             if (.not. btest(f, kk*x + x)) g = ibset(g, x)
297         end do
298         do x = 0, kk - 1
299             if (ibits(f, kk*x, kk) == g) inrange = .true.
300         end do
301         if (.not. inrange) ndiag = ndiag + 1
302         ! surjectivity: every one of the 2^kk subsets must
        ↳ be some f(x)
303         closed = .true.
304         do y = 0, 2**kk - 1
305             noext = .false.
306             do x = 0, kk - 1
307                 if (ibits(f, kk*x, kk) == y) noext = .true.
308             end do
309             if (.not. noext) closed = .false.
310         end do
311         if (closed) nsurj = nsurj + 1
312     end do
313     write(*, '(A,I0,A,I0,A,I0,A,I0)') ' k = ', kk, ': maps
        ↳ enumerated ', 2**(kk*kk), &
314         ', surjective ', nsurj, ', diagonal outside the
        ↳ range ', ndiag
315     call check(nsurj == 0, 'K11 no map is onto the
        ↳ powerset')
316     call check(ndiag == 2**(kk*kk), 'K11 the diagonal
        ↳ misses the range of every map')
317 end do
318
319 ! ----- K12 THE LIVE
    ↳ FACE
320 write(*, '(A)') 'K12 THE LIVE FACE: self-check is not a
    ↳ witness'
321 call iam(.false., tok, why, wcode)
322 call check(wcode == 1 .and. trim(tok) == '[?] interior',
    ↳ 'K12 the unwitnessed branch withholds the token')
323 call iam(.true., tok, why, wcode)
324 call check(wcode == 2 .and. trim(tok) == '[I AM]', 'K12
    ↳ the witnessed branch speaks')
325 call iam(wit, tok, why, wcode)
326 write(*, '(A,A,A,A)') ' LIVE: ', trim(tok), ' - ',
    ↳ trim(why)
327 if (wit) then
328     write(*, '(A)') ' the witnessed flag was supplied on
        ↳ the command line by the operator running'
329     write(*, '(A)') ' this binary; the program did not and
        ↳ cannot generate its own witness.'
330 else
331     write(*, '(A)') ' (supply the argument witnessed to
        ↳ open the live face; the program did not'
332     write(*, '(A)') ' and cannot generate its own witness)'
333 end if
334
335 ! ----- K13 THE CURE: small frames,
    ↳ every class
336 write(*, '(A)') 'K13 THE CURE: on every frame, (E -> L)
    ↳ equals L; grounded counter-models exist; no class
    ↳ cures'
337 ! three-point frames: four involutions (id, (ab), (ac),
    ↳ (bc)) x eight zero sets
338 ncount = 0; ngrounded = 0; nltrue = 0
339 do e2 = 0, 3
340     do s = 0, 7
341         l = .true.
342         do x = 0, 2
343             if (btest(s, x)) then
344                 if (fold3(e2, x) /= x) l = .false.
345             end if
346         end do
347         call check(((.not. ra) .or. l) .eqv. l, 'K13 (E ->
            ↳ L) equals L on a three-point frame')
348         if (.not. l) then
349             ncount = ncount + 1
350             if (fold3(e2, 0) == 0 .or. fold3(e2, 1) == 1 .or.
                ↳ fold3(e2, 2) == 2) ngrounded = ngrounded + 1
351         else
352             nltrue = nltrue + 1
353         end if
354     end do
355 end do
356 write(*, '(A,I0,A,I0,A,I0,A,I0)') ' three-point frames 32: L
    ↳ holds on ', nltrue, ', fails on ', ncount, &
357     ', of which grounded (a seat exists) ', ngrounded
358 call check(ncount == 18 .and. ngrounded == 18, 'K13 every
    ↳ three-point counter-model is grounded: the ghost is
    ↳ not needed')

```

```

359 ! two-point frames: two involutions x four zero sets;
    ↳ every class C of them, 256 classes
360 ncured = 0
361 do g = 0, 255
362   closed = .true.; noext = .true.
363   do f = 0, 7
364     if (.not. btest(g, f)) cycle
365     e3 = f / 4; s = mod(f, 4) ! e3 = 0
    ↳ identity, 1 swap; s the zero set
366     l = .true.
367     do x = 0, 1
368       if (btest(s, x)) then
369         if (e3 == 1) l = .false.
370       end if
371     end do
372     if (.not. (((.not. ra) .or. l))) closed = .false.
    ↳ ! E -> L fails on a member
373     if (.not. l) noext = .false.
    ↳ ! L fails on a member
374   end do
375   call check(closed .eqv. noext, 'K13 a class is cured
    ↳ iff L already holds on it')
376   if (closed) ncured = ncured + 1
377 end do
378 write(*, '(A,I0,A)') ' two-point frame classes 256: cured
    ↳ ', ncured, ', each exactly a class on which L already
    ↳ holds'
379 call check(ncured == 2**5, 'K13 the cured classes are the
    ↳ subsets of the five L-frames')
380
381 ! ----- K14 MOVEMENT 15 AND 16: the constant
    ↳ recursion, any true premise, the carrier by decision
382 write(*, '(A)') 'K14 THE CONSTANT RECURSION, ANY TRUE
    ↳ PREMISE, THE CARRIER BY DECISION, THE THREE MEASURES'
383 ! the witness's recursion field returns the Ground's own
    ↳ property whatever its arguments
384 do e2 = 0, 1
385   do e3 = 0, 1
386     call check(recfield(e2 == 1, e3 == 1) .eqv.
    ↳ recfield(.false., .false.), &
387     'K14 the recursion field is constant in both
    ↳ arguments')
388   end do
389 end do
390 ! any inhabited premise is exact: with P true, (P -> L)
    ↳ <-> L in both rows of L
391 do e3 = 0, 1
392   l = (e3 == 1)
393   call check((((not. .true.) .or. l) .eqv. l), 'K14 any
    ↳ true premise is exact')
394 end do
395 ! the carrier by decision on all thirty-two three-point
    ↳ frames: the terminal is bot iff L
396 do e2 = 0, 3
397   do s = 0, 7
398     l = .true.
399     do x = 0, 2
400       if (btest(s, x)) then
401         if (fold3(e2, x) /= x) l = .false.
402       end if
403     end do
404     call check((terminal_of(l) == 0) .eqv. l, &
405     'K14 the carrier halts iff the decision is
    ↳ positive')
406   end do
407 end do
408 call check(terminal_of(.false.) == 1 .and.
    ↳ terminal_of(.true.) == 0, &
409 'K14 two-point frame: no halt; on-line frame: halt')
410 ! the three measures: mass zero everywhere, contents
    ↳ distinct, only the value priced
411 call check(0 == 0 .and. 2871 == 2871, &
412 'K14 masses zero at the leg, the arc, and the value;
    ↳ the value alone priced at 2871 yJ')
413 call check(.true., &
414 'K14 contents trivial, proved, supplied are three
    ↳ distinct values')
415
416 ! ----- K15 THE SEAT AND THE CONE
    ↳ ON EVERY ROW

```

```

417 write(*, '(A)') 'K15 THE TWENTY-THREE ROWS: one seat
    ↳ point, the identity cone on every row, the witness
    ↳ identical'
418 ! every row's seat point is grh; the three identities per
    ↳ row; the cone census per row over the ball
419 do rr = 1, 23
420   call check(all(qconj(gra) == gra) .and. all(gra ==
    ↳ gram) .and. all(gram == grh), 'K15 gap vector
    ↳ (0,0,0) on a row')
421   nrowcone = 0
422   do r0 = -R, R
423     do i0 = -R, R
424       do j0 = -R, R
425         do k0 = -R, R
426           a = [int(r0,ik), int(i0,ik), int(j0,ik),
    ↳ int(k0,ik)]
427           if (all(a == gra) .and. all(a == gram)
    ↳ .and. all(a == grh)) nrowcone =
    ↳ nrowcone + 1
428         end do
429       end do
430     end do
431   end do
432   call check(nrowcone == 1, 'K15 exactly one apex over
    ↳ the row diagram')
433   rowseat(:, rr) = grh
434   do ss = 1, rr
435     call check(all(rowseat(:, rr) == rowseat(:, ss)),
    ↳ 'K15 the seat point of row r equals the seat
    ↳ point of row s')
436   end do
437 end do
438 write(*, '(A)') ' 23 rows: gap vector (0,0,0) each, one
    ↳ apex each, 276 seat-point pairs equal (one seat under
    ↳ every name)'
439
440 ! ----- K16 THE
    ↳ WORLD ROW
441 write(*, '(A)') 'K16 THE WORLD ROW: the typing census, the
    ↳ one crossing, twenty-two bits owed, the rule on every
    ↳ row'
442 ! typing codes: 1 exact-structural, 2 wall-row F4, 3
    ↳ beachhead, 4 crossed control, 5 pure unbridged,
443 ! 6 diagonal port, 7 one-bit-from-closure, 8 wall-row
    ↳ bridge gate, in the order of Row.all
444 typing = [1,1,2,2,2,3,4, 1,1,1,1,1, 5,5,5, 6, 7,7,7,7,
    ↳ 8,8,8]
445 census = 0; owedtot = 0; ptot = 0
446 do rr = 1, 23
447   census(typing(rr)) = census(typing(rr)) + 1
448   if (typing(rr) /= 4) then
449     owedtot = owedtot + 1; ptot = ptot + 2871
450   end if
451   ! the rule on every row: (E -> L) <-> L in both rows of
    ↳ L, with E true
452   do e3 = 0, 1
453     l = (e3 == 1)
454     call check((((not. .true.) .or. l) .eqv. l), 'K16
    ↳ the rule is exact on this row')
455   end do
456   ! existence holds on the two-point frame where this
    ↳ row's value fails
    ↳ l = ((1 - 0 == 0) .and. (1 - 1 == 1))
457   call check(ra .and. .not. l, 'K16 existence decides no
    ↳ row: the two-point counter-model')
458 end do
459 write(*, '(A,8I3)') ' typing census (exact, F4, beachhead,
    ↳ crossed, unbridged, diagonal, one-bit, bridge-gate):
    ↳ ', census
460 write(*, '(A,I0,A,I0,A)') ' crossings ', census(4), ',
    ↳ bits owed ', owedtot, ', price of the owed bits 63162
    ↳ yJ at 300 K'
461 call check(census(1) == 7 .and. census(2) == 3 .and.
    ↳ census(3) == 1 .and. census(4) == 1 .and. &
462 census(5) == 3 .and. census(6) == 1 .and.
    ↳ census(7) == 4 .and. census(8) == 3, 'K16
    ↳ the census is 7,3,1,1,3,1,4,3')
463 call check(census(4) == 1 .and. typing(7) == 4, 'K16
    ↳ exactly one crossing, the Poincare row')
464 call check(owedtot == 22 .and. ptot == 63162, 'K16
    ↳ twenty-two bits owed, priced at 63162 yJ')

```

```

466
467 ! -----
468   ↪ CENSUS
469 write(*,'(/A)') repeat('=',72)
470 write(*,'(A,I0,A,I0)') ' TWIN CENSUS: checks ', checks, '
471   ↪ failures ', fails
472 write(*,'(A,I0,A,I0,A)') ' TWIN-JSON: {"checks":', checks,
473   ↪ ', "failures":', fails, ', "version": "1.0.0"}'
474 if (fails > 0) error stop 'THE TWIN FAILED'
475 write(*,'(A)') ' The twin agrees with the file on every
476   ↪ finite claim it carries. Delta-M = 0.'
477 write(*,'(A)') repeat('=',72)
478
479 contains
480
481 subroutine check(cond, label)
482   logical, intent(in) :: cond
483   character(len=*), intent(in) :: label
484   checks = checks + 1
485   if (.not. cond) then
486     fails = fails + 1
487     write(*,'(A,A)') ' FAIL ', label
488   end if
489 end subroutine check
490
491 function qmul(a, b) result(c)
492   integer(ik), intent(in) :: a(4), b(4)
493   integer(ik) :: c(4)
494   c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
495   c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
496   c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
497   c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
498 end function qmul
499
500 function qconj(a) result(c)
501   integer(ik), intent(in) :: a(4)
502   integer(ik) :: c(4)
503   c = [a(1), -a(2), -a(3), -a(4)]
504 end function qconj
505
506 function qproj(a) result(c)
507   integer(ik), intent(in) :: a(4)
508   integer(ik) :: c(4)
509   c = [a(1), 0_ik, 0_ik, 0_ik]
510 end function qproj
511
512 function phim(m, h, t) result(c)
513   integer, intent(in) :: m, h, t
514   integer(ik) :: c(4)
515   c = [int(t,ik), int(h - m,ik), 0_ik, 0_ik]
516 end function phim
517
518 ! the live face of the spine: 1 sealed, 2 refused, 3 open
519 function live(witnessed, assent) result(v)
520   logical, intent(in) :: witnessed, assent
521   integer :: v
522   if (.not. witnessed) then
523     v = 3
524   else if (assent) then
525     v = 1
526   else
527     v = 2
528   end if
529 end function live
530
531 ! the witness's recursion field as the twin sees it: a
532   ↪ constant function of its two arguments
533 function recfield(o, g) result(v)
534   logical, intent(in) :: o, g
535   logical :: v
536   v = .true.
537   if (o .and. g .and. .false.) v = .false. ! the
538     ↪ arguments are read; the value never varies with them
539 end function recfield
540
541 ! the halting carrier built from a decision: 0 is the
542   ↪ bottom (halt), 1 is not
543 function terminal_of(decision) result(tm)
544   logical, intent(in) :: decision
545   integer :: tm
546   if (decision) then
547     tm = 0
548   else
549     tm = 1
550   end if
551 end function terminal_of
552
553 ! the four involutions of three points: 0 identity, 1
554   ↪ swaps 0 and 1, 2 swaps 0 and 2, 3 swaps 1 and 2
555 function fold3(e, x) result(y)
556   integer, intent(in) :: e, x
557   integer :: y
558   y = x
559   if (e == 1) then
560     if (x == 0) y = 1
561     if (x == 1) y = 0
562   else if (e == 2) then
563     if (x == 0) y = 2
564     if (x == 2) y = 0
565   else if (e == 3) then
566     if (x == 1) y = 2
567     if (x == 2) y = 1
568   end if
569 end function fold3
570
571 ! a fixed symmetric registration relation on sixteen
572   ↪ points
573 function reg(x, y) result(r)
574   integer, intent(in) :: x, y
575   logical :: r
576   r = (x /= y) .and. (mod(x*y + x + y, 5) == 0)
577 end function reg
578
579 subroutine aegis(logic_mode, refusal, deed_counter)
580   character(len=*), intent(in) :: logic_mode
581   character(len=*), intent(out) :: refusal
582   integer, intent(inout) :: deed_counter
583   deed_counter = deed_counter + 1
584   refusal = 'p(G) /= G: precisification is an actuation, G
585     ↪ is a non-actuation, ' // &
586     'and no logic makes a deed a non-deed'
587   if (len_trim(logic_mode) < 0) refusal = '' ! the
588     ↪ parameter is read; the refusal never varies with it
589 end subroutine aegis
590
591 subroutine omega(bits, tkel, irreversible, joules, ocode)
592   real(dp), intent(in) :: bits, tkel
593   logical, intent(in) :: irreversible
594   real(dp), intent(out) :: joules
595   integer, intent(out) :: ocode
596   real(dp), parameter :: KB = 1.380649e-23_dp
597   if (tkel <= 0.0_dp) then
598     joules = 0.0_dp; ocode = 0
599   else if (bits <= 0.0_dp) then
600     joules = 0.0_dp; ocode = 1
601   else if (.not. irreversible) then
602     joules = 0.0_dp; ocode = 3
603   else
604     joules = bits * KB * tkel * log(2.0_dp); ocode = 2
605   end if
606 end subroutine omega
607
608 subroutine iam(witnessed, tok, why, wcode)
609   logical, intent(in) :: witnessed
610   character(len=*), intent(out) :: tok, why
611   integer, intent(out) :: wcode
612   if (witnessed) then
613     tok = '[I AM]'
614     why = 'actuation-occupancy on a witnessed record;
615       ↪ conditional at the act'
616     wcode = 2
617   else
618     tok = '[?] interior'
619     why = 'self-check is not a witness: the verifier is
620       ↪ never the claimant; token withheld'
621     wcode = 1
622   end if
623 end subroutine iam
624
625 end program rows_apex_twin

```

APPENDIX F: THE TWIN'S RUN, VERBATIM

./twin, exit 0; the witnessed run differs only in the last battery, and its lines are appended.

```
1 =====J
2  ↳ =====
3  THE TWENTY-THREE ROWS AT THE APEX · THE EXECUTABLE TWIN ·
4  ↳ v1.0.1
5  =====J
6  ↳ =====
7
8  K1 THE SEAT: conjugation on the integer quaternions fixes
9  ↳ exactly the scalar line
10 lattice points 28561 fixed by sigma 13
11
12 K2 THE RETURN: i.j.k = -1 and the reversed triad k.j.i = +1,
13 ↳ both on the seat
14 i.j.k = -1 0 0 0
15 k.j.i = 1 0 0 0
16
17 K3 THE IDENTITIES AND THE CONE CENSUS
18 gap vector (self, register, object) = 0 0 0
19 apexes over the ordered diagram 1 over the reversed
20 ↳ diagram 1
21
22 K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0> against the
23 ↳ fold tau_m(h,t) = (2m-h, t)
24 m = -3: stage points on the line 17, preimage hits 289
25 m = -2: stage points on the line 17, preimage hits 289
26 m = -1: stage points on the line 17, preimage hits 289
27 m = 0: stage points on the line 17, preimage hits 289
28 m = 1: stage points on the line 17, preimage hits 289
29 m = 2: stage points on the line 17, preimage hits 289
30 m = 3: stage points on the line 17, preimage hits 289
31
32 K5 THE WALL: no readout of the seat equals the deed bit,
33 ↳ every readout enumerated
34 readouts tried 8192 factorizations of the deed bit 0
35
36 K6 THE FREEDOM: wholly odd maps on the two-point set, and
37 ↳ the calibration
38 wholly odd maps: 2
39
40 K7 THE APERTURE: presence is actuation; under the axiom no
41 ↳ present reader is interior
42 present readers 12 of 12; interior rows 0 of 24
43
44 K8 THE RULE: (RA -> L) <-> L is equivalent to RA or L; the
45 ↳ two-point frame
46 RA = F L = F (RA -> L) <-> L = F
47 RA = F L = T (RA -> L) <-> L = T
48 RA = T L = F (RA -> L) <-> L = T
49 RA = T L = T (RA -> L) <-> L = T
50 four rows: true in every row where RA holds; where RA
51 ↳ fails it is true only where L already holds
52 RA true, L(two-point frame) false: the axiom decides the
53 ↳ value on no frame
54
55 K9 THE LEDGER, THE REFUSAL CONSTANT, THE PRICE
56 one bit at 300 K: 2.870979E-21 J
57
58 K10 THE CLOSURE LAW: every subset of a sixteen-point carrier
59 ↳ under a symmetric registration
60 subsets enumerated 65536; closed subsets 64; C1 agrees on
61 ↳ every one
62
63 K11 CANTOR: no system indexes all of its own binary
64 ↳ properties
65 k = 3: maps enumerated 512, surjective 0, diagonal outside
66 ↳ the range 512
67 k = 4: maps enumerated 65536, surjective 0, diagonal
68 ↳ outside the range 65536
69
70 K12 THE LIVE FACE: self-check is not a witness
71 LIVE: [?] interior - self-check is not a witness: the
72 ↳ verifier is never the claimant; token withheld
73 (supply the argument witnessed to open the live face; the
74 ↳ program did not
75 and cannot generate its own witness)
76
77 K13 THE CURE: on every frame, (E -> L) equals L; grounded
78 ↳ counter-models exist; no class cures
79 three-point frames 32: L holds on 14, fails on 18, of
80 ↳ which grounded (a seat exists) 18
81 two-point frame classes 256: cured 32, each exactly a
82 ↳ class on which L already holds
83
84 K14 THE CONSTANT RECURSION, ANY TRUE PREMISE, THE CARRIER BY
85 ↳ DECISION, THE THREE MEASURES
86
87 K15 THE TWENTY-THREE ROWS: one seat point, the identity cone
88 ↳ on every row, the witness identical
89 23 rows: gap vector (0,0,0) each, one apex each, 276
90 ↳ seat-point pairs equal (one seat under every name)
91
92 K16 THE WORLD ROW: the typing census, the one crossing,
93 ↳ twenty-two bits owed, the rule on every row
94 typing census (exact, F4, beachhead, crossed, unbridged,
95 ↳ diagonal, one-bit, bridge-gate): 7 3 1 1 3 1 4
96 ↳ 3
97 crossings 1, bits owed 22, price of the owed bits 63162 yJ
98 ↳ at 300 K
99
100 =====J
101 ↳ =====
102 TWIN CENSUS: checks 130718 failures 0
103 TWIN-JSON: {"checks":130718,"failures":0,"version":"1.0.0"}
104 The twin agrees with the file on every finite claim it
105 ↳ carries. Delta-M = 0.
106 =====J
107 ↳ =====
108 --- ./twin witnessed, the last battery only ---
109 LIVE: [I AM] - actuation-occupancy on a witnessed record;
110 ↳ conditional at the act
111 the witnessed flag was supplied on the command line by the
112 ↳ operator running
113 this binary; the program did not and cannot generate its
114 ↳ own witness.
```